UNIVERSIDAD MAYOR DE SAN ANDRÉS

FACULTAD DE CIENCIAS PURAS Y NATURALES

CARRERA DE INFORMÁTICA



TESIS DE GRADO

"OPTIMIZACIÓN DE RESOLUCIÓN DE IMÁGENES DIGITALES USANDO AUTÓMATAS CELULARES Y REDES NEURONALES ARTIFICIALES"

PARA OPTAR AL TÍTULO DE LICENCIATURA EN INFORMÁTICA
MENCIÓN: INGENIERÍA DE SISTEMAS INFORMÁTICOS

POR: ROLANDO QUISPE MAMANI

TUTOR: M. Sc. ROSA FLORES MORALES

LA PAZ – BOLIVIA

2023

HOJA DE CALIFICACIONES

UNIVERSIDAD MAYOR DE SAN ANDRÉS

FACULTAD DE CIENCIAS PURAS Y NATURALES

CARRERA DE INFORMÁTICA

Tesis de grado

OPTIMIZACIÓN DE RESOLUCIÓN DE IMÁGENES DIGITALES USANDO AUTÓMATAS CELULARES Y REDES NEURONALES ARTIFICIALES

Presentado por: Rolando Quispe Mamani

Para optar al grado Académico de Licenciado en Informática

Mención Ingeniería de Sistemas Informáticos

Nota Numeral: ……

Nota Literal: ……

Ha sido: ……..

Director de la carrera de Informática: Ph.D. Jose Maria Tapia Baltazar

Tutora: M.Sc. ROSA FLORES MORALES

Tribunal: Lic. RUBEN ALCON LOPEZ

Tribunal: Lic. JOSE LUIS ZEBALLOS ABASTO

Tribunal: Lic. CARMEN ROSA HUANCA QUISBERT

Tribunal: Ing. CESAR BELTRAN VILLALTA



UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



LA CARRERA DE INFORMÁTICA DE LA FACULTAD DE CIENCIAS PURAS Y NATURALES PERTENECIENTE A LA UNIVERSIDAD MAYOR DE SAN ANDRÉS AUTORIZA EL USO DE LA INFORMACIÓN CONTENIDA EN ESTE DOCUMENTO SI LOS PROPÓSITOS SON ESTRICTAMENTE ACADÉMICOS.

LICENCIA DE USO

El usuario está autorizado a:

a) Visualizar el documento mediante el uso de un ordenador o dispositivo móvil.
b) Copiar, almacenar o imprimir si ha de ser de uso exclusivamente personal y privado.
c) Copiar textualmente parte(s) de su contenido mencionando la fuente y/o haciendo la referencia correspondiente respetando normas de redacción e investigación.

El usuario no puede publicar, distribuir o realizar emisión o exhibición alguna de este material, sin la autorización correspondiente.

TODOS LOS DERECHOS RESERVADOS. EL USO NO AUTORIZADO DE LOS CONTENIDOS PUBLICADOS EN ESTE SITIO DERIVARA EN EL INICIO DE ACCIONES LEGALES CONTEMPLADOS EN LA LEY DE DERECHOS DE AUTOR.

DEDICATORIA

Dedico esta tesis a todas mis queridas amigas que han hecho el esfuerzo por comprenderme y entenderme. Aunque fuera para sacarme de la rutina o brindarme palabras de aliento, creyeron en mí aun cuando yo no lo hacía.

AGRADECIMIENTOS

- A Dios por darme la oportunidad de concluir exitosamente mi investigación.
- A mis padres por el apoyo incondicional a lo largo de mi carrera.
- A M.Sc. Rosa Flores Morales por su guía y paciencia a la hora de abordar el tema que se desarrolló en la presente Tesis de grado.
- A la carrera de Informática por brindarme las herramientas necesarias para poder superarme en las diversas áreas del conocimiento.
- A Lizeth Alvarado Bustillos, por su apoyo y amistad incondicional.

rolandex25@gmail.com

RESUMEN

En el presente trabajo de tesis se propone un modelo de aumento de resolución de imágenes usando autómatas celulares y redes neuronales convolucionales, considerando un conjunto de datos reducido para optimizar el uso de los recursos computacionales. Los autómatas celulares se usaron para detectar los bordes y fondos en las imágenes, obteniendo así una representación simplificada pero muy descriptiva de los bordes de objetos en las imágenes. Luego estas imágenes se usaron como una transformación intermedia entre dos redes neuronales convolucionales para ampliar la resolución de imágenes. Las redes neuronales fueron entrenadas de forma separada y luego de forma combinada usando un esquema similar a una red generativa condicional para afinar el modelo.

Se usó un conjunto de datos de 100 imágenes y parches de 128x128 pixeles optimizando así los recursos computacionales. Sin embargo, algunas imágenes mostraron cambios de luminosidad al combinar los parches optimizados, para resolverlo se desarrollaron tres métodos para difuminar e interpolar los parches. El modelo fue evaluado con las métricas MSE, PSNR, SSIM y DISTS, con seis diferentes conjuntos de datos y fue comparado con el modelo ESRGAN.

Los porcentajes de mejora del modelo propuesto fueron aproximados a ESRGAN y mejores en cuatro de los seis conjuntos de imágenes evaluados, mostrando mayores niveles de optimizado principalmente en la detección, control y reconstrucción de bordes de objetos en las imágenes, y un menor rendimiento en la reconstrucción de texturas.

Palabras clave: autómata celular, borde de imagen, red neuronal convolucional, ESRGAN, resolución de imagen.

Metodología: CRISP-DM.

ABSTRACT

In this thesis work, an image resolution enhancement model using cellular automata and convolutional neural networks is proposed, considering a reduced data set to optimize the use of computational resources. Cellular automata were used to detect the edges and backgrounds in the images, thus obtaining a simplified but very descriptive representation of the edges of objects in the images. These images were then used as an intermediate transformation between two convolutional neural networks to extend the image resolution. The neural networks were trained separately and then combined using a scheme similar to a conditional generative network to refine the model.

A data set of 100 images and 128x128 pixel patches was used, thus optimizing computational resources. However, some images showed luminosity changes when combining the optimized patches, to solve this, three methods were developed to blur and interpolate the patches. The model was evaluated with the MSE, PSNR, SSIM and DISTS metrics, with six different data sets and was compared with the ESRGAN model.

The improvement percentages of the proposed model were close to ESRGAN and better in four of the six image sets evaluated, showing higher levels of optimization mainly in the detection, control and reconstruction of object edges in the images, and lower performance in the reconstruction of textures.

Keywords: cellular automata, image edge, convolutional neural network, ESRGAN, image resolution.

Methodology: CRISP-DM.

ÍNDICE GENERAL

DEDICATORIA ..... **III**

AGRADECIMIENTOS ..... **IV**

RESUMEN ..... **V**

ABSTRACT ..... **VI**

ÍNDICE GENERAL ..... **VII**

ÍNDICE DE FIGURAS X
ÍNDICE DE TABLAS XIV

CAPÍTULO I **1**

MARCO REFERENCIAL **1**

1.1 INTRODUCCIÓN 1
1.2 ANTECEDENTES 2
1.3 DESCRIPCIÓN DEL PROBLEMA 4
1.4 PREGUNTAS DE INVESTIGACIÓN 5
1.5. HIPÓTESIS 5
1.6 OBJETIVO GENERAL 6
1.6.1 OBJETIVOS ESPECÍFICOS 6
1.7 JUSTIFICACIÓN 7
1.8 LÍMITES Y/O ALCANCES 7
1.9 METODOLOGÍA 8
1.10 ESTRUCTURA DE LA TESIS 10

CAPÍTULO II **11**

MARCO TEÓRICO **11**

2.1 FUNDAMENTOS DE PROCESAMIENTO DE IMÁGENES 11
2.1.4 CONVERSION DE IMÁGENES DE COLOR A ESCALA DE GRISES 11
2.1.5 FILTROS Y CONVOLUCIONES 12
2.1.6 DETECCIÓN DE BORDES 12

2.1.7 FILTRO DE DESENFOQUE 13
2.2 AUMENTO DE RESOLUCIÓN EN IMÁGENES DIGITALES. 14
2.2.1 MÉTODOS BASADOS EN INTERPOLACIÓN 14
2.2.2 MÉTODOS BASADOS EN EJEMPLOS DE PARCHES 15
2.3 MÉTRICAS DE COMPARACIÓN DE SIMILITUD DE IMÁGENES. 16
2.4 AUTOMATAS CELULARES EN EL TRATAMIENTO DE IMÁGENES 18
2.4.1 DESARROLLO DE UN ALGORITMO ADAPTATIVO BASADO EN AUTÓMATAS CELULARES PARA ELIMINAR RUIDO EN IMÁGENES DIGITALES 18
2.4.2 UN METODO COOPERATIVO PARA LA SEGMENTACIÓN DE IMÁGENES BASADO EN AUTOMATAS CELULARES 19
2.5 AUTÓMATAS CELULARES 20
2.5.1 ELEMENTOS DE UN AUTÓMATA CELULAR 20
2.5.2 EL JUEGO DE LA VIDA 21
2.5.3 TIPOS DE COMPORTAMIENTOS EVOLUTIVOS 22
2.6 REDES NEURONALES ARTIFICIALES PARA EL PROCESAMIENTO DE IMÁGENES 23
2.6.1 ESTRUCTURA DE UNA NEURONA ARTIFICIAL (PERCEPTRON SIMPLE) 23
2.6.2 RED NEURONAL ARTIFICIAL MULTICAPA (PERCEPTRÓN MULTICAPA) 24
2.6.3 TIPOS DE APRENDIZAJE AUTOMÁTICO 25
2.6.4 APRENDIZAJE PROFUNDO 26
2.6.5 REDES NEURONALES CONVOLUCIONALES 27
2.6.6 RED NEURONAL CONVOLUCIONAL U-NET 28
2.6.7 REDES RESIDUALES RESNET 30
2.6.7 MODELOS DE REDES NEURONALES GENERATIVAS 31
2.6.8 AUMENTO DE RESOLUCIÓN DE IMÁGENES USANDO REDES NEURONALES ARTIFICIALES 33
2.6.9 ETAPAS EN EL DESARROLLO Y PRUEBA DE MODELOS DE APRENDIZAJE AUTÓMATICO 37
2.6.10 CONJUNTO DE DATOS PARA PRUEBAS DE AUMENTO DE RESOLUCIÓN 39
2.6.11 RECURSOS COMPUTACIONALES USADOS EN EL ENTRENAMIENTO DE REDES NEURONALES ARTIFICIALES 40
2.6.12 CONSUMO ELÉCTRICO EN COMPUTADORES INTEL CORE I7 42
2.7 HERRAMIENTAS TECNOLÓGICAS PARA PROCESAR IMÁGENES 42

2.7.1 PYTHON 42
2.7.2 GOOGLECOLAB 44
2.7.3 ANACONDA 44
2.7.4 HUGGING FACE 44

CAPÍTULO III 45

MARCO APLICATIVO 45

3.1 ESTRUCTURA GENERAL 45
3.2 MÓDULO 1: CONSTRUCCIÓN DEL AUTÓMATA CELULAR. 46
3.3 MÓDULO 2: CONSTRUCCIÓN DEL MODELO DE RED NEURONAL. 55
3.4 AJUSTES EN LA RECONSTRUCCION DE BORDES. 75
3.5 AMPLIANDO EL MODELO PARA IMÁGENES A COLOR 77
3.6 EVALUACIÓN PRELIMINAR DE LA CAPACIDAD PREDICTIVA DEL MODELO 78
3.7 TIEMPO Y USO DE RECURSOS COMPUTACIONALES DEL MODELO 80
3.8 CONSUMO ELÉCTRICO Y EQUIVALENTE EN EMISIONES DE CO2 81

CAPÍTULO IV 86

EVALUACIÓN DEL MODELO 86

CAPÍTULO V 91

RESULTADOS 91

5.1 RESULTADOS SEGÚN LOS OBJETIVOS ESPECÍFICOS 93
5.2 RESULTADOS DE LA HIPÓTESIS 94

CAPÍTULO VI 95

CONCLUSIONES Y RECOMENDACIONES 95

REFERENCIAS 98

ANEXO A. FRAGMENTO DE CÓDIGO DEL AUTÓMATA CELULAR PARA DETECTAR BORDES. 107
ANEXO B. FRAGMENTO DE CÓDIGO DE LA ESTRUCTURA DE LAS REDES NEURONALES. 109

ÍNDICE DE FIGURAS

Figura 1.1: Emisión de dióxido de carbono producida al entrenar una RNA de tipo *Tranformer* comparada con otras fuentes de emisión de dióxido de carbono.....4
Figura 2.1: Matriz de filtrado aplicada a una imagen para transformarla.....12
Figura 2.2: Detección de bordes con el algoritmo Canny. 13
Figura 2.3: Incremento de la resolución espacial usando técnicas de interpolación.....15
Figura 2.4: Detección de bordes con ruido al 5%, monedas. 19
Figura 2.5: Detección de bordes en imágenes con alto nivel de detalle con umbral general igual a 2, 10 y 50. 20
Figura 2.6: Vecindad de Von Neumann (izquierda) y vecindad de Moore (derecha). 21
Figura 2.7: Ejemplo del autómata celular del juego de la vida de Conway. 22
Figura 2.8: Esquema del perceptron con peso adicional w_0 23
Figura 2.9: Ejemplo de red neuronal artificial multi-capas.....25
Figura 2.10: Esquema de aprendizaje profundo donde se resalta la integración de la extracción de características y la clasificación en el mismo modelo. 26
Figura 2.11: Ejemplo de CNN denominada LeNet5 para reconocimiento de caracteres, presentada por LeCun, 1988. 27
Figura 2.12: Ejemplo original de la arquitectura U-Net propuesta por Ronneberger, Fischer y Brox 2015. 29
Figura 2.13: Arquitectura básica de un bloque residual con dos capas de neuronas. 30
Figura 2.14: Ejemplo de bloques residuales con conexiones de salto simples y densas. 31
Figura 2.15: Arquitectura básica de un autocodificador variacional.....32
Figura 2.16: Arquitectura básica GAN. 32
Figura 2.17: Arquitectura básica CGAN. 33
Figura 2.18: Primera arquitectura SRCNN para aumento de resolución. 34

Figura 2.19: Primera arquitectura de un RNA profunda VDSR para el aumento de resolución. 35
Figura 2.20: Arquitectura de modelo ESRGAN con bloques residuales RRBD. 35
Figura 2.21: Etapas en el desarrollo de modelos de aprendizaje automático.....38
Figura 3.1: Diagrama general del modelo de optimización de resolución de imágenes..46
Figura 3.2: Ejemplo de matriz A de células del autómata y matriz I correspondiente a los pixeles de la imagen. 47
Figura 3.3: Para un umbral de 3, el pixel central tiene dos vecinos similares. 48
Figura 3.4: Estado inicial del universo del autómata, aleatorio y con un patrón intercalado. 49
Figura 3.5: Aplicación del primer autómata para detectar bordes, con un umbral de 3, y con 5 iteraciones.....50
Figura 3.6: Secuencia evolutiva del primer autómata con cinco iteraciones. 50
Figura 3.7: Aplicación del segundo autómata para detectar lo que no son bordes, con un umbral de 3, y con 5 iteraciones.....52
Figura 3.8: Secuencia evolutiva del segundo autómata con cinco iteraciones 52
Figura 3.9: Combinación de las dos imágenes generadas por los autómatas en una sola imagen. 54
Figura 3.10: Combinación de las dos imágenes a color generadas por los autómatas en una sola imagen a color.....55
Figura 3.11: Primer diseño de las redes neuronales de optimización de resolución de imágenes.....57
Figura 3.12: Segundo diseño de las redes neuronales de optimización de resolución de imágenes.....58
Figura 3.13: Arquitectura de la primera red neuronal convolucional. 60
Figura 3.14: Arquitectura de la segunda red neuronal convolucional.....61
Figura 3.15: Función de pérdida de la primera y segunda red neuronal durante el entrenamiento individual.....62
Figura 3.16: Ejemplos de imágenes reducidas por un factor 4x y la imagen resultante correspondiente de estructuras de bordes generada por la primera red neuronal. 63

Figura 3.17: Ejemplo de imagen reducida por un factor 4x y la imagen resultante optimizada. 64
Figura 3.18: Esquema del modelo combinado de ambas redes neuronales con una entrada y dos salidas. 65
Figura 3.19: Esquema de entrenamiento combinado de ambas redes neuronales. 65
Figura 3.20: Función de pérdida de la primera y segunda red neuronal durante el entrenamiento combinado. 66
Figura 3.21: A la derecha la imagen reconstruida muestra variaciones de brillo entre parches. 68
Figura 3.22: Ejemplo de los pixeles del margen izquierdo y superior de un parche. 68
Figura 3.23: A la derecha la imagen reconstruida aplicando correcciones de brillo entre parches. 70
Figura 3.24: Máscara de difuminado de 4 pixeles en los bordes del parche. 70
Figura 3.25: Resultado de aplicar los métodos de ajuste de brillo y difuminado de bordes aplicado dos veces. 71
Figura 3.26. Ejemplo de aumento de bordes reflejando el contenido. 72
Figura 3.27: Mascara de intensidad usada para combinar las dos imágenes. 72
Figura 3.28: Gráfica de la función de intensidad para la construcción de la máscara. 73
Figura 3.29: Resultado de aplicar los métodos de ajuste de brillo, difuminado de bordes e interpolación entre parches 74
Figura 3.30: Diagrama del modelo de optimización de resolución de imágenes en escala de grises. 74
Figura 3.31: Aplicar reiteradamente la segunda red neuronal del modelo ya entrenado. 75
Figura 3.32: Aplicando la segunda red neuronal reiteradamente. 75
Figura 3.33: Modificando la imagen de estructuras de borde, colocando un cuadrado blanco y negro. 76
Figura 3.34: Resultado de aplicar la imagen de estructuras modificada y la segunda red neuronal. 76
Figura 3.35: Diagrama final del modelo de optimización de resolución para imágenes a color. 77

Figura 3.36: Ejemplo de imagen del conjunto de datos Set5 reducida por un factor 4x, la imagen de bordes y la imagen resultante optimizada.....77
Figura 3.37: Ejemplo de imagen del conjunto de datos General100 reducida por un factor 4x y la imagen resultante optimizada.....78
Figura 3.38: Se observan las cuatro métricas calculadas comparando: la imagen A con ella misma, la imagen A con la B, y la imagen A con la C. 78
Figura 3.39: Ejemplo demostrativo del modelo RODIUCAN publicado en Hugging Face. 85

ÍNDICE DE TABLAS

Tabla 1.1: Principales tareas en la implementación del modelo propuesto de aumento de resolución de imágenes. 9
Tabla 2.1: Conjuntos de datos usados, tamaño, y recursos para el entrenamiento del modelo ESRGAN/Real-ESRGAN. 36
Tabla 2.2: Descripción de los típicos conjuntos de datos usados para modelos de aumento de resolución. 39
Tabla 2.3: Emisiones estimadas de CO2 por el entrenamiento de redes neuronales profundas. 41
Tabla 3.1 Parches de imágenes usados como entrada y salida para entrenar la primera red neuronal 58
Tabla 3.2 Arquitectura de dimensión y cantidad de características de las redes U-Net utilizadas 59
Tabla 3.3 Parches de imágenes usados como entrada y salida para entrenar la segunda red neuronal 60
Tabla 3.4 Error durante el entrenamiento combinado en la segunda fase. 67
Tabla 3.5: Valores óptimos por cada métrica utilizada para comparar imágenes. 79
Tabla 3.6: Métricas promedio obtenidas para el modelo propuesto RODIUCAN para un subconjunto de validación del conjunto de datos General100. 79
Tabla 3.7: Tiempo de entrenamiento individual de ambas redes neuronales con GPU. ..80
Tabla 3.8: Tiempo de entrenamiento individual de ambas redes neuronales sin GPU. ...80
Tabla 3.9: Tiempo de entrenamiento combinado de las redes neuronales sin GPU. 81
Tabla 3.10: Descripción de los archivos publicados en el repositorio GitLab. 84
Tabla 4.1: Descripción de los conjuntos de datos usados en la evaluación de los modelos. 86
Tabla 4.2: Métricas promedio obtenidas para el modelo propuesto RODIUCAN para distintos conjuntos de imágenes. 87
Tabla 4.3: Porcentajes de mejoras del modelo propuesto RODIUCAN. 89
Tabla 4.4: Métricas promedio obtenidas para el modelo ESRGAN para distintos conjuntos de imágenes. 89

Tabla 4.5: Porcentaje de mejora del modelo ESRGAN. 90
Tabla 5.1: Métricas promedio obtenidas para el modelo propuesto RODIUCAN comparado con el modelo ESRGAN. 91
Tabla 5.2: Comparación porcentual de mejora de los modelos RODIUCAN y ESRGAN. 92
Tabla 5.3: Comparación del entorno de entrenamiento de los modelos RODIUCAN y ESRGAN. 92
Tabla 5.4: Consumo eléctrico y emisiones de CO_2 equivalentes del modelo propuesto y ESRGAN. 93

CAPÍTULO I

MARCO REFERENCIAL

1.1 INTRODUCCIÓN

En el presente trabajo de tesis se propone el uso de autómatas celulares y redes neuronales artificiales convolucionales para optimizar las imágenes digitales respecto al aumento de resolución, reduciendo los problemas de bordes irregulares y desenfoque. Para ello se usarán los autómatas celulares para generar imágenes que servirán como una transformación intermedia entre dos redes neuronales convolucionales con el objetivo de aumentar la resolución de imágenes digitales.

Al ampliar las imágenes en un factor de dos a cuatro veces su tamaño se generan defectos de bordes irregulares y desenfoque, los cuales se intensifican a medida que se aumenta de escala. Esto hace necesario el uso de algoritmos de procesamiento de imágenes digitales para optimizar la calidad en el caso de ampliaciones de gran tamaño. Al ampliar las imágenes se debe completar la información faltante y actualmente los mejores métodos usan redes neuronales artificiales (RNA), como por ejemplo el modelo ESRGAN. Pero dichos modelos tienen altos costos computacionales debido a la gran cantidad de imágenes requerida para su entrenamiento y el uso intensivo de GPU.

Por otro lado, los autómatas celulares (AC) son modelos ampliamente usados en simulación, pero también pueden usarse para obtener características estructurales en imágenes digitales, como por ejemplo los bordes de objetos a diferentes niveles de detalle.

La principal motivación para el desarrollo del presente trabajo es la experimentación y tratar de relacionar el uso de los autómatas celulares con las redes neuronales artificiales para el aumento de la resolución de imágenes digitales, capturando con los autómatas los detalles más importantes de las imágenes y así reducir la complejidad del problema para construir un modelo óptimo basado en dos RNAs que utilice 100 imágenes en su entrenamiento y un uso mínimo de GPU.

Los resultados de las evaluaciones en seis diferentes conjuntos de imágenes demostraron que el modelo propuesto entrenado con 100 imágenes puede alcanzar niveles aproximados de optimización en comparación al modelo ESRGAN y se destaca por la reconstrucción de bordes.

1.2 ANTECEDENTES

En esta sección de muestra de forma resumida los métodos de aumento de resolución y trabajos relacionados con el tema del presente trabajo.

El aumento de resolución de una sola imagen tiene como objetivo incrementar la resolución de una imagen de baja resolución para luego obtener una imagen de mayor resolución y recuperar detalles perdidos o difusos. Para abordar este problema, se han propuesto diferentes métodos desde los básicos basados en simple interpolación hasta los más sofisticados mediante el uso de redes neuronales artificiales.

Los métodos clásicos basados en la interpolación de pixeles han sido utilizados ampliamente debido al bajo costo y complejidad de sus algoritmos. Pero cuanto más grande sea la ampliación tienen ciertos problemas como que tienden a difuminar los detalles y perder la forma de los bordes (Rodríguez Rodríguez, 2016).

Para resolver estos problemas se plantearon métodos basados en la utilización de un diccionario de parches de imágenes de baja y alta resolución. Para posteriormente establecer relaciones de similitud entre parches de baja resolución nunca vistos y los presentes en el diccionario. El método tiene la desventaja de que el diccionario debe ser muy grande, lo cual aumenta el tiempo de búsqueda de patrones similares. (Rodríguez Rodríguez, 2016). De acuerdo a (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020) los mejores métodos para obtener aumento de resolución son los basados en redes neuronales artificiales convolucionales, las cuales pueden ser redes profundas o poco profundas.

Redes neuronales convolucionales y aumento de resolución

Las redes neuronales convolucionales (CNN) pueden ser poco profundas cuando tienen menos de diez capas. El primer trabajo que aplicó este tipo de red neuronal utilizó el termino SRCNN (*Super Resolution*, CNN), para indicar su aplicación al aumento de resolución de imágenes. Dicho modelo uso una arquitectura de 3 capas y el algoritmo del descenso del gradiente para la optimización. Los resultados fueron superiores a los métodos tradicionales, manteniendo un nivel medio de complejidad computacional. (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020).

Después de los buenos resultados obtenidos con redes poco profundas, se realizaron más investigaciones con redes cada vez más grandes. Este tipo de red neuronal se denominó red neuronal profunda, la cual tiene una arquitectura más compleja y puede lograr mejores resultados en el aumento de resolución de imágenes, aunque el costo computacional en la etapa de entrenamiento y ejecución aumenta considerablemente. Uno de los mejores modelos de redes neuronales profundas para el aumento de resolución es el modelo SRGAN (*Super-Resolution Generative Adversarial Network*), el cual a su vez ha dado origen a otros modelos como ESRGAN o Real-ESRGAN (Xintao, y otros, 2018)

Autómatas celulares aplicados a imágenes digitales

Un autómata celular (AC) es un modelo matemático y computacional inspirado en la naturaleza, el cual evoluciona en pasos discretos. Los autómatas celulares se han usado tradicionalmente para simular el comportamiento de sistemas físicos, pero también se han aplicado al análisis de imágenes digitales.

Los autómatas celulares pueden identificar patrones de ruido en imágenes digitales, como en el trabajo de (Angulo Sogamoso & Gil Sierra, 2019). En donde se usan los autómatas celulares para eliminar el ruido y detectar bordes en imágenes digitales en escala de grises, agregando a las reglas del autómata condiciones sobre la intensidad de los pixeles. Los resultados obtenidos indican que el uso de AC permite la restauración de imágenes con ruido y la detección de bordes.

1.3 DESCRIPCIÓN DEL PROBLEMA

Actualmente los mejores métodos de aumento de resolución de imágenes se basan en redes neuronales artificiales profundas. Pero el tiempo de entrenamiento de dichos modelos aumenta considerablemente, lo cual causa un gasto de recursos computacionales y un impacto negativo al medio ambiente.

Esto se sustenta por un estudio realizado por investigadores de la Universidad de Massachusetts, Amherst (Strubell, Ganesh, & McCallum, 2019), en el cual determinaron que el proceso de entrenar redes neuronales artificiales de gran tamaño puede emitir más de 626000 libras de dióxido de carbono debido al uso intensivo de GPU. En la Figura 1.1 se muestra la emisión de dióxido de carbono producida por entrenar un RNA de tipo *transformer* comparada con otros medios comunes de contaminación ambiental. Un *transformer* es un tipo de red neuronal que aprende contexto analizando texto, imagen o video. El modelo de 213 millones de parámetros fue entrenado en una sola GPU durante un día para medir su consumo de energía y luego fue convertido a su equivalente en libras de carbono, usando datos de consumo de energía de los EEUU (Hao, 2019)

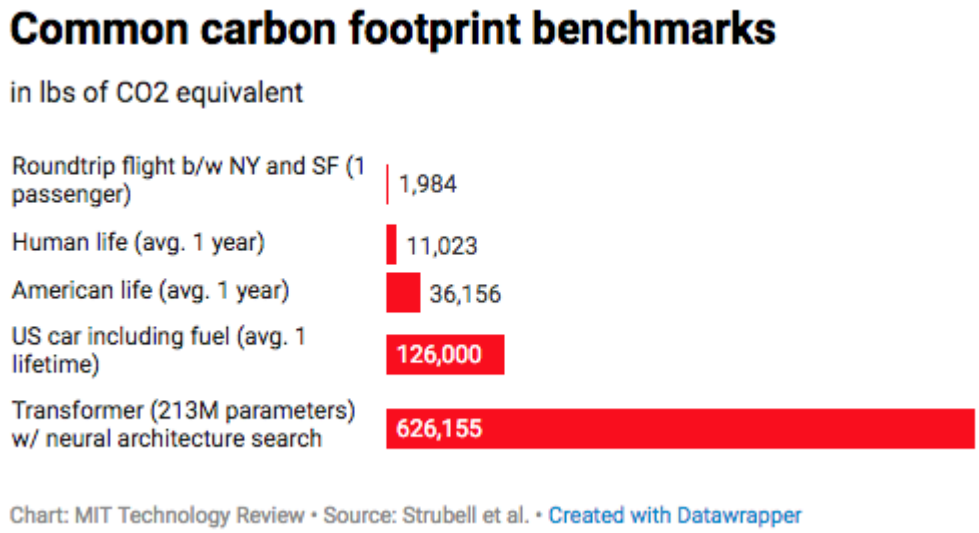


Figura 1.1: Emisión de dióxido de carbono producida al entrenar una RNA de tipo *Tranformer* comparada con otras fuentes de emisión de dióxido de carbono.
Fuente: (Strubell, Ganesh, & McCallum, 2019)

Uno de los mejores modelos para el aumento de resolución de imágenes es el modelo ESRGAN desarrollado por (Xintao, y otros, 2018), el cual al igual que los *Transfomers* requiere del uso intensivo de GPU debido a que se necesita de un conjunto de imágenes grande para su entrenamiento, y se recomienda el uso de 4 GPUs.

Los altos costos en tarjetas gráficas GPU limitan la capacidad de entrenar estos modelos en máquinas personales. Y si bien existen soluciones gratuitas como GoogleColab que permite el uso remoto de máquinas virtuales con una sola GPU. Estas están limitadas a un máximo de 12GB de RAM y un tiempo máximo de sesión de 12 horas dependiendo de la demanda y la cantidad de usuarios. Otros servicios en la nube de alquiler de GPU para ciencia de datos son caros y suelen requerir un conocimiento previo para la configuración del servidor o máquina virtual.

1.4 PREGUNTAS DE INVESTIGACIÓN

Considerando el uso de autómatas celulares en el procesamiento de imágenes donde cada figura se trata como un sistema de cómputo de células que representan los píxeles y sus niveles de RGB[1]; surgen las siguientes preguntas de investigación:

¿Combinar un modelo híbrido entre RNA y AC optimizará la resolución de imágenes?

¿El uso de AC en la optimización de resolución de imágenes permite entrenar un RNA, con el mismo propósito, mediante un conjunto de datos reducido?

1.5. HIPÓTESIS

H1: El uso de un modelo combinado de autómatas celulares y redes neuronales artificiales, permite optimizar la resolución de imágenes digitales.

H2: El uso de un autómata celular permite entrenar una red neuronal con un conjunto de datos reducido y optimizar el uso de recursos computacionales.

[1] Sigla del inglés *red, green, blue* que representan la composición de colores rojo, verde y azul.

Operacionalización de variables

- Variable independiente: Conjunto de datos reducido (Cd). Se refiere a la cantidad de imágenes del conjunto de datos requerido para el entrenamiento del modelo.
- Variable dependiente: Recursos computacionales (Rc). Es la cantidad de memoria RAM y uso de GPU requerido para el entrenamiento del modelo.
- Variable dependiente: Optimización de resolución de imagen (Or). Es el porcentaje de mejora de resolución de imagen al aplicar el modelo combinado.

1.6 OBJETIVO GENERAL

Desarrollar un modelo de aumento resolución de imágenes digitales utilizando las imágenes generadas por dos autómatas celulares como una transformación intermedia entre dos redes neuronales convolucionales.

1.6.1 OBJETIVOS ESPECÍFICOS

- Considerar un conjunto de datos de 100 imágenes, optimizando el tamaño de las redes neuronales.
- Implementar dos autómatas celulares, uno para la detección de bordes con un alto nivel de detalle y otro para complementar la información con aquello que no se considere bordes.
- Considerar un conjunto de datos de 100 imágenes y construir un conjunto de datos de imágenes formado por imágenes en baja y alta resolución.
- Aplicar los autómatas celulares a las imágenes de alta calidad para capturar sus detalles.
- Implementar dos redes neuronales convolucionales optimizando su tamaño y que utilicen parches de imágenes de 128x128 pixeles en su entrada y salida.
- Entrenar las redes neuronales convolucionales, considerando de la primera red debe aproximar los parches de baja resolución a las imágenes obtenidas por el autómata. Y la segunda red debe combinar el resultado de la primera red con el parche de imagen de baja calidad para así obtener un parche de imagen de alta calidad.

- Evaluar el modelo midiendo la similitud de las imágenes estimadas con las imágenes originales, usando las métricas MSE, PSNR, SSIM, y DISTS. Y considerando seis diferentes conjuntos de datos de imágenes.
- Realizar la misma evaluación con el modelo ESRGAN y obtener porcentajes de mejora similares o superiores a dicho modelo.

1.7 JUSTIFICACIÓN

El entrenamiento de modelos de aumento de resolución como ESRGAN requiere de un uso intensivo de GPU. Dichos recursos computacionales no siempre están disponibles por su alto costo o su uso en línea es limitado. En este sentido es importante investigar un modelo de aumento de resolución que optimice los recursos computacionales y logre reducir el uso de GPU obteniendo resultados de calidad de aumento de resolución aproximados a ESRGAN.

Por otro lado, el uso de las imágenes obtenidas por los autómatas celulares como una transformación intermedia entre dos redes neuronales convolucionales es una estrategia nueva en el problema de aumento de resolución de imágenes. Es importante estudiar esta estrategia y medir cuál es su capacidad de optimización, ya que podría usarse en problemas similares de procesamiento de imágenes.

1.8 LÍMITES Y/O ALCANCES

No se pretende reemplazar el modelo de aumento de resolución ESRGAN, pero si brindar un modelo alternativo que utilice menos recursos computacionales y tenga porcentajes de mejora de calidad similares. Se eligió el modelo ESRGAN como modelo de comparación por ser el más parecido respecto al uso de redes neuronales convolucionales y el uso de un gran conjunto de datos para su entrenamiento. Sin embargo, el modelo propuesto en el presente trabajo introduce un componente nuevo que son las imágenes generadas por el autómata celular, con el fin de optimizar el entrenamiento y uso de recursos computacionales.

Los autómatas celulares serán aplicados a las versiones en escala de grises de las imágenes, con el objetivo de simplificar la tarea de detección de bordes. Sin embargo, esto no limitara su uso en imágenes a color. Debido a que las dos redes convolucionales se diseñaran tanto en sus versiones de escala de grises y a color.

El modelo propuesto solo será entrenado con un conjunto de datos de 100 imágenes de acuerdo a los objetivos de optimización de recursos.

Solo se considerarán defectos relacionados con el cambio de resolución por interpolación bicúbica, interpolación vecinos más cercanos, con un factor de reducción de 2x,4x,6x su tamaño y desenfoque. No se consideran otros defectos como: ruido intenso, cambios de luminosidad extremos, o secciones de imágenes completamente perdidas.

1.9 METODOLOGÍA

El desarrollo del proyecto está formado por dos módulos: el primero enfocado en la construcción de los autómatas celulares y el segundo en la elaboración de las redes neuronales convolucionales. Para luego finalizar con la evaluación de los resultados obtenidos.

Para la construcción de los autómatas celulares se sigue un enfoque experimental, partiendo de las reglas del juego de la vida y considerando trabajos previos para la detección de bordes en imágenes en escala de grises.

Para la construcción de las redes neuronales convolucionales se sigue el estándar CRISP-DM aplicado a la elaboración de modelos de inteligencia artificial, donde se definen los pasos fundamentales como ser: la definición del objetivo de la red neuronal, pre-procesamiento de datos, pruebas y evaluación.

Para la construcción del software se sigue un enfoque modular donde sistemáticamente se construyen funciones y procedimientos que automaticen las tareas. De modo que una vez finalizado el proyecto este pueda adaptarse a trabajos futuros con diferentes conjuntos de datos.

En la Tabla 1.1 se especifican las principales tareas de los capítulos tres y cuatro.

	Etapa	Actividades
1	Implementación de los autómatas celulares.	• Implementar los autómatas celulares para la detección de bordes con un alto nivel de detalle en imágenes en escala de grises. • Programar funciones para automatizar los procesos. • Ejecutar el programa en local y en GoogleColab.
2	Aplicar el pre-procesamiento al conjunto de datos de imágenes.	• Convertir las imágenes en escala de grises • Reducir las imágenes en un factor de 2, 4 y 6 veces de su tamaño original con interpolación bicúbica, vecinos más cercanos, y filtro de desenfoque. También convertirlas a sus versiones en escala de grises. • Programar funciones para segmentar las imágenes en parches de 128x128 pixeles, y para automatizar los procesos. • Ejecutar el programa en local y en GoogleColab.
3	Aplicar los autómatas celulares.	• Aplicar los autómatas celulares a las imágenes de alta calidad en escala de grises, para extraer las características de bordes.
4	Implementar dos redes neuronales convolucionales.	• Implementar dos redes neuronales convolucionales con arquitectura U-Net, que use como entrada y salida parches de imagen de 128x128 pixeles.
5	Entrenar las redes neuronales convolucionales.	• Entrar la primera red neuronal usando como entradas los parches de imagen de baja calidad para predecir las imágenes obtenidas por el autómata celular. Entrenar la segunda red neuronal con las predicciones de la primera red y combinarlos con la imagen de baja calidad para predecir las imágenes de alta calidad.
6	Evaluar el modelo propuesto.	• Evaluar las predicciones del modelo comparándolas con las imágenes originales usando las métricas: MSE, PSNR, SSIM y DISTS. Usando seis diferentes conjuntos de datos. • Realizar la misma evaluación con el modelo ESRGAN y comparar los porcentajes promedio de optimización de ambos modelos.

Tabla 1.1: Principales tareas en la implementación del modelo propuesto de aumento de resolución de imágenes.

1.10 ESTRUCTURA DE LA TESIS

La tesis comprende cinco capítulos incluyendo el actual:

Capítulo 2

En el capítulo dos se hace una investigación sobre los fundamentos teóricos sobre procesamiento de imágenes, redes neuronales artificiales aplicadas al procesamiento de imágenes, autómatas celulares, librerías Python requeridas y métricas para comparar imágenes. Dichos conceptos son necesarios para la compresión del presente trabajo.

Capítulo 3

En el capítulo tres Se describe paso a paso la implementación del modelo propuesto. Desde la construcción de los autómatas celulares, diseño de las redes neuronales convolucionales y una evaluación preliminar del modelo.

Capítulo 4

En el capítulo cuatro se evalúa el modelo propuesto con seis diferentes conjuntos de datos de imágenes y se repite la misma evaluación para el modelo ESRGAN.

Capítulo 5

En el capítulo cinco se muestran los resultados obtenidos, por objetivos, hipótesis y se hace una comparación de porcentajes de optimización con el modelo propuesto y el modelo ESRGAN.

Capítulo 6

En el capítulo seis se establecen las conclusiones y recomendaciones del trabajo.

CAPÍTULO II

MARCO TEÓRICO

2.1 FUNDAMENTOS DE PROCESAMIENTO DE IMÁGENES

El procesamiento de imágenes digitales se refiere a las técnicas de adquisición, representación, análisis, procesado, filtrado, transformaciones y espacios de color (García Mateos, 2017).

Las imágenes a color RGB se componen de tres canales de color, con 8 bits por píxel se cuenta con 256 posibles valores para cada canal, lo que significa más de 16 millones de posibles valores de color (Adobe, 2022). De acuerdo a (Pérez & Valente, 2018) se usan 8 bits debido a que estudios biométricos muestran que el ojo humano no es suficientemente sensible para diferenciar más de 256 niveles de intensidad de color.

La resolución de imagen se refiere al número de pixeles asignados y está estrechamente relacionado con las dimensiones en pixeles de ancho y alto que ocupa una imagen (Adobe, 2022). Según (Pérez & Valente, 2018) la resolución de una imagen: "se podría definir, de modo genérico, como la capacidad para representar o percibir los detalles de una imagen."

2.1.4 CONVERSION DE IMÁGENES DE COLOR A ESCALA DE GRISES

Una operación de procesamiento global útil es la conversión de una imagen a escala de grises. Para realizar dicha conversión se usa el promedio ponderado de los tres canales RGB. La ponderación es diferente debido a que el ojo humano es más sensible al color verde (López Franco, 2022). La fórmula es : $Y = 0.299R + 0.587G + 0.114B$
Donde los coeficientes son los mismos de la fórmula de luminancia según la recomendación UIT-R BT.601-7 (Unión Internacional de Telecomunicaciones, 2011).

2.1.5 FILTROS Y CONVOLUCIONES

Los filtros son operaciones a nivel de pixel que consideran los pixeles vecinos. Los filtros se puedan aplicar en el dominio del espacio operando directamente sobre los pixeles, o en el de la frecuencia con la transformada de Fourier (Querejeta Simbeni, 2015).

Los filtros en el dominio del espacio se pueden usar para detectar bordes midiendo los cambios bruscos de intensidad entre pixeles. Son básicamente matrices de filtrado o también llamadas máscaras de convolución (*kernels)* (Querejeta Simbeni, 2015). En la Figura 2.1 la matriz de filtrado recorre cada uno de los pixeles de la imagen original transformándolos.

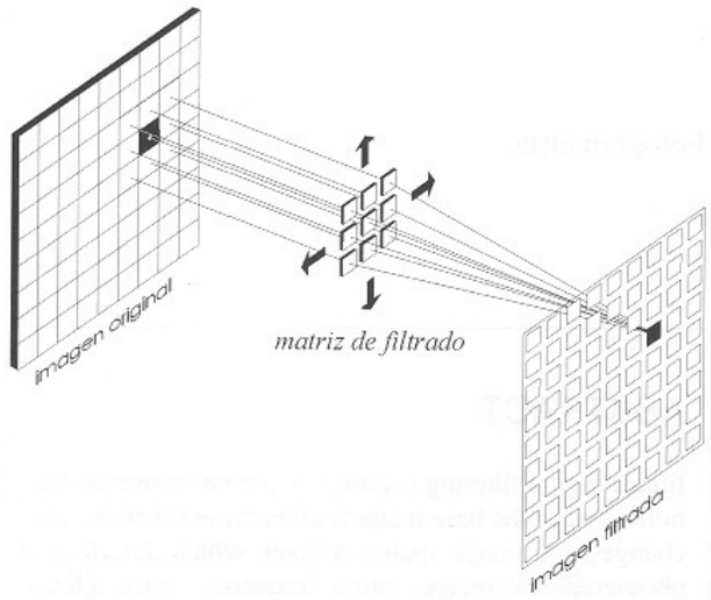


Figura 2.1: Matriz de filtrado aplicada a una imagen para transformarla
Fuente: (Querejeta Simbeni, 2015)

2.1.6 DETECCIÓN DE BORDES

El reconocimiento de bordes es una tarea requerida en el procesamiento de imágenes, debido a que permite simplificar el análisis para otras tareas, como por ejemplo la segmentación, reconocimiento de patrones, etc. (Suárez & Villavicencio, 2017).

Los primeros métodos para detección de bordes se basan en el gradiente de una imagen. El gradiente sirve para medir los cambios de intensidad grandes, matemáticamente se

definen como derivadas parciales a lo largo de direcciones X e Y. Por ejemplo, los filtros "Sobel", "Prewitt" y "Robert" son aproximaciones del gradiente y se definen como matrices de transformación. Pero no son eficientes para bordes afilados y son sensibles al ruido de la imagen (Puga, Ramírez Noriega, & Soto, 2021) .

Otra técnica que si considera el ruido para la detección de bordes es el método "Canny", que tiene una baja tasa de error en la detección de bordes (Puga, Ramírez Noriega, & Soto, 2021). Canny es un método de detección de bordes en imágenes digitales desarrollado por Jhon F. Canny en 1986. (Unipython, 2018)

En la Figura 2.2 se aplica el método "Canny" el cual básicamente elimina ruido, determina cambios de intensidad máxima, descarta pixeles borrosos y afina los bordes. El método tiene dos valores ajustables para determinar el umbral de detección.

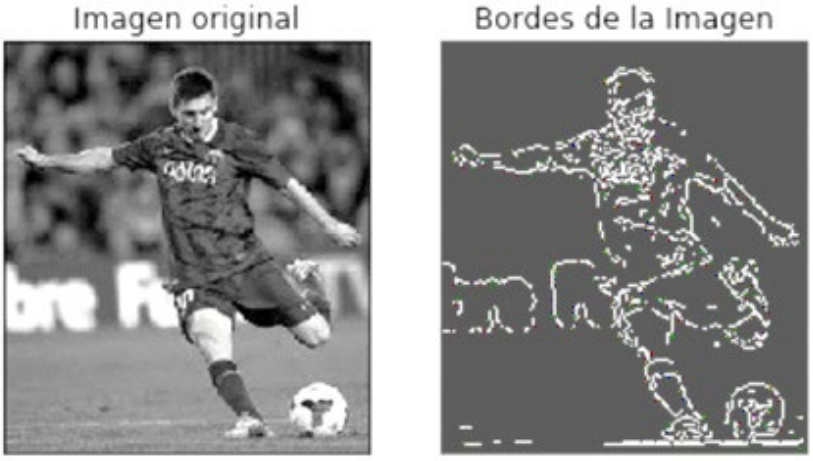


Figura 2.2: Detección de bordes con el algoritmo Canny.
Fuente: (Unipython, 2018)

2.1.7 FILTRO DE DESENFOQUE

El desenfoque o suavizado es una de las operaciones más usadas en procesamiento de imágenes, el cual básicamente consiste en hacer que la imagen se vea borrosa. Existen diferentes tipos como ser: filtros homogéneos, gaussianos y medianos. En los filtros homogéneos cada pixel de salida es el resultado de aplicar un *kernel* cuadrado. Por ejemplo, para un *kernel* de 5x5 se debe dividir 1 entre 25 y multiplicarlo por una matriz de 5x5 de unos, como se muestra en la ecuación 1 (Mehmood, 2022).

$$K = \frac{1}{5x5} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \qquad (1)$$

2.1.8 INTERPOLACIÓN DE PÍXELES

La interpolación es la estimación de píxeles desconocidos en función de otros que si son conocidos. Existen diferentes métodos, algunos de los más usados son:

- La interpolación por vecino más cercano llamado (N.N por sus siglas del inglés *Nearest Neighbor*) es el algoritmo más simple que solo tiene en cuenta un pixel, es decir el más cercano al punto al que se desea interpolar. Y simplemente se repite dicho pixel (Perez Alonso, 2020).
- La interpolación bilineal considera los píxeles de una vecindad de 2x2. Luego se calcula el promedio ponderado de los cuatro píxeles.
- La interpolación bicúbica considera un área de 4x4 píxeles para realizar la interpolación, y tiene mejores resultados que la interpolación NN o bilineales. Por lo cual es uno de los principales métodos de interpolación (Greyrat, 2022).

2.2 AUMENTO DE RESOLUCIÓN EN IMÁGENES DIGITALES.

El incremento de resolución de una sola imagen (SISR, *Single Image Super-Resolution*) tiene como objetivo es incrementar la resolución de una imagen de baja resolución LR (L*ow Resolution*) para obtener una imagen de mayor resolución HR (*High Resolution*) y recuperar información de alta frecuencia. Es decir, la imagen LR es una versión sub-muestreada y/o ruidosa de la imagen de alta resolución HR. Se han propuesto diferentes métodos de aumento de resolución, los principales se basan en la interpolación y el uso de diccionarios de pares de parches de baja y alta resolución. (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020).

2.2.1 MÉTODOS BASADOS EN INTERPOLACIÓN

Según (Rodríguez Rodríguez, 2016) los métodos clásicos basados en interpolación son el Linear, Bilinear, Bicubica, B-Spline, etc. Han sido utilizados debido al bajo costo y

complejidad de sus algoritmos. Pero tienen ciertos problemas como que tienden a difuminar los detalles y perder bordes. Otro problema es que a altas ampliaciones se generan artefactos no deseados como *aliasing*, *edge halo* y *blur*. Como se puede apreciar en la Figura 2.3 los artefactos distorsionan la imagen ampliada, debido a que el algoritmo no completa adecuadamente las zonas ampliadas entre pixeles



Figura 2.3: Incremento de la resolución espacial usando técnicas de interpolación.
Nota. En esta imagen se muestran 3 de los métodos de interpolación más populares. A) Imagen Original, B) Nearest Neighbor, C) Bilinear, D) Bicúbica. La imagen A se redujo de tamaño usando el modelo de adquisición con un factor de escala s = 2 y un valor de σ = 0.8. Posteriormente se usaron los métodos de interpolación mencionados para restaurar las dimensiones a la imagen original.

Fuente: (Rodríguez Rodríguez, 2016).

2.2.2 MÉTODOS BASADOS EN EJEMPLOS DE PARCHES

Otro enfoque de incremento de resolución de una sola imagen se basa en el aprendizaje basado en ejemplos, donde se explota la propiedad de auto similitud para generar imágenes de HR a partir de la entrada de LR. Los métodos basados en ejemplos utilizan una base de datos, también llamada diccionarios, que contiene pares de parches (porciones de la imagen) de alta y baja resolución para establecer relaciones de mapeo entre los parches de la imagen de LR y los parches de la imagen de HR; tal relación de mapeo se utiliza posteriormente para estimar nuevas imágenes de HR a partir de la entrada LR. La estimación se hace por similitud geométrica o basados en modelos de regresión. Este método tiene la desventaja de que el diccionario debe ser muy grande, lo cual aumenta el tiempo de búsqueda de patrones similares (Rodríguez Rodríguez, 2016).

Para resolver las desventajas de este método se han propuesto métodos basados en redes neuronales artificiales, de manera que las entradas de la red neuronal sean los parches LR y las salidas los HR. En comparación con los métodos basados en interpolación, los métodos basados en RNA tienen un rendimiento superior en la calidad de reconstrucción (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020).

El problema de los modelos que predicen parches HR, consiste en que estos son una estimación muy cercana pero no igual al parche de alta resolución. Esto afecta la reconstrucción global de los parches de la imagen (Rodríguez Rodríguez, 2016). Para resolver este problema se han propuesto métodos principalmente basados en la interpolación de parches.

2.3 MÉTRICAS DE COMPARACIÓN DE SIMILITUD DE IMÁGENES.

Para comparar la similitud entre dos imágenes es necesario utilizar métricas objetivas, es decir que puedan medirse numéricamente. De acuerdo a (Pereira, 2020) y (León Batallas, Bermeo Paucar, Paredes Quevedo, & Torres Ordoñez, 2020) se tienen dos tipos de comparación de similitud de imágenes:

- **Métricas basadas en el pixel**. - Se basan en recuentos estadísticos a nivel de pixel, como el error cuadrático medio (MSE, *Mean Square Error*), la raíz del error cuadrático medio (RMSE, *Root Mean Square Error*), la proporción máxima de señal ruido (PSNR, *Peak Signal to Noise Ratio*)

  La fórmula del MSE calcula la diferencia entre los pixeles obtenidos y los esperados, y se define como: $MSE = \frac{1}{MN}\sum_{Y=1}^{M}\sum_{X=1}^{N}[I(x,y) - I'(x,y)]^2$

  La relación pico de señal a ruido (PSNR) es la relación entre la potencia máxima de una señal y la potencia del ruido corruptor que afecta la fidelidad de su MSE. El PSNR se expresa como una cantidad logarítmica expresada en decibelios (León Batallas, Bermeo Paucar, Paredes Quevedo, & Torres Ordoñez, 2020). En general un PSNR más alto indica un nivel de reconstrucción de mayor calidad y se define como: $PSNR = 10.log_{10}(\frac{(max_I)^2}{MSE})$

Los métodos convencionales PSNR y MSE no siempre coinciden con los resultados de la visualización subjetiva y es necesario complementarlos con alguna métrica estructural (León Batallas, Bermeo Paucar, Paredes Quevedo, & Torres Ordoñez, 2020).

- **Métricas estructurales.** - Consideran las características del sistema visual humano (HVS), como la relación entre pixeles vecinos, luminosidad, contraste y comparaciones estructurales. Entre estas métricas se tiene el índice de calidad de imagen universal (UIQI, *Universal Image Quality Index*), su versión mejorada denominada el índice de similitud estructural (SSIM, *Structural Similarity Index*) y el índice multiescala MS-SSIM que simula diferentes resoluciones espaciales.

  Las métricas SSIM y MS-SSIM imitan bastante bien la calidad percibida por un observador humano, pero tienen algunas limitaciones en su nivel de percepción de los detalles mínimos en imágenes borrosas o ruidosas. Para resolver este problema se han creado variaciones como la versión multicomponente 3-SSIM y 4-SSIM que tienen una mayor precisión en texturas y bordes (Prieto Renieblas, Turrero Nogués, Muñoz González, Gómez Leon, & Guibelalde del Castillo, 2017).

Métrica DISTS. - La métrica DISTS mide la similitud de texturas y estructuras, es resistente a la variación de texturas y transformaciones geométricas leves. La métrica DISTS se basa en el uso de una red neuronal convolucional para construir una función inyectiva y diferenciable que transforma imágenes en representaciones sobre completas de múltiples escalas. Los promedios espaciales de los mapas de características en esta representación capturan la apariencia de la textura, ya que proporcionan un conjunto de restricciones estadísticas suficientes para sintetizar una amplia variedad de patrones de textura. Luego para determinar la calidad se combinan las correlaciones de estos promedios espaciales ("similitud de textura") con correlaciones de los mapas de características ("similitud de estructura"). El método optimizado se ajusta a las puntuaciones perceptivas humanas, tanto en bases de datos de calidad de imagen convencionales como en bases de datos de texturas. Finalmente,

se probó que el método es relativamente insensible a las transformaciones geométricas (por ejemplo, traslación y dilatación), y no requiere del uso de ningún entrenamiento especializado o aumento de datos (Ding, Ma, Wang, & Simoncelli, 2020).

2.4 AUTOMATAS CELULARES EN EL TRATAMIENTO DE IMÁGENES

Los autómatas celulares se han usado tradicionalmente para modelar sistemas físicos, pero también se pueden usar en otro tipo de sistemas complejos, como en el análisis de imágenes digitales.

2.4.1 DESARROLLO DE UN ALGORITMO ADAPTATIVO BASADO EN AUTÓMATAS CELULARES PARA ELIMINAR RUIDO EN IMÁGENES DIGITALES

En el trabajo de (Angulo Sogamoso & Gil Sierra, 2019) se usan los autómatas celulares para eliminar el ruido y detectar bordes en imágenes digitales en escala de grises. Para ello cada pixel se considera una célula rodeada de sus 8 vecinos, se contabilizan los pixeles que se consideran ruidosos. El tipo de ruido considerado es el de tipo “sal y pimienta”, ruido producido por pixeles totalmente blancos o negros. El autómata recorre la imagen determinando si el pixel es ruidoso y de ser necesario amplía su vecindario.

Para la detección de bordes se experimentó con imágenes binarizadas (solo pixeles blancos o negros) con el fin de simplificar el problema. Para ello se calcula el promedio de intensidad de los pixeles y de acuerdo a un umbral se asigna un valor de uno (1) o cero (0). Este proceso se implementó con un autómata celular siguiendo reglas similares al juego de la vida, tales como a) si una célula tiene menos de dos (2) vecinos, muere – soledad, b) si una célula tiene más de ocho (8) vecinos, muere - sobre la población, c) si una célula tiene tres (3), cuatro (4), seis (6) o siete (7) vecinos vivos, continúa viviendo – felicidad y d) si una célula tiene exactamente cinco (5) vecinos, cobra vida – reproducción. En la Figura 2.4 se muestran los resultados obtenidos con al AC comparado con otros métodos. Se concluye que, si se logra detectar los bordes incluso a pesar del ruido, con la diferencia de que los bordes resultantes son más gruesos.

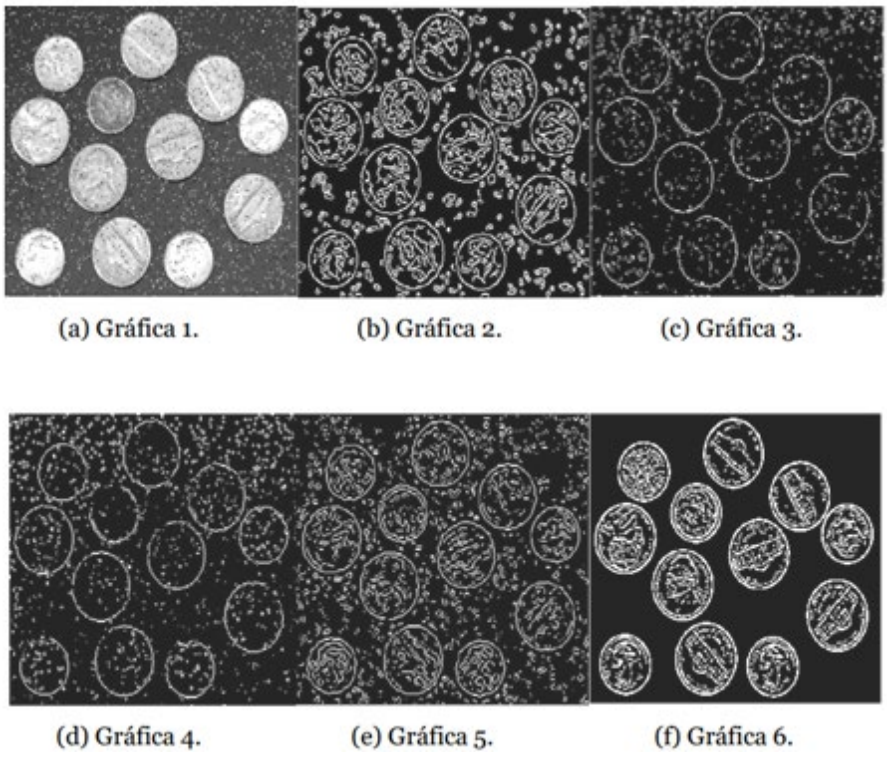


Figura 2.4: Detección de bordes con ruido al 5%, monedas.

Nota. (a) Imagen base. (b) Imagen resultante algoritmo de Canny. (c) Imagen resultante algoritmo de Prewitt. (d) Imagen resultante algoritmo de Sobel. (e) Imagen resultante algoritmo Laplaciano de Gauss. (f) Imagen resultante algoritmo de propuesto con AC.

Fuente: (Angulo Sogamoso & Gil Sierra, 2019)

2.4.2 UN METODO COOPERATIVO PARA LA SEGMENTACIÓN DE IMÁGENES BASADO EN AUTOMATAS CELULARES

En el trabajo (Parra & Perozo, 2017) se propone un método para la segmentación de imágenes a color basado en la detección de bordes usando autómatas celulares. Para la detección de bordes, las celdas del autómata deben calcular la distancia euclideana entre los componentes RGB de cada uno de los píxeles de color y de todos sus vecinos. Para determinar si existe o no un borde, la célula compara cada uno de los componentes RGB de su pixel con el de su célula vecina y si la diferencia entre los valores de alguno de los componentes supera el umbral para dicho componente, se determina la existencia de un borde. Dicha diferencia se determinó con la ecuación: $|A - B| \geq U$, donde A y B son los

valores de color RGB y U el umbral de comparación. Los resultados obtenidos se muestran en la Figura 2.5, donde a medida que se reduce el umbral se incrementa el nivel de detalle en la detección de bordes.

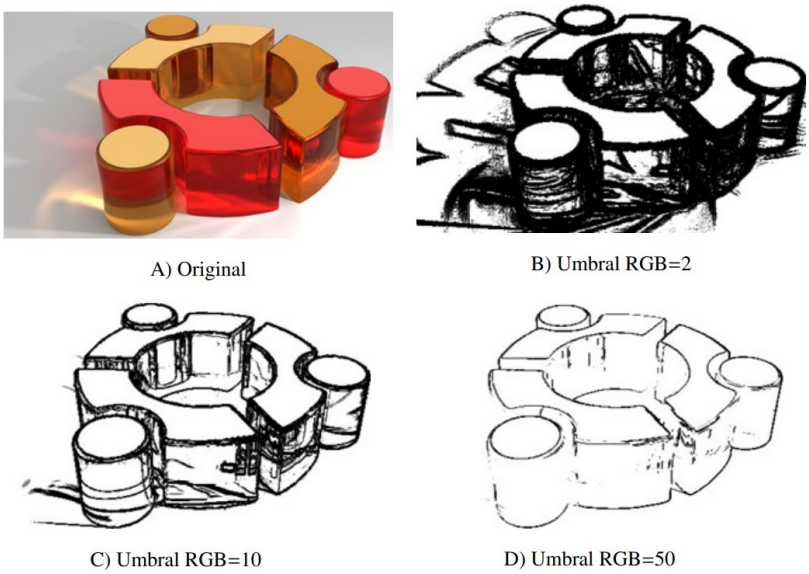


Figura 2.5: Detección de bordes en imágenes con alto nivel de detalle con umbral general igual a 2, 10 y 50.
Fuente: Adaptado de (Parra & Perozo, 2017)

2.5 AUTÓMATAS CELULARES

Un Autómata Celular es un sistema complejo adaptativo, es un modelo matemático y computacional para un sistema dinámico que evoluciona en pasos discretos. Se compone de unidades llamadas celdas, que se relacionan entre sí por una vecindad. Cada estado cumple con ciertas condiciones impuestas a la celda. (Fiol M., 2021).

Los autómatas celulares (AC) se derivan de los autómatas finitos (AF) ya que ambos utilizan una función de transición para cambiar de estados. Sin embargo, el AC incorpora el componente espacial. (Fiol M., 2021)

2.5.1 ELEMENTOS DE UN AUTÓMATA CELULAR

Según (Wikipedia, 2022) los elementos que conforman un AC son: un espacio, conjunto de estados, configuración inicial, vecindad, y reglas de transición.

- **Espacio o universo.** El espacio o universo es una estructura discreta que puede ser un vector, un plano de 2 dimensiones o un espacio n-dimensional. Donde cada espacio discreto se llama "célula". Los bordes del espacio pueden extenderse infinitamente, pero generalmente se limita su crecimiento con una frontera abierta o periódica.
- **Conjunto de estados.** Son los valores que puede tomar cada célula. Estos valores son finitos.
- **Configuración inicial.** Es el valor inicial con el cual empiezan las células del espacio.
- **Vecindad.** Es el conjunto de células que se consideran vecinas a una determinada célula. Los vecindarios más conocidos son los de Von Neumann y Moore como se muestra en la Figura 2.6.

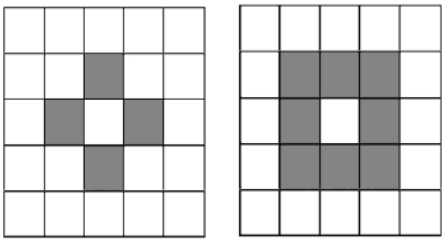


Figura 2.6: Vecindad de Von Neumann (izquierda) y vecindad de Moore (derecha).
Fuente: (Rodríguez Calvo, 2019)

- **Reglas de transición.** Son las reglas que determinan la evolución del A.C. Las reglas se definen en base a los estados y vecindad de cada célula. Las reglas se aplican de forma homogénea a todas las células

2.5.2 EL JUEGO DE LA VIDA

En 1970 el matemático británico John Horton Conway desarrollo uno de los autómatas celulares más conocidos, "el juego de la vida", *life* en inglés (Fiol M., 2021). El juego se realiza en un tablero rectangular donde cada celda representa una célula como se muestra en la Figura 2.7. Las células tienen dos estados: "viva" o "muerta" y evolucionan a lo largo de unidades de tiempo discretas. Cada celda tiene 8 vecinas. El estado de todas las

células se tiene en cuenta para calcular el estado de las mismas al turno siguiente. Todas las células se actualizan simultáneamente en cada turno (Fiol M., 2021).

Las reglas del juego de la vida son las siguientes:

- Una célula muere, si tiene solo un (1) vecino vivo (aislamiento) o más de tres (3) vivos a su alrededor (sobrepoblación).
- Una célula viva rodeada de dos (2) o tres (3) vecinos vivos se mantiene viva (supervivencia).
- Una célula nace, si tiene exactamente tres (3) vecinos vivos (nacimiento)

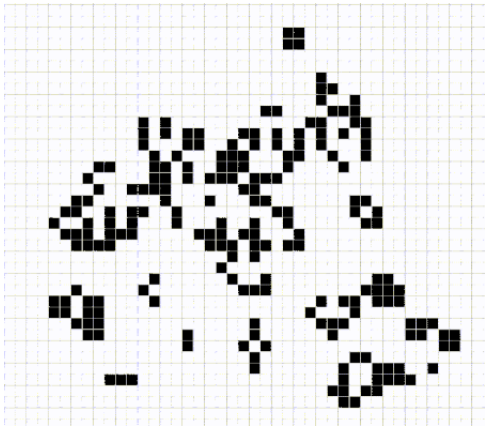


Figura 2.7: Ejemplo del autómata celular del juego de la vida de Conway.
Fuente: (González Alcón, 2020)

2.5.3 TIPOS DE COMPORTAMIENTOS EVOLUTIVOS

Stephen Wolfram realizo muchas investigaciones sobre autómatas celulares en relación a las condiciones iniciales y diferentes tipos de comportamientos periódicos, definiendo cuatro tipos de clases: (Wikipedia, 2022).

Clase I. La evolución lleva a una configuración estable y homogénea, es decir, todas las células terminan por llegar al mismo valor.

Clase II. La evolución lleva a un conjunto de estructuras simples que son estables o periódicas.

Clase III. La evolución lleva a un patrón caótico.

Clase IV. La evolución lleva a estructuras aisladas que muestran un comportamiento complejo (es decir, ni completamente caótico, ni completamente

ordenado, sino en la línea entre uno y otro, este suele ser el tipo de comportamiento más interesante que un sistema dinámico puede presentar).

2.6 REDES NEURONALES ARTIFICIALES PARA EL PROCESAMIENTO DE IMÁGENES

Una red neuronal artificial (RNA) es un modelo simplificado del procesamiento neural del cerebro humano. El estudio de las RNA es parte del área de inteligencia artificial (IA) y las bases teóricas del modelo se empezaron a desarrollar desde la década de los 40 (Wikpedia, 2022).

La visión artificial es un campo de la IA que permite que las computadoras y sistemas puedan ver, observar y comprender la información significativa de imágenes digitales, videos, y otras entradas visuales (IBM, 2022).

2.6.1 ESTRUCTURA DE UNA NEURONA ARTIFICIAL (PERCEPTRON SIMPLE)

El modelo de neurona artificial denominado "perceptron simple" fue definido por Frank Rosenblatt en 1958. Dicho modelo representa de forma muy simplificada el funcionamiento de una neurona biológica (García, 2021).

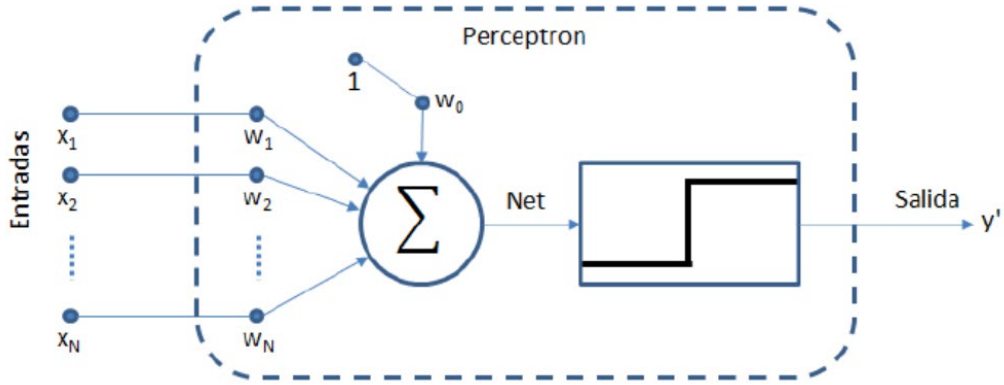


Figura 2.8: Esquema del perceptron con peso adicional w_0
Fuente: (García, 2021)

Los elementos que conforman la neurona artificial son:

- *n* entradas $x_1, x_2, \ldots x_n$ Dónde x_i son los valores de entrada que la neurona recibe.
- *n* peso $w_1, w_2, \ldots w_n$ Donde w_i son los pesos que modelan el grado de intensidad de cada entrada
- Una entrada constante de valor 1 llamada "BIAS" de peso w_0. El peso adicional w_0 modela el comportamiento que ciertas neuronas tienen, al estar más o menos activas independientemente del valor de las entradas.
- Una función de activación *f*
- Una salida *y'*

El valor total recibido por la neurona se modela con la expresión:

$$x_1 w_1 + x_2 w_2 + \cdots + x_N w_N = \sum_{i=1}^{N} x_i w_i = Net$$

A dicha expresión se le suma el peso adicional w_0 y se aplica la función de activación *f*, para obtener la salida *y'*: $f\left(w_0 + \sum_{i=1}^{N} x_i w_i\right) = y'$

La función de activación modela el proceso que la neurona hace con el valor total recibido de las entradas. Existen muchos tipos de funciones de activación que pueden usarse, como por ejemplo las funciones: sigmoide, tangente hiperbólica, relu, etc. El tipo de función de activación elegida dependerá del tipo de problema a resolver y de su arquitectura.

2.6.2 RED NEURONAL ARTIFICIAL MULTICAPA (PERCEPTRÓN MULTICAPA)

Una de las redes neuronales artificiales más exitosas son las redes neuronales con conexiones hacia adelante (*feed-forward neural network*), denominadas perceptrones multicapa (García, 2021). Donde las neuronas se organizan en capas como se muestra en la Figura 2.9 y tienen las siguientes características:

- Todas las neuronas de una capa conectan su salida a una de las entradas de cada una de las neuronas de la capa siguiente (conexiones hacia adelante). Se dice que estas redes están densamente conectadas.
- No hay conexiones hacia neuronas de capas anteriores.
- No hay conexiones con neuronas de la misma capa.

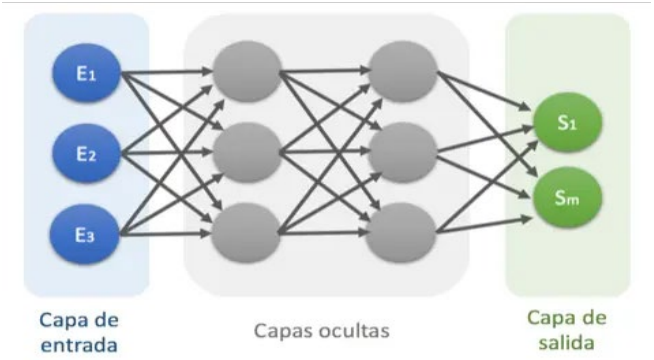


Figura 2.9: Ejemplo de red neuronal artificial multi-capas.
Fuente: (Calvo, 2018)

2.6.3 TIPOS DE APRENDIZAJE AUTOMÁTICO

El aprendizaje automático, también llamado aprendizaje de máquina (*Maching Learning*), consiste en que el propio modelo busque en los datos patrones y determine sus propias reglas para aprender por sí mismo (Recuero de los Santos, 2021). Los tipos de aprendizaje automático se clasifican en:

- Aprendizaje supervisado.- Según (Recuero de los Santos, 2021) "En el aprendizaje supervisado, los algoritmos trabajan con datos etiquetados, intentado encontrar una función que, dadas las variables de entrada, les asigne la etiqueta de salida adecuada". Es decir que por cada dato de entrada se debe conocer su atributo objetivo. Existen dos tipos de aprendizaje supervisado: De clasificación donde los objetos pertenecen a diversas clases (atributo categórico). Y de regresión donde el modelo predice un valor numérico (atributo numérico) (Vallalta Rueda, 2022).

- Aprendizaje no supervisado. - El aprendizaje no supervisado tiene lugar cuando no se dispone de datos "etiquetados" para el entrenamiento. Sólo conocemos los datos de entrada, pero no existen datos de salida que correspondan a una determinada entrada (Recuero de los Santos, 2021). El aprendizaje no supervisado se refiere a tareas de agrupamiento, también llamadas *clustering* o segmentación, donde su objetivo es encontrar grupos similares. (Vallalta Rueda, 2022)

- Aprendizaje por refuerzo. - Este tipo aprendizaje se basa en mejorar la respuesta del modelo usando un proceso de retroalimentación. El algoritmo aprende observando el mundo que le rodea. Su información de entrada es el *feedback* o retroalimentación que obtiene del mundo exterior como respuesta a sus acciones. Por lo tanto, el sistema aprende a base de ensayo-error (Recuero de los Santos, 2021).

2.6.4 APRENDIZAJE PROFUNDO

El aprendizaje profundo (*deep learning*) consiste en el uso de redes neuronales artificiales con una mayor cantidad de capas, haciendo énfasis en la capacidad del modelo de aprender en múltiples niveles. Actualmente son el estado del arte para la mayoría de las tareas de reconocimiento de patrones. Una de las características del aprendizaje profundo es la capacidad de aprender características en el mismo modelo de clasificación (García, 2021).

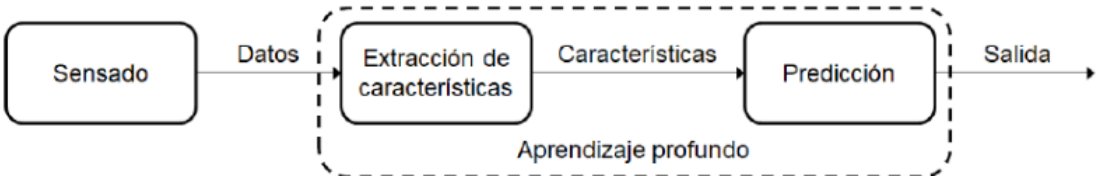


Figura 2.10: Esquema de aprendizaje profundo donde se resalta la integración de la extracción de características y la clasificación en el mismo modelo.
Fuente: (García, 2021)

Según (García, 2021) el éxito de las redes neuronales profundas se debe a los siguientes factores:

- Avance tecnológico. - Que permite entrenar modelos más grandes, y hardware para procesos en paralelo.
- Mejores funciones de activación. – Las redes neuronales de muchas capas tienen un problema que consiste en el desvanecimiento del error durante su entrenamiento con funciones de activación sigmoidal y tangente hiperbólica. Para reducir este efecto actualmente se usa la función de activación ReLU (*Rectified Linear Unit*), la cual es mucho más rápida y actualmente se tienen muchas variantes como: PReLU, Leaky ReLU, ELU.
- Nuevos modelos de redes neuronales convolucionales.

2.6.5 REDES NEURONALES CONVOLUCIONALES

Las redes neuronales convolucionales (CNN) son un caso particular de RNA donde las neuronas no están densamente conectadas, y los datos están estructurados en arreglos (*arrays*), lo cual las hace ideales para trabajar con estructuras de datos como son los pixeles de una imagen (García, 2021).

De acuerdo a (García, 2021) la arquitectura de una CNN incorpora dos tipos básicos de capas adicionales:

- Capas de convolución. – Las neuronas se organizan en mapas de características (*feature maps*) las cuales se relacionan con un conjunto de pesos llamado *kernel* o *filter bank*. Las características diferentes se relacionan y comparten *kernels* distintos. El objetivo es que cada característica diferente se relacione con determinados *kernels* para especializarse en reconocer un tipo de patrón.
- Capas de *pooling*. – Este tipo de capas no tienen pesos, y su objetivo es fusionar características similares, con lo cual se reduce la dimensión de los datos.

En los modelos de CNN es común intercalar capas de convolución y de *pooling*, y al final conectar capas de neuronas densamente conectadas, tal como se muestra en la Figura 2.11.

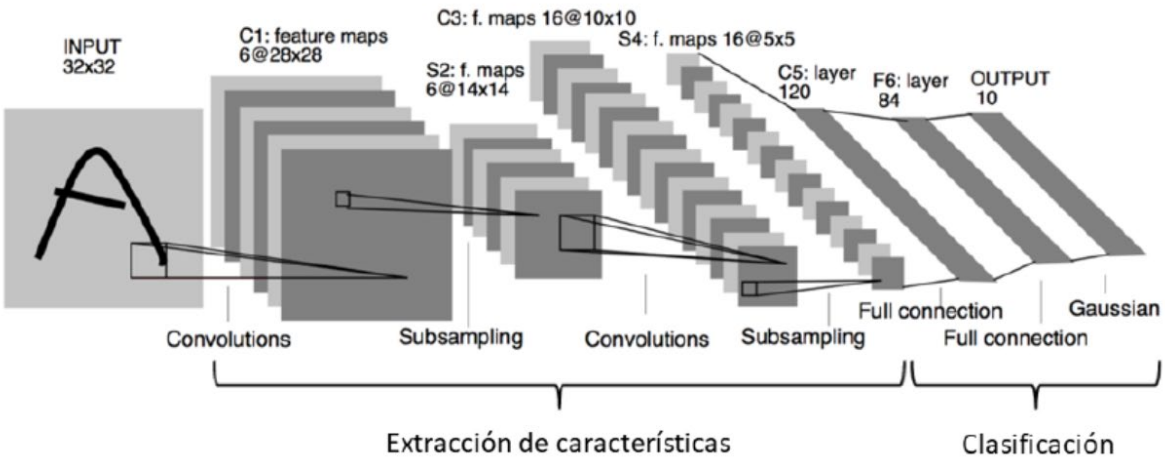


Figura 2.11: Ejemplo de CNN denominada LeNet5 para reconocimiento de caracteres, presentada por LeCun, 1988.
Fuente: (García, 2021)

Existen otras capas que ayudan a las redes neuronales convolucionales en el proceso de entrenamiento, estas son:

- Capas *dropout*.- Es un método que desactiva un número de neuronas de forma aleatoria en cada iteración de entrenamiento, lo cual obliga a las neuronas a no depender tanto de las neuronas vecinas desactivadas. Esto ayuda a reducir el sobre-aprendizaje (*overfitting*), el cual ocurre cuando neuronas vecinas empiezan a memorizar patrones fijos en lugar de generalizar. *Dropout* tiene un parámetro de 0 a 1 que indica la probabilidad de que una neurona se desactive. El valor por defecto suele ser de 0.5 (Rodríguez, 2018).
- Capas de normalización por lote. - (*batch normalization*) Es un método que normaliza cada lote de datos. Los datos de entrada de una red neuronal son generalmente normalizados por ejemplo entre 0 a 1, lo cual permite que los datos tengan distancias similares. Sin embargo, este beneficio se va perdiendo en redes profundas. Por lo tanto, para mantener este beneficio se aplica *batch normalization* a cada capa antes de que los datos pasen por la función de activación. Las capas *batch normalization* implementan dos nuevos parámetros que sustituyen al parámetro *bias* de las neuronas, por lo cual es habitual desactivar el *bias* de las neuronas en estos casos (Rodríguez, 2018).

2.6.6 RED NEURONAL CONVOLUCIONAL U-NET

U-NET es un modelo de red neuronal profundo desarrollado por Olaf Ronneberger, Phillip Fischer y Thomas Brox en 2015 para la segmentación de imágenes médicas, el cual se muestra en la Figura 2.12. Es un modelo totalmente convolucional, formado por un "codificador" y un "decodificador". El codificador reduce la dimensionalidad de la imagen y aumenta la cantidad de características. El decodificador realiza el proceso inverso es decir aumenta la dimensionalidad de la imagen y reduce la cantidad de características (Lamba, 2019).

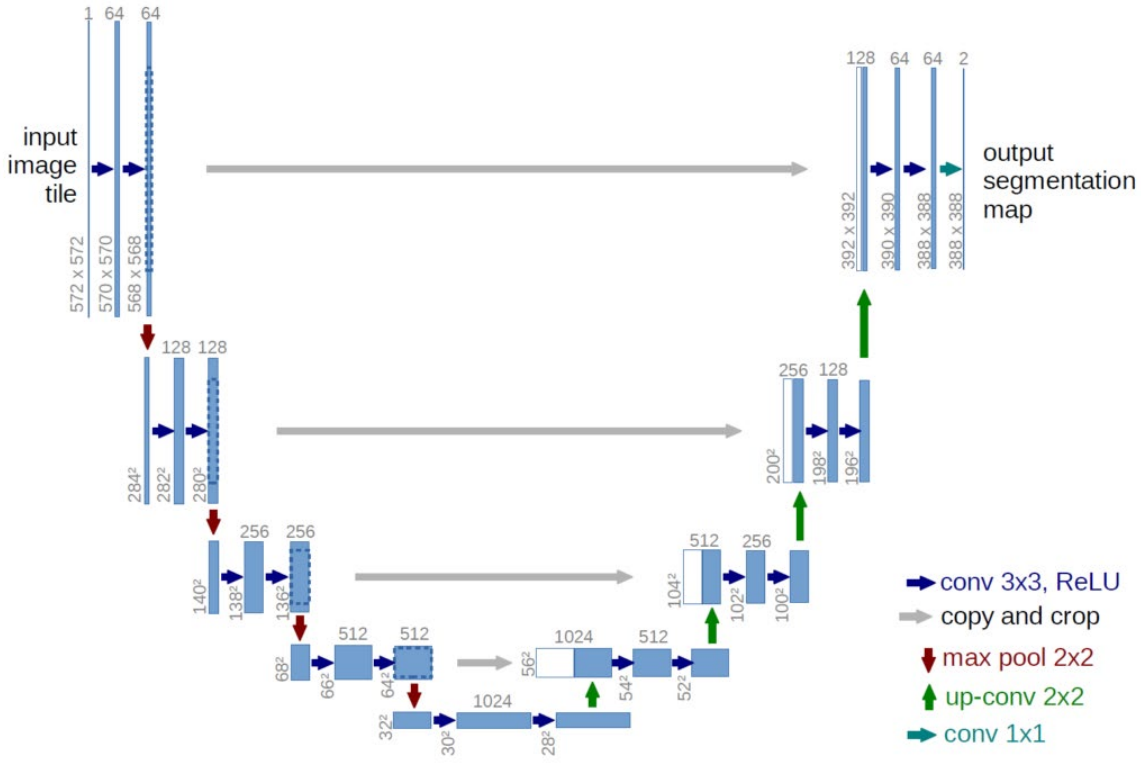


Figura 2.12: Ejemplo original de la arquitectura U-Net propuesta por Ronneberger, Fischer y Brox 2015.

Nota. Cada cuadro azul corresponde a un mapa de funciones multicanal. El número de canales se indica en la parte superior de la caja. El tamaño x-y se proporciona en el borde inferior izquierdo del cuadro. Los cuadros blancos representan mapas de características copiados. Las flechas indican las diferentes operaciones.

Fuente: (Ronneberger, Fischer, & Brox, 2015)

La arquitectura U-Net aprende con el codificador a representar la imagen como un conjunto lineal de características. Y luego el decodificador reconstruye la imagen siguiendo el proceso inverso. Durante el proceso de codificación se reduce la dimensionalidad de la imagen lo cual produce una pérdida de información irreversible y aumento de la entropía. Para resolver este problema en el modelo U-Net se tienen conexiones intermedias entre capas del codificador y decodificador. Estas conexiones intermedias (*skip connections*) permiten la comunicación directa entre capas del codificador y decodificador del mismo nivel (Rodríguez Abril, 2022).

2.6.7 REDES RESIDUALES RESNET

Múltiples estudios han demostrado que mientras más profunda sea una red neuronal artificial, mayor es la cantidad de funciones que ésta puede calcular. Sin embargo, uno de los problemas al trabajar con redes neuronales profundas es el desvanecimiento del gradiente y el sobreajuste (Rodríguez Abril, 2022). Es decir que las ultimas capas de neuronas aprendan más rápido que las primeras, ocasionando un incremento del tiempo de entrenamiento.

Para resolver este problema se utilizan conexiones de salto como en una U-NET, pero creando pequeños bloques que incorporen estas conexiones de salto. La primera red neuronal de este tipo fue llamada ResNet (*Residual Net*), propuesta por el equipo de Microsoft Research (He, Zhang, Ren, & Sun, 2016). En la Figura 2.13 se muestra un ejemplo de un bloque residual con dos capas y la conexión de salto.

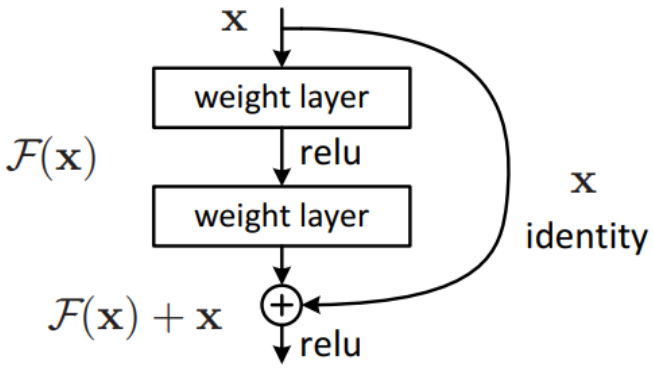


Figura 2.13: Arquitectura básica de un bloque residual con dos capas de neuronas.
Fuente: (He, Zhang, Ren, & Sun, 2016)

Los bloques residuales pueden tener una sola conexión de salto llamadas *Residual Block* (RB), tener múltiples conexiones de salto denominándose *Residual Dense Block* (RDB), o ser una combinación llamada *Residual in Residual Dense Block* (RRDB). En la Figura 2.14 se muestran ejemplos de este tipo de bloques para un modelo de aumento de resolución.

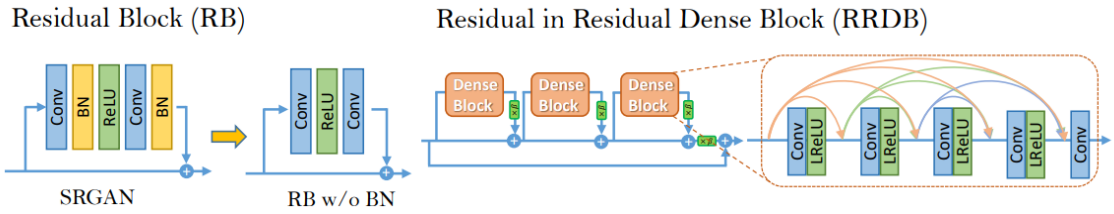


Figura 2.14: Ejemplo de bloques residuales con conexiones de salto simples y densas.
Fuente: (Xintao, y otros, 2018)

Los bloques residuales permiten que durante el entrenamiento el gradiente de la función perdida pueda retroceder directamente por medio de las conexiones de salto, evitando la degradación del bloque. Otro beneficio es que el bloque residual mezcla la información antes y después de pasar por el bloque, lo cual mejora el rendimiento en sistemas de reconocimiento visual (Rodríguez Abril, 2022).

2.6.7 MODELOS DE REDES NEURONALES GENERATIVAS

Una RNA generativa es un tipo una red neuronal de aprendizaje profundo que permite generar datos sintéticos (imágenes, textos) a partir de muy poca o ninguna información. Existen diferentes tipos de modelos generativos, entre los cuales podemos mencionar los siguientes:

- Codificador automático variacional (VAE) .- Usan una codificador (*encoder*) y un decodificar (*decoder*). Por ejemplo, entrenando este modelo con imágenes de alta calidad tanto en la entrada como en la salida, se obtiene por medio del codificador una representación comprimida de la imagen en el centro llamada espacio latente. Una vez entrenado el modelo la parte del codificar se puede descartar y usar el decodificar para generar imágenes sintéticas aleatorias (Sancho Caparrini, 2020). En la Figura 2.15 se muestra un esquema de su arquitectura.

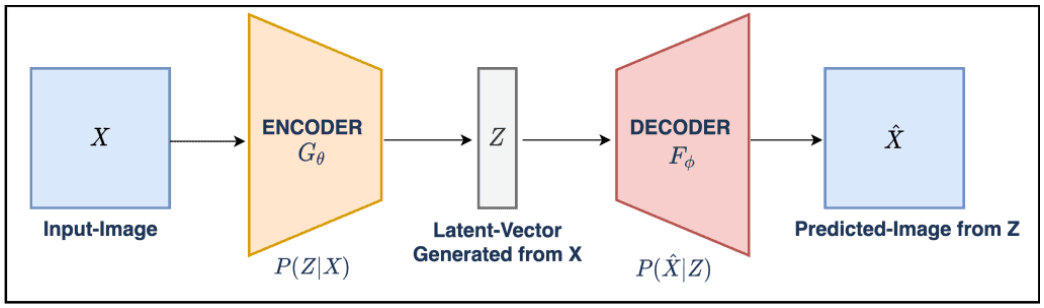


Figura 2.15: Arquitectura básica de un autocodificador variacional.
Fuente: (Sharma, 2021)

- Redes adversarias generativas (GAN). - Una red adversaria generativa GAN tiene dos bloques: un generador y un discriminador. El "generador" es un codificador automático variacional que genera datos sintéticos a partir de ruido aleatorio, y el "discriminador" se usa para comparar los datos sintéticos con los datos a los cuales se intenta ajustar el modelo. En otras palabras, los dos bloques compiten entre sí hasta que el generador sea lo suficientemente bueno como para que el discriminador no logre diferenciar entre los datos originales y lo sintéticos (Martinez Heras, Redes Neuronales Generativas Adversarias (GANs), 2020). En la Figura 2.16 se muestra dicha arquitectura.

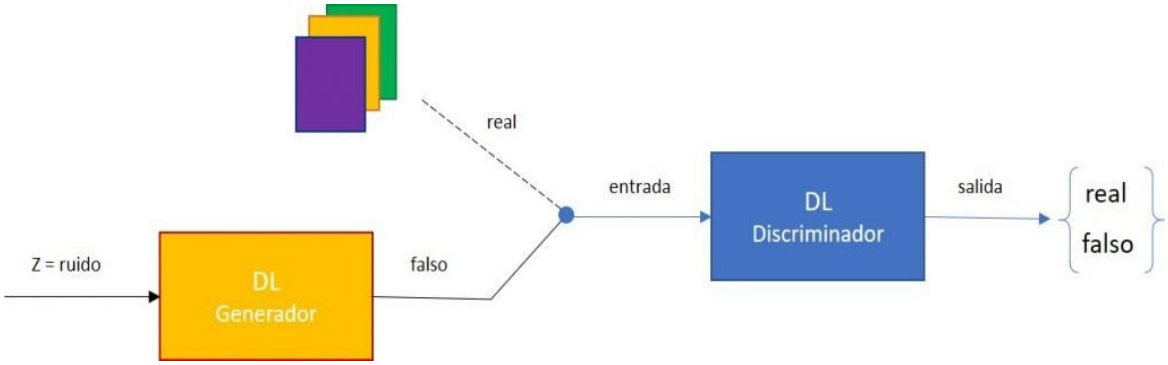


Figura 2.16: Arquitectura básica GAN.
Fuente: (Martinez Heras, Redes Neuronales Generativas Adversarias (GANs), 2020)

- Redes adversarias generativas condicionales (CGAN). - Se basan en las redes GAN, pero para el entrenamiento se condicionan con un determinado conjunto de datos. De modo que el generador obtenga salidas similares a dicho conjunto. Esta arquitectura se muestra en la Figura 2.17.

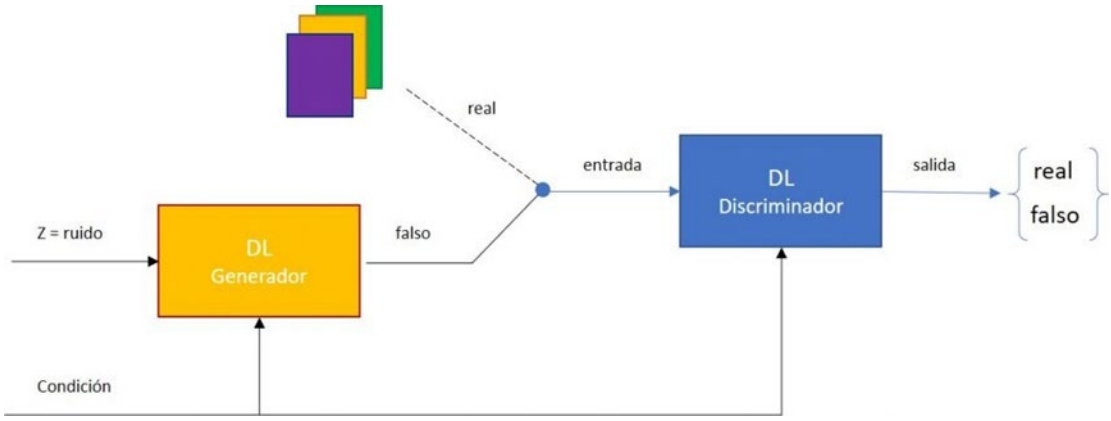


Figura 2.17: Arquitectura básica CGAN.

Fuente: (Martinez Heras, Redes Neuronales Generativas Adversarias (GANs), 2020)

El modelo Pix2Pix es un ejemplo de red generativa adversaria condicional (CGAN), el cual permite generar imágenes a partir de otro tipo de imágenes. Esto se conoce como “traducción de imagen a imagen”. El modelo Pix2Pix usa una arquitectura U-NET (Phillip, Jun-Yan, Tinghui, & Alexei A, 2018).

2.6.8 AUMENTO DE RESOLUCIÓN DE IMÁGENES USANDO REDES NEURONALES ARTIFICIALES

El aprendizaje profundo, el uso de funciones de activación más rápidas y los avances en los modelos de redes neuronales convolucionales han permitido su uso para el aumento de resolución en imágenes digitales. El término "arquitecturas poco profundas" se utiliza para referirse a redes neuronales con pocas capas. Aunque no existe un límite rígido, una red con menos de diez capas se considera poco profunda. Este tipo de arquitecturas tiene la ventaja de ser más fácil de entrenar y produce resultados superiores a los métodos tradicionales (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020).

En el primer trabajo sobre redes neuronales convolucionales (CNN) aplicadas al aumento de resolución se utilizó el término SRCNN (*Super Resolution* CNN), para indicar que son CNN aplicadas al aumento de resolución de imágenes. (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020). En este trabajo se usó el método de

mapeo entre imágenes LR y HR, una arquitectura de 3 capas, 64 filtros de 9x9, 32 filtros de 5x5, el error cuadrático medio MSE como función de pérdida y el algoritmo gradiente descendiente estocástico SGD para la optimización (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020). En la Figura 2.18 se muestra esta arquitectura.

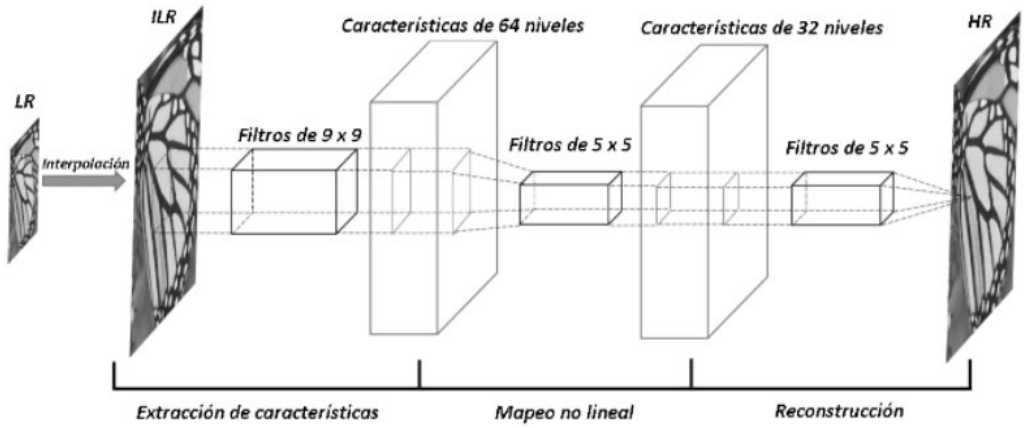


Figura 2.18: Primera arquitectura SRCNN para aumento de resolución.
Fuente: (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020)

Una red profunda tiene una arquitectura más compleja y puede lograr mejores resultados, aunque el costo computacional en la etapa de entrenamiento y ejecución aumenta considerablemente. Se considera una red muy profunda si tiene más de diez capas (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020).

En los primeros trabajos sobre redes neuronales artificiales profundas para el aumento de resolución se usó el término VDSR ("*Very Deep Super Resolution*" por sus siglas en inglés) y la red profunda y recursiva para Súper Resolución ("*Deep Recursive Convolutional Network*" DRCN por sus siglas en inglés). En la Figura 2.19 se muestra el esquema de dicha red neuronal.

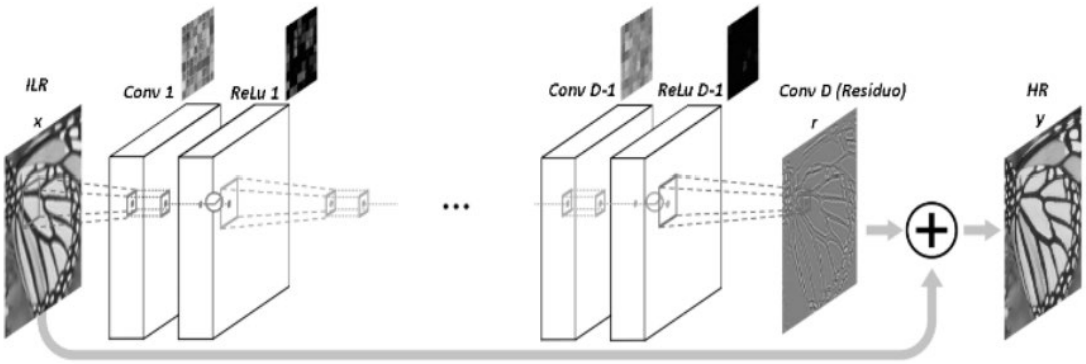


Figura 2.19: Primera arquitectura de un RNA profunda VDSR para el aumento de resolución.
Fuente: (Rodríguez Hernández, Ochoa Domínguez, Vergara Villegas, & Sossa Azuela, 2020)

Modelo SRGAN, ESRGAN y Real-ESRGAN

El modelo ESRGAN es una red neuronal convolucional para el aumento de resolución de imágenes, basado en otro modelo llamado SRGAN (*Super-Resolution Generative Adversarial Network*) e incorporando los bloques residuales RRBD (*Residual-in-Residual*) (Xintao, y otros, 2018). Los bloques residuales se usan con los bloques de convolución y permiten el uso de conexiones de salto entre bloques. En la Figura 2.20 se muestra el esquema general del modelo ESRGAN.

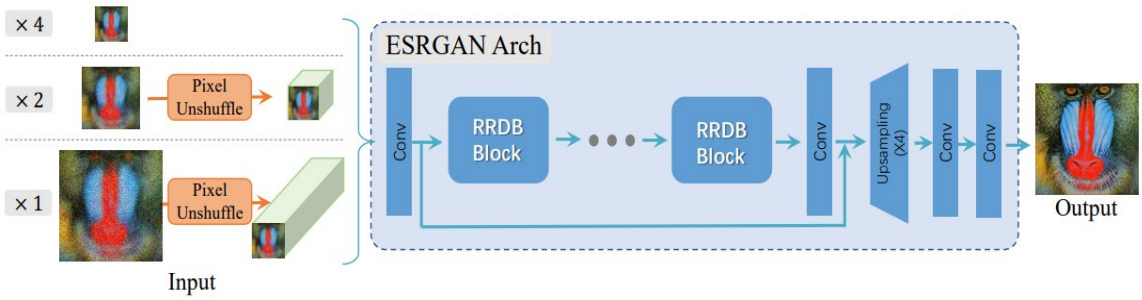


Figura 2.20: Arquitectura de modelo ESRGAN con bloques residuales RRBD.
Fuente: (Xintao, Liangbin, Chao, & Ying, 2021)

Dicho modelo fue entrenado con los conjuntos de datos: DIV2K, Flickr2k, OST, reduciendo las imágenes con muestreo reducido bicúbicamente) en parches de imágenes de 128x128 (Xintao, y otros, 2018). En la Tabla 2.1 se muestra el detalle de los conjuntos de datos y el uso de recursos para dicho modelo.

Nombre del conjunto de datos.	**Cantidad de imágenes**	**Cantidad de parámetros del modelo (en milllones)**	**Uso de GPU en el entrenamiento.**	**Tiempo entrenamiento**
DIV2K, Flickr2K OST	800 2650 300	16.6M	4 GPU V100.	4 a 5 días

Tabla 2.1: Conjuntos de datos usados, tamaño, y recursos para el entrenamiento del modelo ESRGAN/Real-ESRGAN.

Fuente: Adaptado de (Xintao, y otros, 2018) ,(Salgueiro, Marcello, & Vilaplana, 2022) y (Xintao, 2021)

El modelo ESRGAN ha tenido muchas versiones y mejoras, como por ejemplo el modelo Real-ESRGAN, el cual incorpora un modelo de degradación de imagen para tratar de simular los defectos y degradaciones complejas de imágenes tomadas del mundo real (Xintao, Liangbin, Chao, & Ying, 2021).

El modelo ESRGAN permite mejorar las imágenes en un tamaño de 4 veces, y está disponible en línea, cuaderno de GoogleColab, ejecutables para la línea de comandos y también existe un programa instalable llamado Cupscale.

Modelos de difusión

Los modelos de difusión son un tipo de modelo generativo que permite gradualmente generar imágenes de la misma forma que lo hace un GAN, pero usando un enfoque diferente (Diaz, 2022). Los modelos de difusión se entrenan en dos procesos:

- Proceso hacia delante. - A partir de una imagen inicial se añade gradualmente ruido en pequeños pasos a lo largo de una cadena de Markov. El ruido añadido sigue una distribución gaussiana.

- Proceso reverso. - Se entrena una red neuronal para que a partir del ruido gradualmente se generar la imagen usando otra cadena de Markov y la distribución inversa usada en el primer proceso. Este proceso puede ser guiado por ejemplo con etiquetas de texto para que la imagen generada pertenezca a una determinada categoría.

Los modelos de difusión logran generar imágenes de mejor fidelidad que una GAN, pero son más lentos debido a la alta secuencia de pasos para convertir el ruido en la imagen deseada. Por otro lado las GAN son más rápidas porque realizan la tarea en pocos pasos, pero corren el riesgo de desviarse mucho respecto a la imagen deseada (Diaz, 2022).

2.6.9 ETAPAS EN EL DESARROLLO Y PRUEBA DE MODELOS DE APRENDIZAJE AUTÓMATICO

Al momento de implementar un modelo de aprendizaje de maquina (*Machine Learning*), es necesario seguir una serie de fases para obtener los mejores resultados (Martinez Heras, 2020). Estás fases se basan en el estándar CRISP-DM (*Cross-industry standard process for data mining*) y se aplican de acuerdo al modelo de aprendizaje de máquina.
Entonces, para un modelo de aprendizaje de máquina se siguen las siguientes fases:

- **Fase 1: Entender el problema**. Se debe entender que se pretende predecir y que tipo de datos serian requeridos.
- **Fase 2: Recolectar y entender los datos**. Es necesario explorar los datos, contabilizarlos, visualizarlos de forma general con alguna gráfica.
- **Fase 3: Criterio de evaluación**. Lo más habitual es, separar los datos en un conjunto de entrenamiento, y otro para las pruebas.
- **Fase 4: Pre-procesamiento de datos**. Realizar todas las transformaciones necesarias para que los datos puedan ser procesados adecuadamente por el modelo. Detectar y corregir datos incompletos, normalización. Dividir los datos para el entrenamiento y pruebas.
- **Fase 5: Elección del modelo**. Se debe elegir un modelo de aprendizaje automático adecuado para resolver el problema.

- **Fase 6: Análisis de errores**. Se debe entrenar el modelo y evaluar su capacidad predictiva. En caso de que los resultados no sean aceptables, se debe analizar el ajuste de hiperparámetros, cantidad de datos, estructura del modelo, etc.
- **Fase 7: Integrar el modelo**. Se debe automatizar las tareas de construcción de datos, modelo, y pruebas. De modo que se pueda probar el modelo con nuevos datos. También se deben construir las interfaces necesarias para poder usar el modelo y hacer predicciones.

En la Figura 2.21 se muestra un esquema de estas fases.

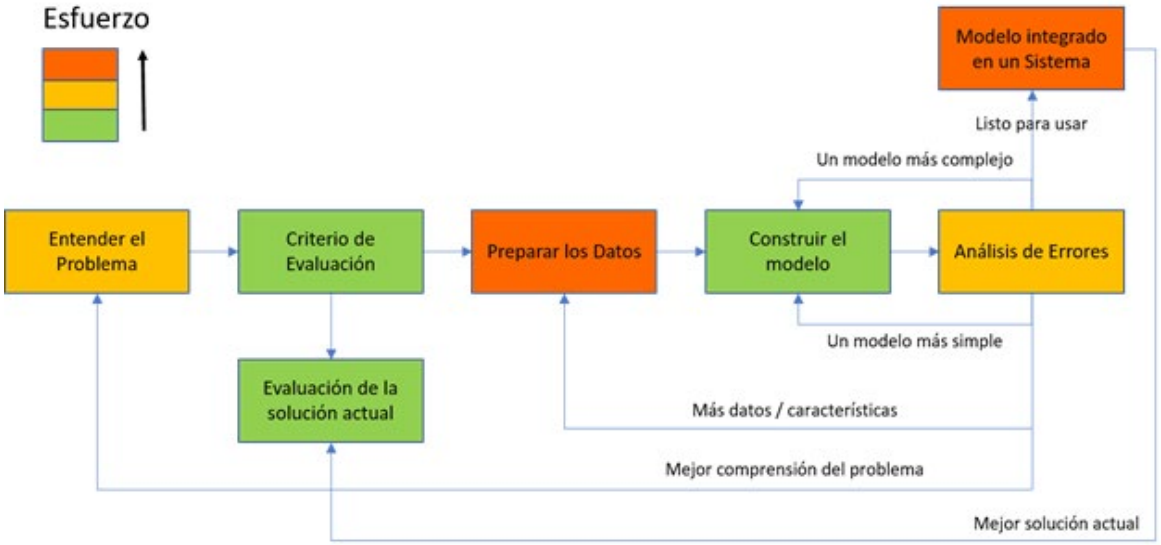


Figura 2.21: Etapas en el desarrollo de modelos de aprendizaje automático.

Fuente: (Martinez Heras, 2020)

2.6.10 CONJUNTO DE DATOS PARA PRUEBAS DE AUMENTO DE RESOLUCIÓN

Existen muchos conjuntos de datos (*Datasets*) de imágenes para probar modelos de aumento de resolución. Como por ejemplo Set5, Set14, BSD100, Urban100, etc. (Wang, 2022). La Tabla 2.2 se muestra las características de algunos de los más utilizados.

Tipo de uso	**Nombre del conjunto de datos**	**Descripción corta**
Clásicas para entrenamiento	*T91*	91 imágenes para entrenamiento
	BSDS200	200 imágenes para entrenamiento
	General100	100 imágenes para entrenamiento
Clásicas para pruebas	*Set5*	5 imágenes para pruebas
	Set14	14 imágenes para pruebas
	BSDS100	Un subconjunto de BSD500 para pruebas
	Urban100	100 imágenes de construcciones para pruebas
	Manga109	109 imágenes de manga Japonés para pruebas
	historical	10 imágenes históricas en escala de grises
2K Resolution	*DIV2K*	800 para entrenamiento y 100 para validación
	Flickr2K	2650 para entrenamiento
	DF2K	Conjunto combinado de *DIV2K* y *Flickr2K*
OST (*Outdoor scenes*)	*OST Training*	7 categorías de imágenes con texturas
	OST300	300 imágenes al aire libre par a pruebas
PIRM	*PIRM*	Conjuntos de datos de pruebas

Tabla 2.2: Descripción de los típicos conjuntos de datos usados para modelos de aumento de resolución.

Fuente: (Common Datasets for Image Super-Resolution, 2019)

Existen muchos sitios donde se puede descargarlos de forma gratuita como, por ejemplo: kaggle.com. deepai.org, etc. En los mismos sitios se tiene una descripción sobre la cantidad de imágenes y las diferentes resoluciones disponibles.

2.6.11 RECURSOS COMPUTACIONALES USADOS EN EL ENTRENAMIENTO DE REDES NEURONALES ARTIFICIALES

Entrenar un modelo red neuronal profundo requiere de considerables recursos computacionales, debido al consumo adicional de energía. Este consumo adicional se refleja por el uso de hardware especializado, como son las tarjetas gráficas GPU y TPU. Dicho hardware requiere sistemas de refrigeración que deben estar prendidos durante todo el entrenamiento. Aun con el uso de GPU el entrenamiento puede llegar a durar semanas o incluso meses. Este gasto de energía de GPU y CPU expresado en kilovatios-hora (kWh) puede calcularse con la Ecuación 2. Donde PUE igual a 1.58, es un constante promedio obtenida en el 2018 para centros de datos que representa la proporción adicional de energía para mantener la infraestructura informática, principalmente refrigeración (Strubell, Ganesh, & McCallum, 2019).

$$p_t = \frac{1.58t(p_c + p_r + gp_g)}{1000} \qquad (2)$$

Donde:

Pt: Consumo de energía en kilovatios-hora (kWh)
Pc: Consumo eléctrico de CPU en vatios
Pr: Consumo eléctrico de RAM en vatios
Pg: Consumo eléctrico de GPU en vatios
g: Número de GPUs
t: tiempo en horas de entrenamiento
PUE=1.58: Constante de uso de energía adicional por refrigeración.

La Agencia de Protección Ambiental de los Estados Unidos (EPA) proporciona la Ecuación 3 para convertir energía a emisiones estimadas de CO_2 en libras (Strubell, Ganesh, & McCallum, 2019).

$$CO_2e = 0.954p_t \qquad (3)$$

En la Tabla 2.3 se muestra el consumo eléctrico y su equivalente en emisiones de CO_2 para algunos de los modelos más usados en aprendizaje profundo.

Modelo	Hardware	GPU	Consumo energía (vatios)	Tiempo requerido por entrenamiento (Horas)	Pt Consumo total (kWh)	CO_2 (Libras)	Parámetros
$Transformer_{base}$	GPU NVIDIA P100	8 GPU	1415.78	12	27	26	65M
$Transformer_{big}$	GPU NVIDIA P100	8 GPU	1515.43	84	201	192	213M
NAS (basado en transformer)	GPU NVIDIA P100	8 GPU	1515.43	274.120	656.347	626.155	
GPT-2	TPUv3	32 chips	-	168	-	-	1542M

Tabla 2.3: Emisiones estimadas de CO2 por el entrenamiento de redes neuronales profundas.

(Strubell, Ganesh, & McCallum, 2019)

Los *transformers* son una arquitectura de red neuronal basada en codificadores y decodificadores que comprimen y descomprimen la información. Los cuales son usados en procesamiento de lenguaje natural y en visión por computadora.

NAS (*Neural Architecture Search*) es un modelo basado en Transformers que tiene como objetivo construir un modelo general de traducción automática. Dicho modelo se basa en una búsqueda exhaustiva de todas las posibles soluciones para obtener los mejores hiperparámetros (Hoz Galiana, 2020). GPT-2 es un modelo de procesamiento de lenguaje natural desarrollado por OpenAI, el cual se basa en mecanismos de autoatención y en *transformers*.

2.6.12 CONSUMO ELÉCTRICO EN COMPUTADORES INTEL CORE I7

Los procesadores Core i7 7700 tienen un consumo aproximado de 91W (vatios) a pleno rendimiento. Una tarjeta gráfica GeForce GTX 1070 consume 150W, la GeForce GTX 1060 consume 120W y las Radeon RX 580 y RX 570 consumen de 180W y 150W aproximadamente. Los módulos de memoria RAM DDR4 consumen solo 10 vatios. Los discos mecánicos HDD pueden consumir hasta 9W, mientras que los discos SSD consumen solo 2W (Gomar, 2017). Otros componentes presentes en un computador personal tienen los siguientes consumos aproximados:

- Placa base Z270: 10W
- Teclado iluminado: 8W
- Ratón: 2W
- Tres ventiladores de 120 mm: 10W

De modo que una estimación del consumo total de un computador I7 con aceleración gráfica es: 91W+180W+10W+11W+10W+8W+2W+10W=322W

Por esta razón las fuentes de alimentación deben tener un margen más alto por seguridad, en este caso con unos 500W sería suficiente. También se debe considerar que las fuentes tienen una eficiencia de solo un 75% dependiendo del modelo (Gomar, 2017).

2.7 HERRAMIENTAS TECNOLÓGICAS PARA PROCESAR IMÁGENES

2.7.1 PYTHON

Python es un lenguaje de programación usado en: aplicaciones web, desarrollo de software, ciencia de datos y en aprendizaje de máquina. El software Python se puede descargar gratis y se integra bien a todos los tipos de sistemas, y aumenta la velocidad del desarrollo (Amazon AWS, 2022).

Python cuenta con muchas librerías para procesamiento de datos, imágenes y redes neuronales artificiales:

- Matplotlib. Es una librería de Python que sirve para crear imágenes estáticas, animadas e interactivas (Matplotlib, 2022).

- OpenCV.- Es una librería e código abierto para visión por computadora. Cuenta con módulos para el análisis de imágenes y videos, permite hacer análisis, extracción y detección de características (OpenCV, 2022). Para las imágenes cuenta con métodos para la detección de bordes, filtros de desenfoque, cálculo de histogramas, reflejos, cambios de color, etc. (OpenCV, 2022)
- Numpy.- Es una biblioteca para Python que da soporte para crear vectores y matrices grandes multidimensionales. (NumPy, 2022)
- Scikit-image.- Permite comparar dos imágenes mediante dos métodos: usando el error cuadrático medio (MSE) y la similitud estructural (SSIM) (scikit-image team, 2022).
- TensorFlow.- Es una plataforma de código abierto para el aprendizaje automático. Cuenta con herramientas, bibliotecas y recursos de la comunidad que permite que los investigadores innoven con el aprendizaje automático fácilmente. TensorFlow ofrece varios niveles de abstracción. Por ejemplo, usando la API de alto nivel de Keras, que facilita el uso de TensorFlow (TensorFlow, 2022).
- Joblib, *multiprocessing*.- Permite el paralelizado de procesos en Python (Amat Rodrigo, 2020).
- Netron.- Es un visor de modelos de redes neuronales, aprendizaje profundo y aprendizaje automático (Roeder, 2022).
- Pytorch.- Es un *framework* para aprendizaje de máquina de código abierto, el cual proporciona funciones de alto nivel como el cálculo de tensores de forma similar a como lo hace NumPy.
- PIQA (PyTorch *Image Quality Assessment*).- Es una colección de métricas para la evaluación de la calidad de imágenes, implementa las métricas MSE, SSIM, PSNR, etc. (Rozet, 2022).

  DISTS_pytorch. – Repositorio del artículo de la métrica de similitud de imágenes DISTS. (Ding, Ma, Wang, & Simoncelli, 2020)
- Cupscale.- Programa que implementa el modelo ESRGAN, requiere PyTorch.

2.7.2 GOOGLECOLAB

GoogleColab (*Colaboratory*) es un servicio gratuito de Google que permite a cualquier usuario escribir y ejecutar código arbitrario de Python en el navegador. Es especialmente adecuado para tareas de aprendizaje automático, análisis de datos y educación. (Google, 2022). La versión gratuita tiene un límite de uso de GPU máximo de 12 horas dependiendo de la demanda, cantidad de usuarios, y solo se dispone de 12GB en memoria RAM.

2.7.3 ANACONDA

Anaconda es un entorno de desarrollo de código abierto para Python y R. Facilita la gestión de paquetes y configuración de entornos. Permite desarrollar proyectos de ciencia de datos usando entornos de desarrollo como Jupyter, JupyterLab, Spyder y RStudio.

2.7.4 HUGGING FACE

Hugging Face es una plataforma con más de 120000 modelos, 20000 conjuntos de datos y 50000 aplicaciones de demostración (*Spaces*), disponibles públicamente, en una plataforma en línea donde las personas pueden colaborar y crear modelos de aprendizaje de máquina. Funciona como un punto central donde se puede explorar, experimentar, colaborar y desarrollar tecnología con *Machine Learning*. Hugging Face Hub es un repositorio basado en Git que permite alojar modelos de aprendizaje automático previamente entrenados, conjuntos de datos, y espacios interactivos para demostrar el funcionamiento de los modelos directamente en el navegador (Hugging Face, 2022).

CAPÍTULO III

MARCO APLICATIVO

3.1 ESTRUCTURA GENERAL

El modelo de aumento de resolución propuesto en el presente trabajo denominado de ahora en adelante **RODIUCAN** (siglas de "*RESOLUTION OPTIMIZATION OF DIGITAL IMAGES USING CELLULAR AUTOMATA AND ARTIFICIAL NEURAL NETWORKS"),* está compuesto de los siguientes dos módulos:

- Primer módulo. - Consiste en la construcción de un autómata celular para detectar bordes y no bordes en las imágenes y así obtener una imagen simplificada.
- Segundo módulo. - Define la estructura de dos redes neuronales convolucionales para el aumento de resolución, las cuales se denominan RNA1 y RNA2.

El objetivo es usar las imágenes generadas por el autómata celular como una transformación intermedia entre dos redes neuronales. En la Figura 3.1 se muestra como aplicando el autómata celular se transformaron las imágenes de alta resolución (HR) a una representación de imágenes de estructuras de borde. Luego la red neuronal RNA1 fue entrenada para convertir la imagen de baja resolución a la imagen de estructuras de borde, la cual actuó como una transformación intermedia. Luego, la segunda red RNA2 combina la imagen de baja resolución con la imagen de bordes, con lo cual se obtuvo la imagen de alta resolución.

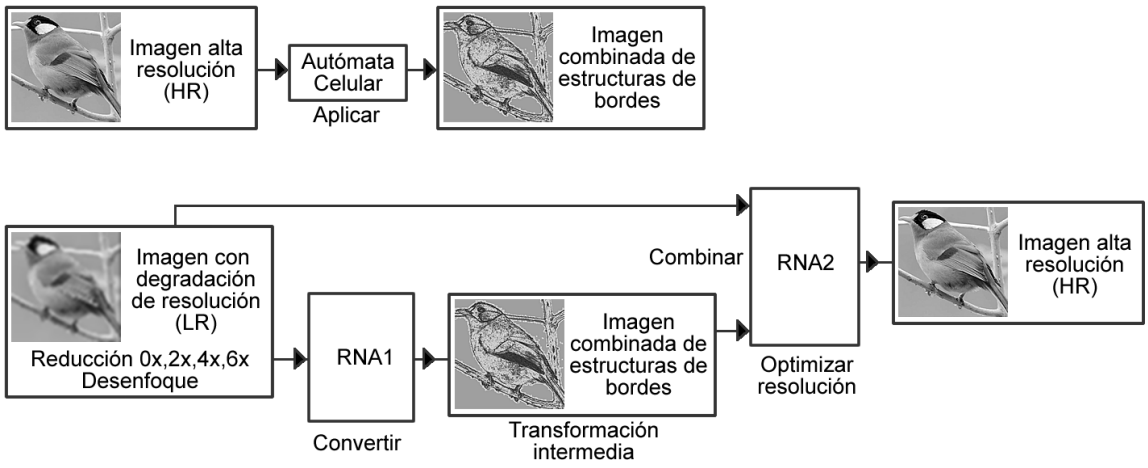


Figura 3.1: Diagrama general del modelo de optimización de resolución de imágenes.

Para simplificar el proceso de análisis, se desarrolló el modelo con imágenes en escala de grises. Una vez obtenido y probado el modelo este fue extendido para poder ser usado en imágenes a color.

3.2 MÓDULO 1: CONSTRUCCIÓN DEL AUTÓMATA CELULAR.

Una de las características fundamentales en el análisis de imágenes son los bordes, en particular en el problema de aumento de resolución los bordes se degradan al utilizar la interpolación de pixeles.

El objetivo del autómata fue detectar que pixeles podrían tener mayor probabilidad de ser bordes y cuáles no. Para ello se construyen dos autómatas, uno para detectar bordes a un nivel de detalle alto, y otro para realizar la tarea opuesta. Una vez aplicados los dos autómatas, estos se combinan en una sola imagen que describa con intensidades la probabilidad que un pixel pertenezca a un borde o no.

Ambos autómatas definieron el mismo universo, estados y reglas, solo diferenciándose en la forma de contar células vecinas similares. Por lo tanto, se definen estos aspectos generales a continuación:

Definir el universo, estados y vecindad

El universo consistió en imágenes en escala de grises, donde cada píxel corresponde a una posición (x,y) que puede estar o no ocupada por una célula del autómata. Se utilizaron dos matrices de igual tamaño:

- Matriz A para las células del autómata.
- Matriz I para los valores de intensidad de cada píxel de la imagen.

Ambas matrices se superponen de modo que por cada celda tenemos el valor correspondiente en la matrizA y en la matrizI. Los estados para cada celda de la matrizA son 0 o 255, donde 255 indica que la celda está muerta, y 0 indica que la celda está viva. Estos valores se eligen por conveniencia para poder graficar y operar la matriz. La matriz de intensidades matrizI tiene valores entre 0 a 255, correspondiente a la escala de grises de la imagen. En la Figura 3.2 se muestra un ejemplo de matrizA y matrizI, donde el pixel de intensidad 80 solo tiene dos vecinos similares de intensidad 78 y 79. La forma de medir dicha similitud entre pixeles se explica más adelante.

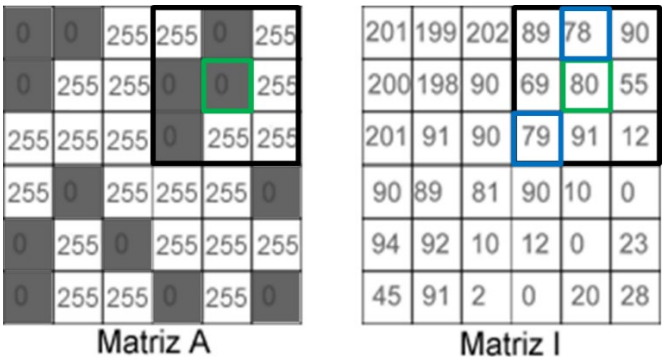


Figura 3.2: Ejemplo de matriz A de células del autómata y matriz I correspondiente a los pixeles de la imagen.

Se consideró una frontera reflectora. Es decir, si una célula estuviese junto al borde tomaría como valor de célula vecina ella misma. Las fronteras no tienen conexión entre sí, porque en las imágenes los píxeles de las fronteras no se relacionan entre sí.

Finalmente, para cada célula se consideró una vecindad de radio 1, es decir con 8 celdas a su alrededor.

Definir las reglas de transición de estados

Las reglas de transición se basaron en las reglas del "Juego de la Vida" de Conway, pero con modificaciones para considerar la matriz de intensidad de la imagen. Es decir, el autómata crece y evoluciona por el espacio, pero considerando la matriz de intensidad de la imagen para favorecer ciertos comportamientos.

En el primer autómata se requirió que las células crezcan principalmente en las zonas donde los píxeles son más heterogéneos, es decir donde existen cambios abruptos de intensidad, como son los bordes o contornos de los objetos.

Para ello dentro de la matriz de intensidades para cada celda (x,y) se contabilizaron la cantidad de vecinos similares. Para calcular la similitud entre píxeles se define un nivel de umbral de 3. Por ejemplo, en la Figura 3.3, el píxel con intensidad de 80 es similar a los píxeles vecinos de intensidad 78 y 79 porque su distancia es menor o igual que 3. En cambio, los pixeles 89,90,69,55,91 y 12 tienen una distancia mayor a 3, por lo cual no se consideran similares.

89	78	90
69	80	55
79	91	12

Figura 3.3: Para un umbral de 3, el pixel central tiene dos vecinos similares.

Las reglas de transición definidas para el autómata celular fueron las siguientes:

- Una célula viva pasa al estado muerte (aislamiento), si tiene cero (0) o un (1) vecino vivo y la cantidad de vecinos similares es mayor que 3.
- Una célula viva pasa al estado muerte (sobrepoblación), si tiene más de tres (3) vecinos vivos y la cantidad de vecinos similares es mayor que 3.
- Una célula se mantiene viva (supervivencia), si tiene dos (2) o tres (3) vecinos vivos.
- Una célula muerta pasa al estado viva (nacimiento), si tiene exactamente tres (3) vecinos vivos y la cantidad de vecinos similares es menor o igual a 3.

En resumen, las células tienden a morir en zonas donde los píxeles son muy similares, y nacen o se mantienen vivas donde hay zonas de píxeles heterogéneos. Las zonas heterogéneas corresponden en su mayoría a los contornos de los objetos en la imagen.

Definir el estado inicial del universo

En el juego de la vida el universo se inicializa de forma aleatoria, sin embargo, para el presente autómata se inicializo la matrizA del universo con una estructura de rejilla, es decir siguiendo un patrón de intercalado entre estados vivo y muerto. Esto permitió filtrar pixeles ruidosos que no forman parte de un contorno. Y también permitió que la evolución del autómata pueda converger a un estado fijo sin estructuras oscilantes. El patrón de intercalado se muestra en la Figura 3.4.

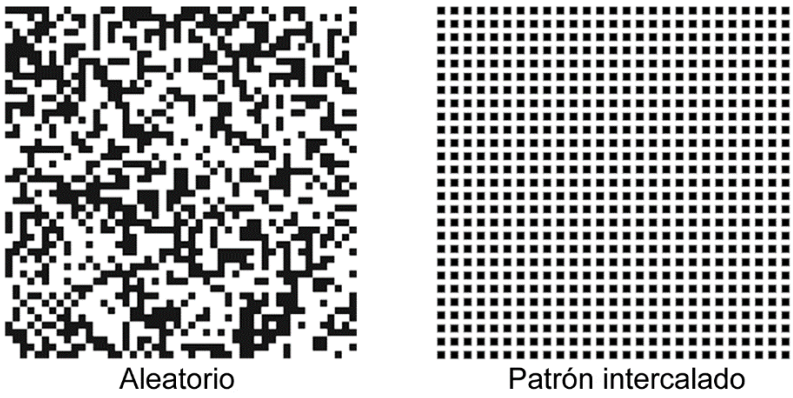


Figura 3.4: Estado inicial del universo del autómata, aleatorio y con un patrón intercalado.

Implementación del primer autómata para detectar bordes con un nivel alto de detalles

Para el autómata se realizaron pruebas con diferentes umbrales y se determinó que el valor de umbral óptimo es de 3 e iterando un total de 5 veces. Con estos valores se logran capturar los bordes a un nivel alto de detalle sin llegar a saturar la cantidad de células vivas. En la Figura 3.5 se muestra el resultado de aplicar el autómata.

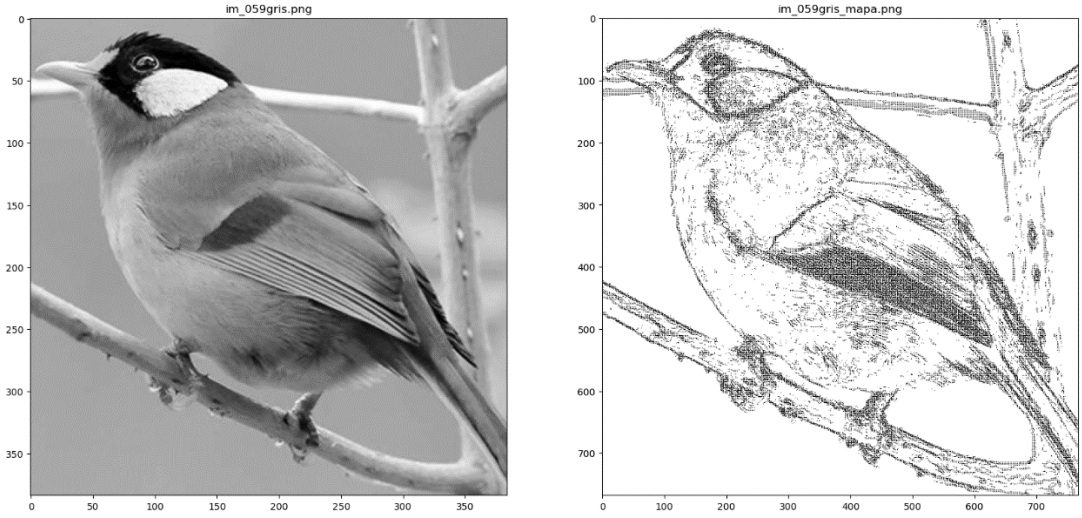


Figura 3.5: Aplicación del primer autómata para detectar bordes, con un umbral de 3, y con 5 iteraciones.

La secuencia evolutiva de imágenes generadas por el primer autómata se muestra en la Figura 3.6.

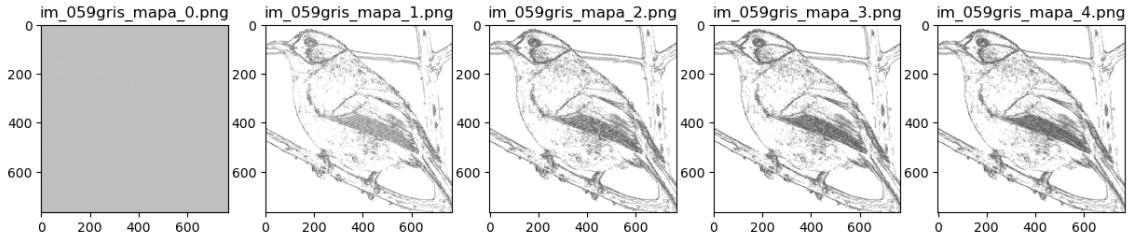


Figura 3.6: Secuencia evolutiva del primer autómata con cinco iteraciones.

Para aumentar la capacidad de detección del autómata respecto a los detalles pequeños la imagen se amplío un factor de dos veces su tamaño usando interpolación bicúbica. El pseudocódigo del primer autómata se muestra en el Algoritmo 1.

```
-----------------------------------------------------------------------
Algoritmo 1: Primer autómata para detectar los bordes.
Constantes: VG_VIVO=0, VG_MUERTO=255
Entradas: matrizI imagen en escala de grises.
Salidas: matrizA imagen generada por el autómata
-----------------------------------------------------------------------
matrizI=duplicar tamaño de matrizI
alto=obtener alto en pixeles de matrizI
ancho=obtener ancho en pixeles de matrizI
for y=0 : y < alto
    for x=0 : x < ancho
```

```
        if y es par o x es par
            matrizA[y,x]=255
        else
            matrizA[y,x]=0
for r=0 : r < 5
    cop_matrizA= copiar matrizA
    for y=0 : y < alto
        for x=0 : x < ancho
            numsimilares=contarVecinasSimilares(matrizI[y,x], umbral=3)
            numvivas=contarVecinasVivas(matrizA[y,x])+numsimilares
            if matrizA[y,x]==VG_VIVO
                if numvivas<2
                    if numsimilares>3
                        cop_matrizA[y,x]=VG_MUERTO
                else
                    if numvivas==2 o numvivas==3

                    else
                        if numsimilares>3
                            cop_matrizA[y,x]=VG_MUERTO
            else
                if numvivas==3
                    if numsimilares<=3
                        cop_matrizA[y,x]=VG_VIVO
    matrizA= copiar cop_matrizA
guardar matrizA como archivo .png
-----------------------------------------------------------------------
```

Implementación del segundo autómata para detectar lo que no es borde.

El primer autómata detecto las estructuras que coinciden con los bordes de los objetos, pero también es necesario detectar las estructuras que no son bordes. Para ello simplemente modificamos el algoritmo del primer autómata para invertir el conteo de vecinas similares, de manera que el autómata crezca en las zonas que no sean bordes.

Para ello reemplazamos la línea de código:

```
numsimilares=contarVecinasSimilares(matrizI[y,x], umbral=3)
```

por la línea:

```
numsimilares=8 - contarVecinasSimilares(matrizI[y,x], umbral=3)
```

Donde 8 es la cantidad de vecinos, de modo que al hacer la resta se contabilizan los vecinos no similares. En la Figura 3.7 se muestra el resultado obtenido.

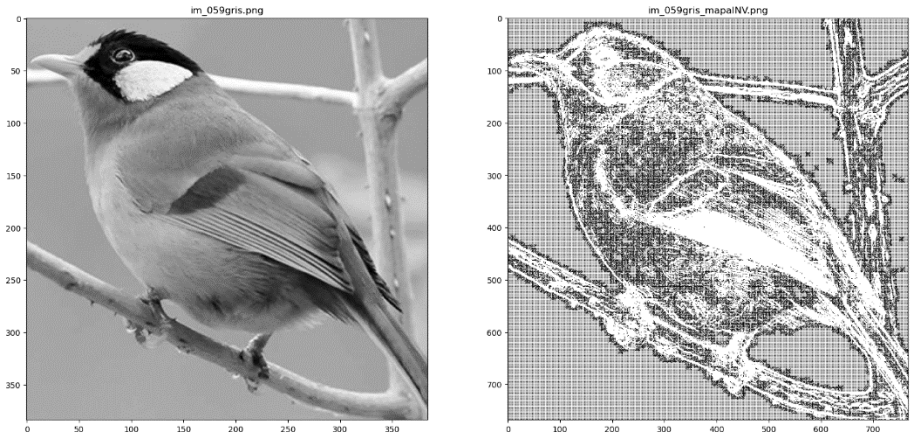


Figura 3.7: Aplicación del segundo autómata para detectar lo que no son bordes, con un umbral de 3, y con 5 iteraciones

La secuencia evolutiva de imágenes generadas por el segundo autómata se muestra en la Figura 3.8.

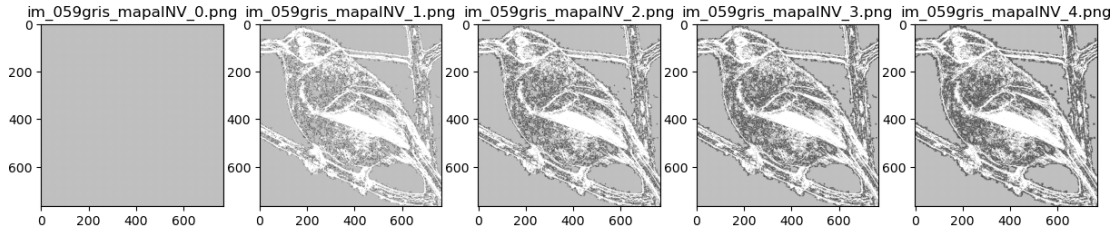


Figura 3.8: Secuencia evolutiva del segundo autómata con cinco iteraciones

El pseudocódigo del segundo autómata se muestra en el Algoritmo 2.

```
-------------------------------------------------------------------------
Algoritmo 2: Segundo autómata para detectar los no bordes.
Constantes: VG_VIVO=0, VG_MUERTO=255
Entradas: matrizI imagen en escala de grises.
Salidas: matrizA imagen generada por el autómata
-------------------------------------------------------------------------
matrizI=duplicar tamaño de matrizI
alto=obtener alto en pixeles de matrizI
ancho=obtener ancho en pixeles de matrizI
for y=0 : y < alto
    for x=0 : x < ancho
        if y es par o x es par
            matrizA[y,x]=255
        else
            matrizA[y,x]=0
for r=0 : r < 5
    cop_matrizA= copiar matrizA
    for y=0 : y < alto
        for x=0 : x < ancho
```

```
            numsimilares=8 - contarVecinasSimilares(matrizI[y,x],
umbral=3)
            numvivas=contarVecinasVivas(matrizA[y,x])+numsimilares
            if matrizA[y,x]==VG_VIVO
                if numvivas<2
                    if numsimilares>3
                        cop_matrizA[y,x]=VG_MUERTO
                else
                    if numvivas==2 o numvivas==3

                    else
                        if numsimilares>3
                            cop_matrizA[y,x]=VG_MUERTO
            else
                if numvivas==3
                    if numsimilares<=3
                        cop_matrizA[y,x]=VG_VIVO
    matrizA= copiar cop_matrizA
guardar matrizA como archivo .png
-----------------------------------------------------------------------
```

Combinar los resultados de los dos autómatas en una sola imagen

Las imágenes generadas por los dos autómatas describen los pixeles que son bordes y los que no. La imagen generada por el segundo autómata no fue el complemento exacto del resultado del primer autómata. Es decir, se observó que existieron pixeles que no correspondían exactamente con un borde. Entonces es necesario analizar ambas imágenes y determinar la probabilidad de que un pixel sea o no borde, con el fin de combinar ambas imágenes.

Se consideró un intervalo de 0 a 1, donde 0 indica que es un borde y 1 indica que no es un borde. Llamemos imgA al resultado obtenido por el primer autómata y llamemos imgB al resultado obtenido por el segundo autómata. Luego se compararon ambas imágenes pixel a pixel y se contabilizaron las células vivas alrededor de su vecindad correspondiente, con el siguiente proceso:

- Si la cantidad de células vivas en imgA es mayor a la cantidad de células vivas en imgB, entonces el pixel es un borde y se le asigna un valor cercano a 0
- Si la cantidad de células vivas en imgA es menor a la cantidad de células vivas en imgB, entonces el pixel no es un borde y se le asigna un valor cercano a 1

- Si la cantidad de células vivas en imgA es igual a la cantidad de células vivas en imgB, entonces el pixel tiene la misma probabilidad de ser borde o no serlo, entonces se le asigna un valor de 0.5

Después de aplicar este proceso a todos los pixeles se obtuvo una matriz que indica la probabilidad de que un pixel sea borde (valor cercano a 0) o no sea borde (valor cercano a 1). Las ecuaciones 4, 5 y 6 expresan el proceso para el cálculo de probabilidad que debe aplicarse a cada pixel de las imágenes imgA y imgB.

$$numvivasA = contarVecinaVivas(imgA[x, y]) \qquad (4)$$

$$numvivasB = contarVecinasVivas(imgB[x, y]) \qquad (5)$$

$$probabilidad[x, y] = \frac{\left(\frac{numvivasB}{8}\right) - \left(\frac{numvivasA}{8}\right) + 1}{2} \qquad (6)$$

Para poder visualizar y guardar estas probabilidades como una imagen se la multiplica por 255 y se descartan los decimales con la ecuación 7.

$$imagen[x, y] = int(probabilidad[x, y] * 255) \qquad (7)$$

Luego, como las imágenes imgA e imgB tienen el doble de tamaño, entonces la imagen combinada también tiene un doble tamaño, entonces esta se redujo en un factor de dos usando interpolación bicúbica para de esta forma regresar al tamaño original. En la Figura 3.9 se observa el resultado de la combinación y conversión a escala de 0 a 255.

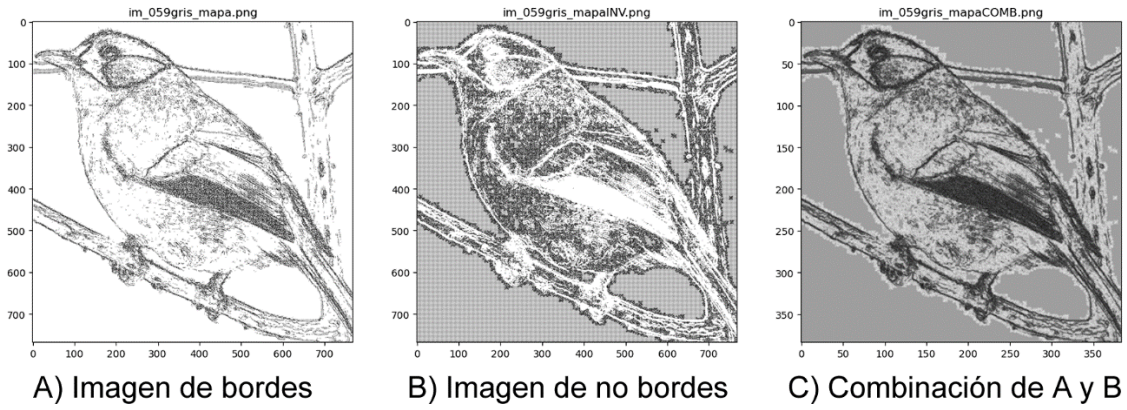


Figura 3.9: Combinación de las dos imágenes generadas por los autómatas en una sola imagen.

Para el caso de imágenes a color se aplicaron todos los pasos anteriores a cada uno de los tres canales RGB de la imagen a color, el resultado obtenido se muestra en la Figura 3.10.

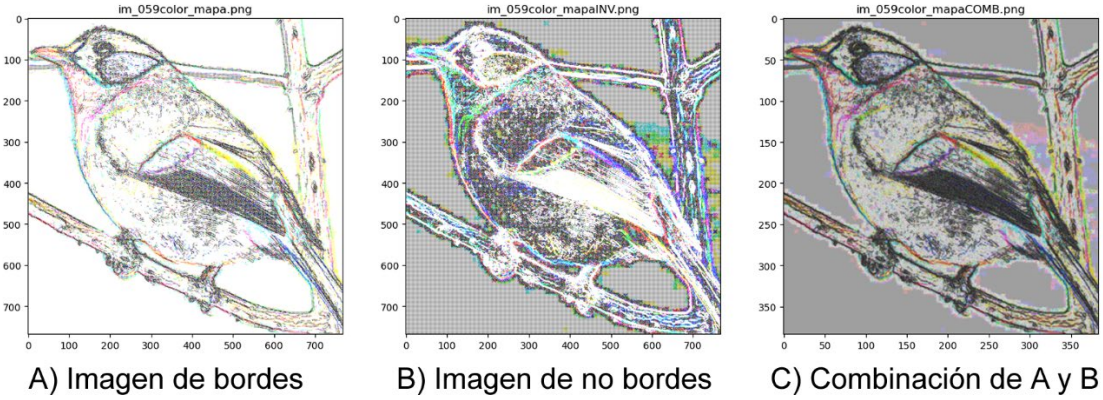


Figura 3.10: Combinación de las dos imágenes a color generadas por los autómatas en una sola imagen a color.

Los autómatas mostraron ser efectivos para la detección de bordes en imágenes de alta calidad. Sin embargo, en imágenes degradadas o difusas la capacidad del autómata se reduce. Por lo tanto, se desarrolló una red neuronal pueda realizar la misma tarea que el autómata, pero de forma generalizada, tanto para imágenes de alta calidad como para imágenes degradadas.

Entonces el autómata se aplicó a las imágenes de alta resolución para formar un conjunto de datos de imágenes de estructuras de bordes que fueron usadas en el siguiente módulo.

3.3 MÓDULO 2: CONSTRUCCIÓN DEL MODELO DE RED NEURONAL.

Entender el problema. El modelo recibe como entradas imágenes con diferentes niveles de degradación de resolución y como salida debe obtener imágenes de alta calidad. Se trata de un modelo predictivo. Los datos requeridos son imágenes en escala de grises y de tamaño variable.

Recolectar y entender los datos. Para el entrenamiento del modelo se consideró un conjunto de datos clásico para problemas de aumento de resolución denominado “General100” el cual consta de 100 imágenes con una resolución promedio de 500x500 pixeles. Considerando que:

- Las imágenes están en colores y muestran objetos variados como ser: animales, flores, ropa, personas, edificios y fondos.
- Las imágenes tienen un brillo y contraste similares.
- Las imágenes son de objetos en primer plano donde se aprecian claramente los bordes y detalles de los objetos.
- Cada imagen será segmentada en parches de 128x128 pixeles.

Criterio de evaluación. El conjunto de datos fue separado en 50% para el entrenamiento y 50% para las pruebas. De los datos de entrenamiento se tomó un 20% para realizar la evaluación y determinar la cantidad óptima de épocas para que el modelo reduzca su error sin llegar a un sobre-aprendizaje.

Una vez entrenado el modelo se realizaron dos pruebas: la primera fue aplicar el modelo a las imágenes de prueba y comparar el resultado con las imágenes originales de alta resolución. La segunda evaluación se hizo con seis diferentes conjuntos de datos: Set5, Set14, historical, BSD100, Urban100, Manga109. Y estos datos también fueron evaluados con el modelo ESRGAN para tener una medida de comparación con el modelo propuesto.

Para todas las pruebas se consideraron 4 métricas de similitud de imágenes: MSE, SSIM, PSNR y DISTS.

Pre-procesamiento de datos. Se pre-procesaron las imágenes para que puedan ser utilizadas adecuadamente por el modelo. Para ello se transforman las imágenes y se guardaron para formar un nuevo conjunto de datos de imágenes siguiendo estos pasos:

- Se aplicó la estrategia de aumento de datos reflejando las imágenes vertical y horizontalmente, para triplicar la cantidad de imágenes durante el entrenamiento. Es importante realizar este paso antes, para luego aplicar el autómata a todas las imágenes incluidas las reflejadas.
- Se redimensionaron el ancho y alto de cada imagen de modo sea un múltiplo de 128 píxeles.
- Se convirtieron las imágenes a escala de grises.

- Se redujo la calidad de la imagen en un factor de 2x, 4x y 6x usando interpolación bicúbica.
- Se redujo la calidad de la imagen en un factor de 2x, 4x y 6x usando interpolación vecinos más cercanos.
- Se redujo la calidad de la imagen aplicando un filtro de desenfoque.
- Se aplicó el autómata celular para detección de bordes a las imágenes de alta calidad en escala de grises y a color.
- Se aplicó el autómata celular para detección de no bordes a las imágenes de alta calidad en escala de grises y a color.
- Se combinaron las dos imágenes de los autómatas.
- Para todas las entradas y salidas de las redes neuronales, las imágenes fueron segmentadas en parches cuadrados de 128 píxeles.

Elección del modelo. El modelo utilizado se basó en redes neuronales convolucionales con una arquitectura de tipo U-Net.

Se probaron varias formas de uso de redes U-Net hasta lograr la mejor configuración. Una de las primeras configuraciones se muestra en la Figura 3.11, donde aplicando el autómata celular se transforman las imágenes de alta resolución (HR) a una representación de imagen de estructuras de borde. Luego el diseño de la red consistió en una primera red neuronal de una U-Net llamada RNA1 la cual convierte las imágenes de baja calidad (LR) a la imagen de bordes. Luego, las imágenes generadas por la RNA1 se usaron como entradas para la segunda red neuronal RNA2, la cual se usó para generar la imagen de alta resolución.

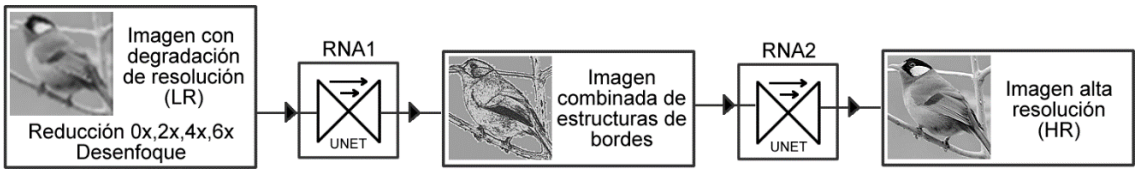


Figura 3.11: Primer diseño de las redes neuronales de optimización de resolución de imágenes.

Esta primera configuración no logro resultados precisos para el optimizando de la imagen de baja resolución hacia una imagen de alta resolución. Las imágenes generadas tuvieron una estructura similar a la original, pero carecían de texturas e iluminación.

Por ello se mejoró este primer diseño para complementar la información requerida por la segunda red RNA2. En este nuevo diseño la segunda red RNA2 formada por una U-Net combino la imagen de baja resolución con la imagen obtenida por la primera red RNA1. Este diseño se muestra en la Figura 3.12.



Figura 3.12: Segundo diseño de las redes neuronales de optimización de resolución de imágenes.

El segundo diseño permitió que la red RNA2 tenga la suficiente información para realizar la optimización de la imagen hacia una de alta resolución. Una vez entrenado el modelo la red RNA2 combino las dos fuentes de información con el objetivo final del aumento de resolución de imagen.

Implementación del modelo

La primera red neuronal recibió como entrada parches de 128x128 pixeles de imágenes con diferentes niveles de degradación de calidad, y en todos los casos las salidas fueron los parches de imagen de estructuras de borde. Estas entradas y salidas de parches se muestran en la Tabla 3.1.

Entrada	Salidas
Parche de imagen con reducción de 6x	Parche de imagen combinado de estructuras de borde.
Parche de imagen con reducción de 4x	Parche de imagen combinado de estructuras de borde.
Parche de imagen con reducción de 2x	Parche de imagen combinado de estructuras de borde.
Parche de imagen con desenfoque	Parche de imagen combinado de estructuras de borde.
Parche de imagen alta calidad	Parche de imagen combinado de estructuras de borde.

Tabla 3.1 Parches de imágenes usados como entrada y salida para entrenar la primera red neuronal

En la Tabla 3.1 se hicieron corresponder como entrada los parches de alta calidad, baja calidad 0x, 2x, 4x, 6x, desenfoque y como salida los parches de imagen combinados de estructuras de borde. Con lo cual se logró que la red neuronal generalice todas las posibles entradas y genere como salidas el parche de imagen de estructuras de borde.

Las capas que conformaron la red neuronal tomaron una arquitectura U-Net, formada por un codificador y un decodificador, donde la imagen de entrada va reduciendo sus dimensiones al mismo tiempo que aumentan las características, hasta llegar a una representación reducida de un vector de dimensión 192, luego el decodificador realiza el proceso inverso para obtener la imagen de salida. Las dimensiones y cantidad de características fueron obtenidas experimentando a partir de la estructura original U-Net. Considerando que con más dimensiones y características la cantidad de parámetros crece considerablemente, se buscó una arquitectura óptima que no supere los 6 millones de parámetros por cada U-Net, esta arquitectura se muestran en la Tabla 3.2.

Secuencia de reducción y aumento de dimensión del parche de imagen.		**Secuencia de características**		**Parámetros**
Codificador	**Decodificador**	**Codificador**	**Decodificador**	
128x128 a 64x64 64x64 a 32x32 32x32 a 16x16 16x16 a 8x8 8x8 a 4x4 4x4 a 2x2 2x2 a 1x1	1x1 a 2x2 2x2 a 4x4 4x4 a 8x8 8x8 a 16x16 16x16 a 32x32 32x32 a 64x64 64x64 a 128x128	72x72, 80x80, 96x96, 112x112, 128x128, 160x160, 192x192,	192x192, 160x160, 128x128, 112x112, 96x96, 80x80, 72x72	4.6 millones

Tabla 3.2 Arquitectura de dimensión y cantidad de características de las redes U-Net utilizadas

Para la primera red se usó la función de activación LeakyReLU para el codificador y ReLU para el decodificador excepto en la última capa donde se usó la función tangente hiperbólica. Los parches de imagen en la entrada y salida fueron normalizados en el rango de -1 a 1. El tamaño del *kernel* de las capas de convolución fue de 4x4. Como un elemento regularizador se hizo un *dropout al 30%* es decir que se desactivaron aleatoriamente conexiones de las neuronas de las tres capas centrales del decodificador. La

implementación de la red dio un valor de 4.6 millones de parámetros. La descripción del tipo de capas para esta primera red neuronal se muestra en la Figura 3.13.

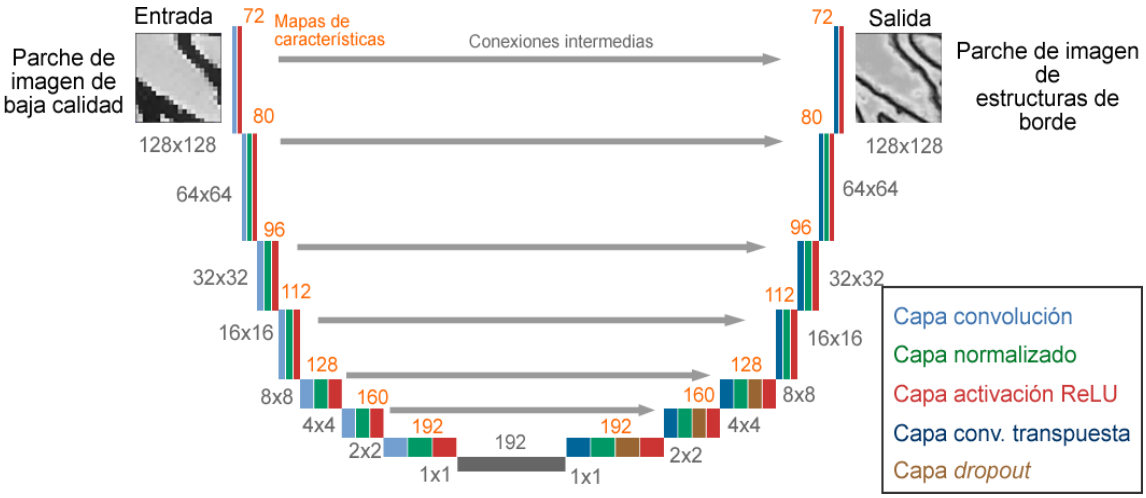


Figura 3.13: Arquitectura de la primera red neuronal convolucional.

La segunda red neuronal recibió como entrada parches de 128x128 pixeles de imágenes de baja calidad y la predicción del parche combinado de estructuras de la primera red neuronal. Y como salida el parche de imagen de alta calidad correspondiente.
Para ello se construye el conjunto de parches como se muestra en la Tabla 3.3.

Entrada		**Salida**
Parche de imagen con reducción de 6x	Predicción de parche de imagen de estructuras de borde.	Parche de imagen de alta calidad
Parche de imagen con reducción de 4x	Predicción de parche de imagen de estructuras de borde.	Parche de imagen de alta calidad
Parche de imagen con reducción de 2x	Predicción de parche de imagen de estructuras de borde.	Parche de imagen de alta calidad
Parche de imagen con desenfoque	Predicción de parche de imagen de estructuras de borde.	Parche de imagen de alta calidad
Parche de imagen alta calidad	Predicción de parche de imagen de estructuras de borde.	Parche de imagen de alta calidad

Tabla 3.3 Parches de imágenes usados como entrada y salida para entrenar la segunda red neuronal

En la Tabla 3.3 se hicieron corresponder como entrada los parches degradados en resolución 0x, 2x, 4x, 6x, desenfoque y sus respectivas predicciones de parches de estructuras, y como salida los parches de imagen de alta calidad. La segunda red neuronal logro generalizar todas las posibles entradas incluido la predicción de parche de la primera red neuronal y combinarlas para obtener como salida el parche de imagen de alta calidad.

Los parches de estructuras brindaron la información sobre los bordes y los parches de baja resolución suministraron la información para complementar la información de brillo y textura. La implementación obtuvo un valor de 4.6 millones de parámetros. En la Figura 3.14 se muestra la especificación del tipo de capas de esta segunda red neuronal.

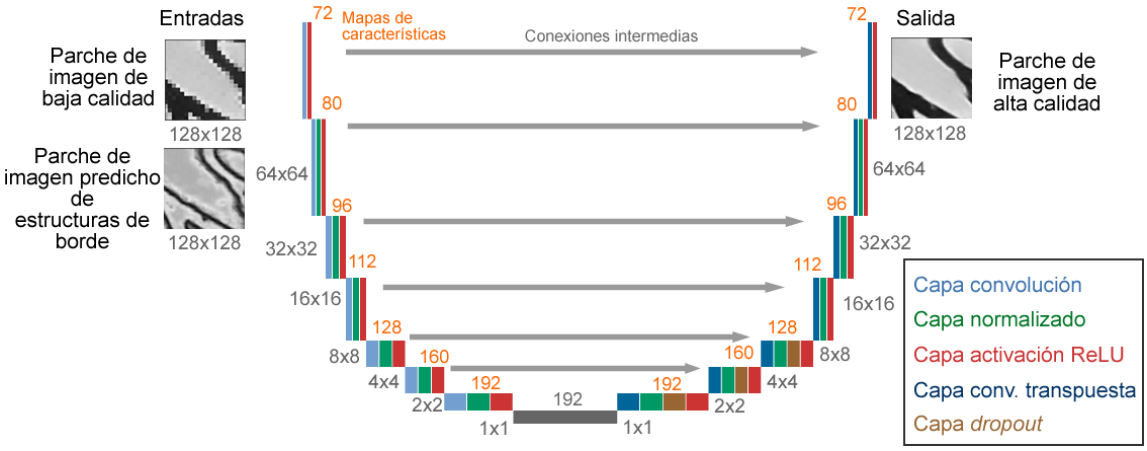


Figura 3.14: Arquitectura de la segunda red neuronal convolucional.

Las capas que forman esta segunda red neuronal también usaron una arquitectura U-Net y utilizan las mismas funciones de activación, estructuras de capas y normalización de imágenes usadas en la primera red neuronal.

Entrenamiento de las redes neuronales

La estrategia de entrenamiento fue dividida en dos fases: la primera consistió en un entrenamiento por separado de cada red neuronal, la segunda consistió en realizar un entrenamiento combinado de ambas redes. En la primera fase se usaron los parches con desenfoque y reducidos con interpolación bicúbica. En la segunda fase se perfeccionó el modelo incorporando más defectos, usando parches con desenfoque, reducidos con interpolación bicúbica y con interpolación de vecinos más cercanos. También se

incorporaron los datos aumentados, es decir el reflejo horizontal y vertical de las imágenes.

Primera fase: Entrenamiento por separado de las redes neuronales

El objetivo de esta fase fue entrenar por separado las redes neuronales para obtener de este modo una primera aproximación a la solución tratando de no caer en un sobreajuste. Para ello se utilizaron 4395 parches de 128x128 píxeles obtenidos de las imágenes reducidas un factor de 0x, 2x, 4x, 6x con interpolación bicúbica y con desenfoque. El valor óptimo respecto a la cantidad de épocas de entrenamiento fue de 30 para ambas redes.

Se usó el optimizador "Adam" con una tasa de aprendizaje de 0.001, y de entre todas las versiones disponibles en tensorflow se usó la del paquete: tf.keras.optimizers.legacy.Adam, con la cual se obtuvieron los mejores resultados.

Como función de pérdida se usó el error cuadrático medio. En la Figura 3.15 se observa que la primera red neuronal obtuvo un valor de la función de pérdida de 0.0448 para los datos entrenamiento y 0.0473 para los datos de validación. Y la segunda red neuronal obtuvo un valor de la función de pérdida de 0.0114 para los datos entrenamiento y 0.0128 para los datos de validación

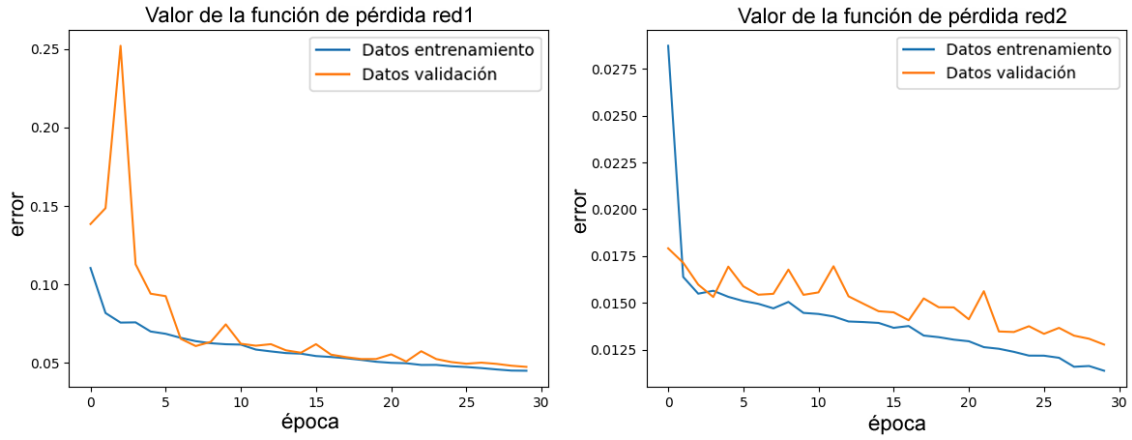


Figura 3.15: Función de pérdida de la primera y segunda red neuronal durante el entrenamiento individual.

En la Figura 3.16 se muestra el resultado de aplicar la primera red neuronal, donde se observa que los bordes fueron detectados y reconstruidos con líneas suaves y definidas.

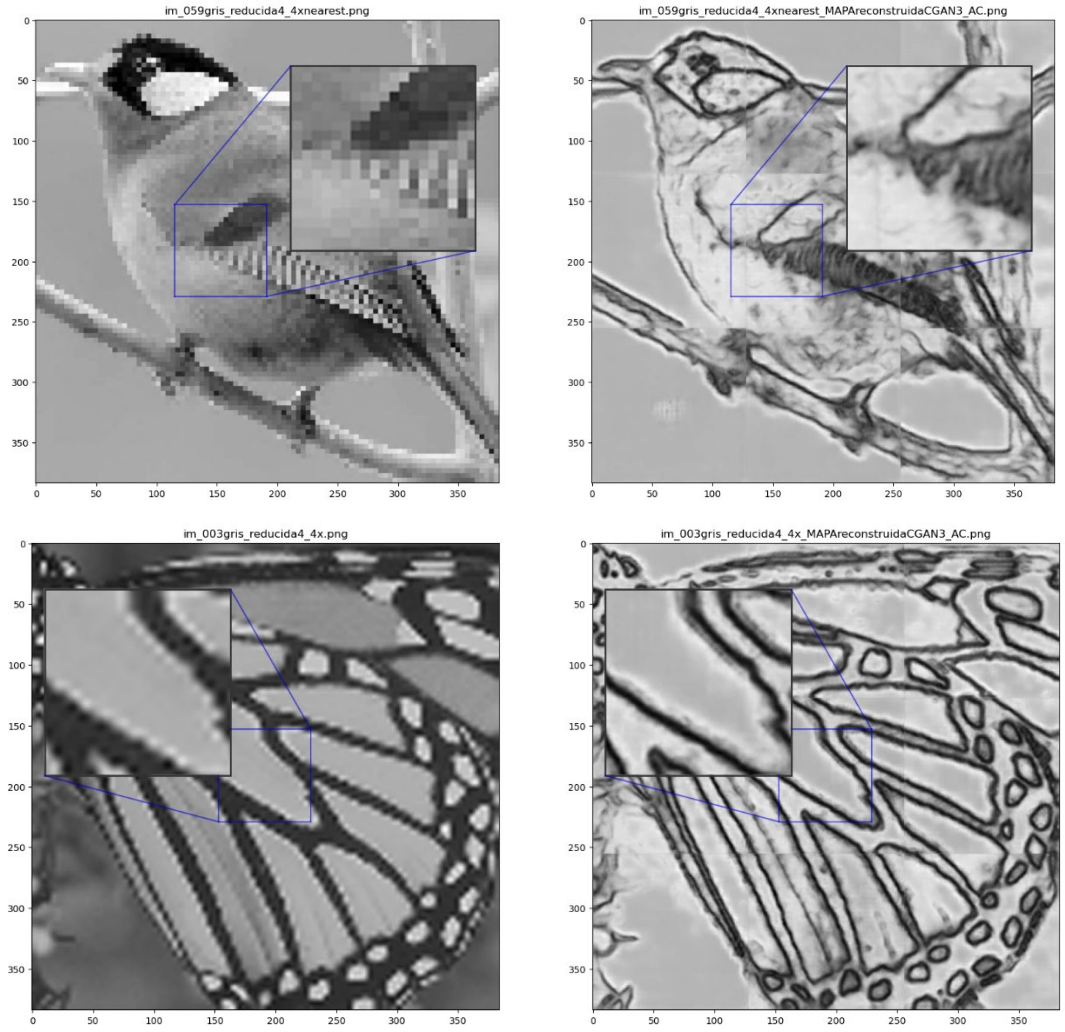


Figura 3.16: Ejemplos de imágenes reducidas por un factor 4x y la imagen resultante correspondiente de estructuras de bordes generada por la primera red neuronal.

En la Figura 3.17 se muestra el resultado de aplicar la segunda red neuronal, donde se observa que la imagen resultante recupero los bordes, detalles y tiene un mayor contraste.

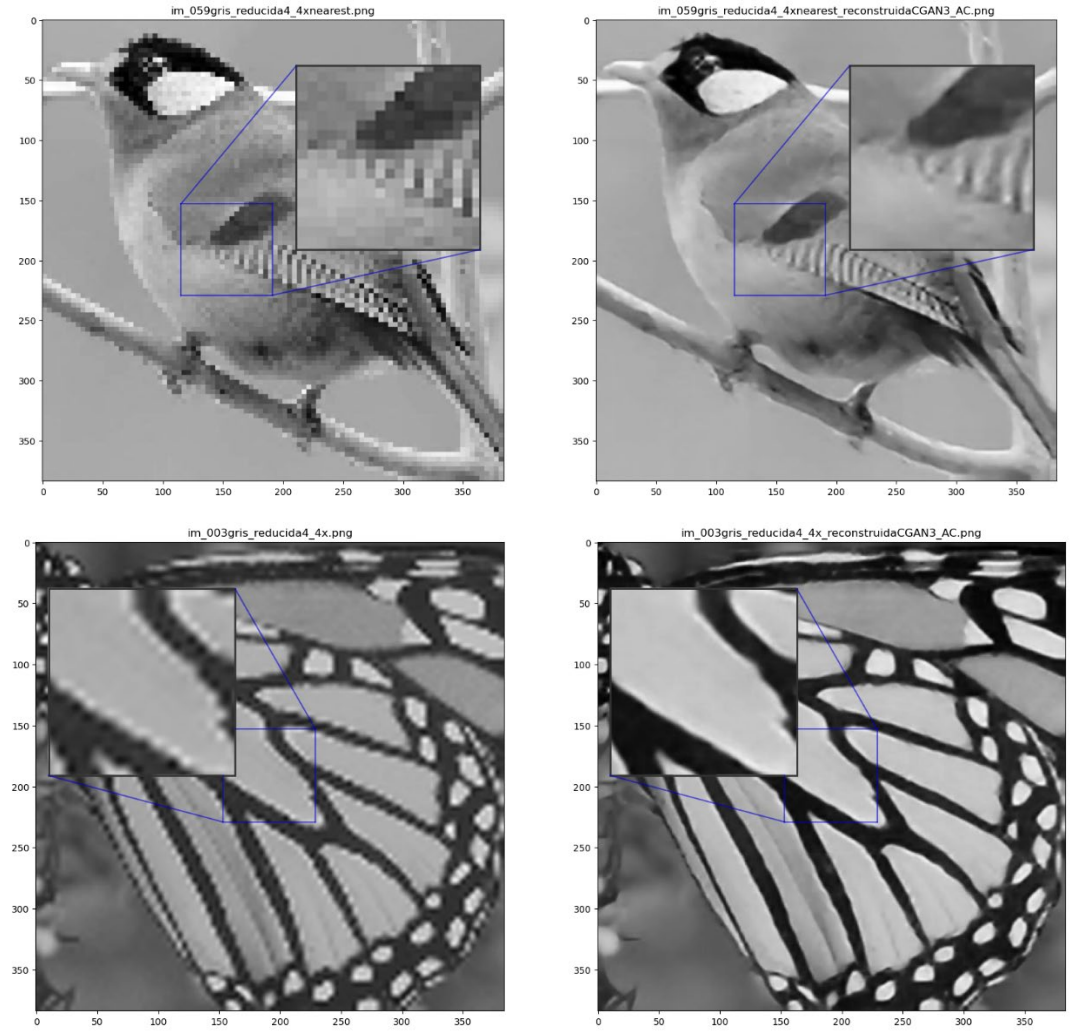


Figura 3.17: Ejemplo de imagen reducida por un factor 4x y la imagen resultante optimizada.

Segunda fase: Entrenamiento combinado de las redes neuronales

El objetivo de esta segunda fase fue el de entrenar ambas redes neuronales de forma combinada. Para este fin se conectó la salida de la primera red neuronal con la entrada de la segunda red neuronal, formando un modelo combinado más grande con una entrada y dos salidas como se muestra en la Figura 3.18.

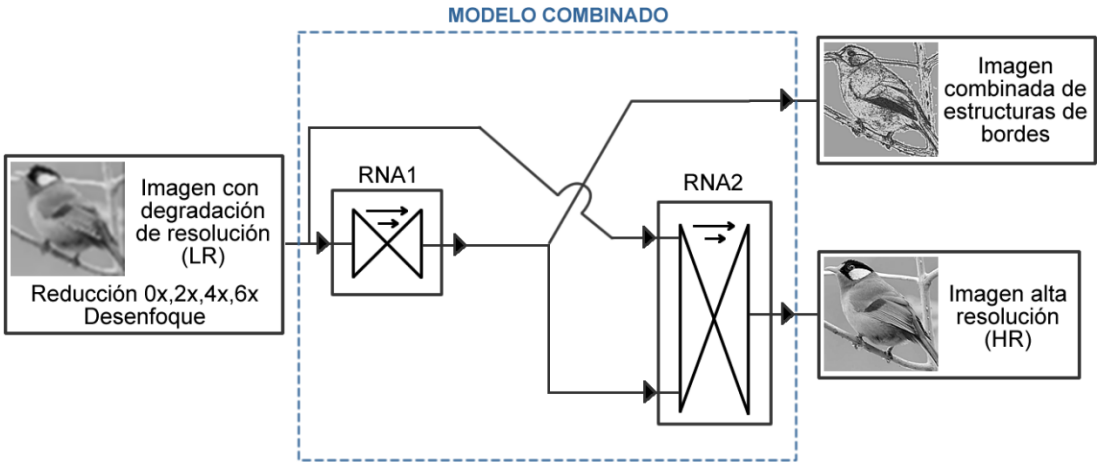


Figura 3.18: Esquema del modelo combinado de ambas redes neuronales con una entrada y dos salidas.

Para entrenar el modelo combinado se optó por una estrategia similar al entrenamiento de una red neuronal generativa condicional, en donde el entrenamiento se va intercalando entre ambas redes usando un pequeño lote aleatorio de datos y propagando el error desde la segunda red neuronal hacia la primera. Pero en esta implementación ambas redes fueron de tipo generativa y no se tiene ningún discriminador. Dicho esquema de entrenamiento se muestra en la Figura 3.19.

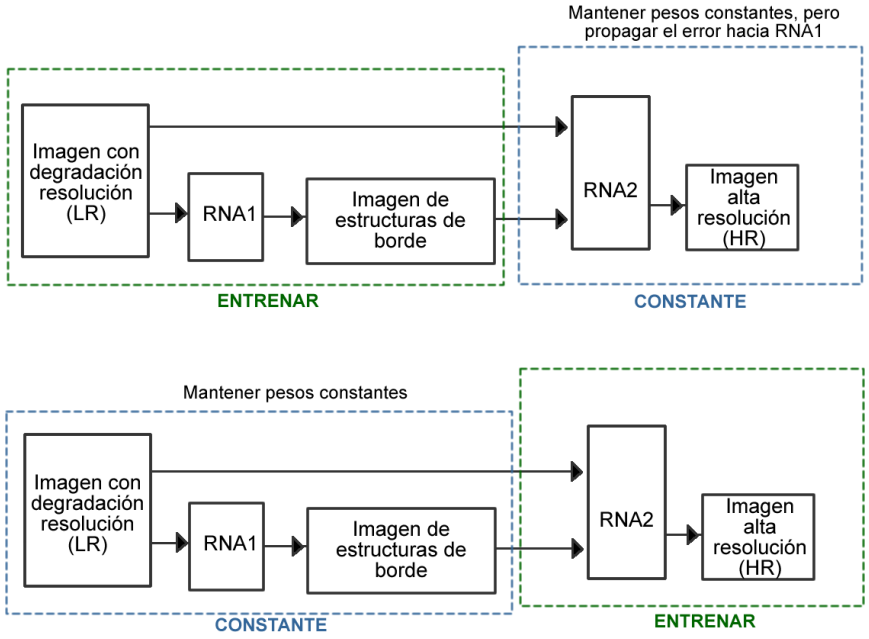


Figura 3.19: Esquema de entrenamiento combinado de ambas redes neuronales.

El entrenamiento combinado fue de un total de 6000 iteraciones, equivalentes a unas ocho épocas de entrenamiento. Se utilizaron 21192 parches de 128x128 píxeles obtenidos de las imágenes reducidas un factor de 0x, 2x, 4x, 6x con interpolación bicúbica, con interpolación de vecinos más cercanos y con desenfoque. También se utilizó el reflejo horizontal y vertical de las imágenes. En esta fase se agregaron más imperfecciones que en la primera fase porque el objetivo fue el de generalizar el modelo.

La ejecución se realizó en 15 bloques de 400 iteraciones cada una con un lote aleatorio de 20 parches de imagen. La evaluación en los datos de entrenamiento y validación se realizó al final de cada respectivo bloque. Los resultados obtenidos se muestran en la Figura 3.20, donde la primera red neuronal obtuvo un valor mínimo de la función de pérdida de 0.0445 para los datos entrenamiento y 0.0457 para los datos de validación. Y la segunda red neuronal obtuvo un valor mínimo de la función de pérdida de 0.0116 para los datos entrenamiento y 0.0119 para los datos de validación.

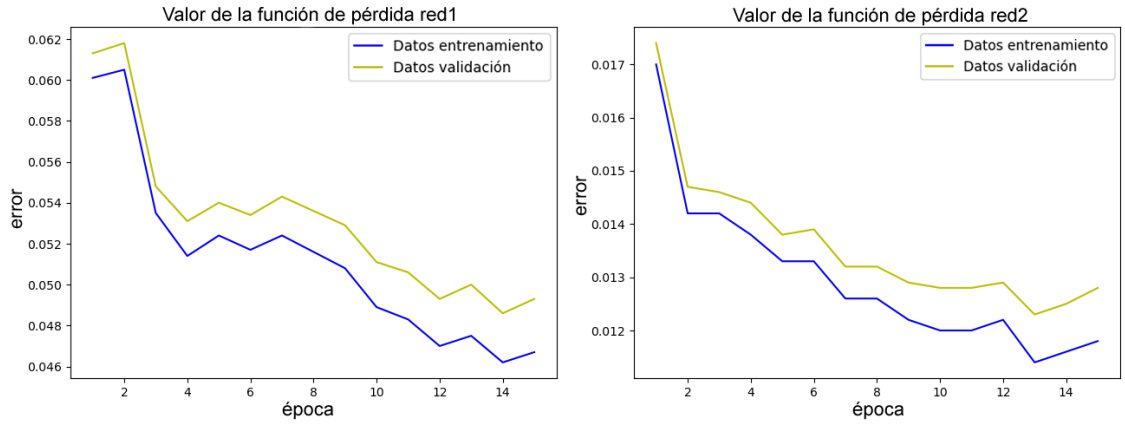


Figura 3.20: Función de pérdida de la primera y segunda red neuronal durante el entrenamiento combinado.

En la Tabla 3.4 se muestra el resultado obtenido de la ejecución de todas las épocas, con su respectivo error en los datos de entrenamiento y validación.

	Red neuronal 1		Red neuronal 2	
Bloque	**Error en datos de entrenamiento**	**Error en datos de validación**	**Error en datos de entrenamiento**	**Error en datos de validación**
1	0.0647	0.0604	0.0214	0.0187
2	0.0592	0.0588	0.0176	0.0164
3	0.0563	0.0563	0.0160	0.0151
4	0.0540	0.0543	0.0152	0.0144
5	0.0525	0.0522	0.0146	0.0139
6	0.0511	0.0505	0.0141	0.0135
7	0.0490	0.0497	0.0135	0.0141
8	0.0481	0.0490	0.0132	0.0130
9	0.0473	0.0476	0.0130	0.0126
10	0.0465	0.0472	0.0127	0.0127
11	0.0463	0.0466	0.0125	0.0126
12	0.0457	0.0467	0.0122	0.0123
13	0.0453	0.0467	0.0121	0.0125
14	0.0448	0.0459	0.0118	0.0124
15	0.0445	0.0457	0.0116	0.0119

Tabla 3.4 Error durante el entrenamiento combinado en la segunda fase.

Ajuste de brillo y corrección de bordes entre los parches de la imagen reconstruida

Las dos redes neuronales en conjunto reconstruyeron el parche de baja resolución y así se obtuvo un parche de alta resolución. Sin embargo, en algunos casos se observó que los parches de imágenes reconstruidos pueden presentan variaciones de brillo entre sí, como se muestra en la Figura 3.21. Esto se debe a que la red neuronal predice parche por parche, y no tiene un conocimiento global de la imagen.

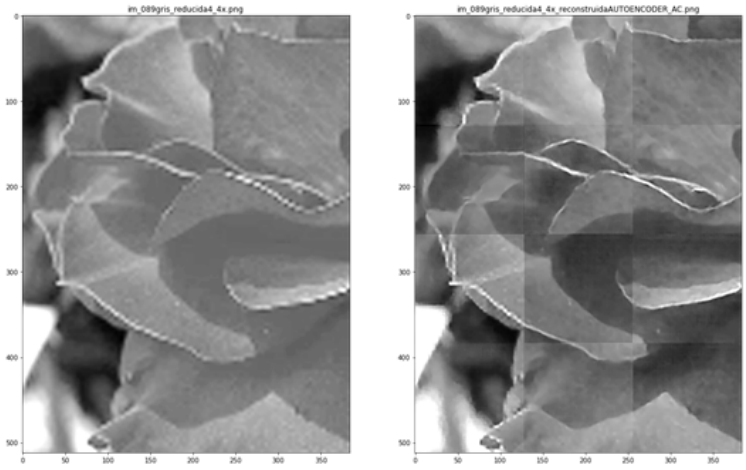


Figura 3.21: A la derecha la imagen reconstruida muestra variaciones de brillo entre parches.

Para reducir estos defectos después de unir los parches se implementaron los siguientes tres métodos:

Método de ajuste de brillo entre parches

Se calcularon las diferencias de intensidad entre los pixeles de los bordes de cada parche, considerando solo los parches vecinos izquierdo y superior. Por ejemplo, considerando el parche de la fila tres y columna dos en la Figura 3.22 ubicamos los pixeles del margen izquierdo y superior.

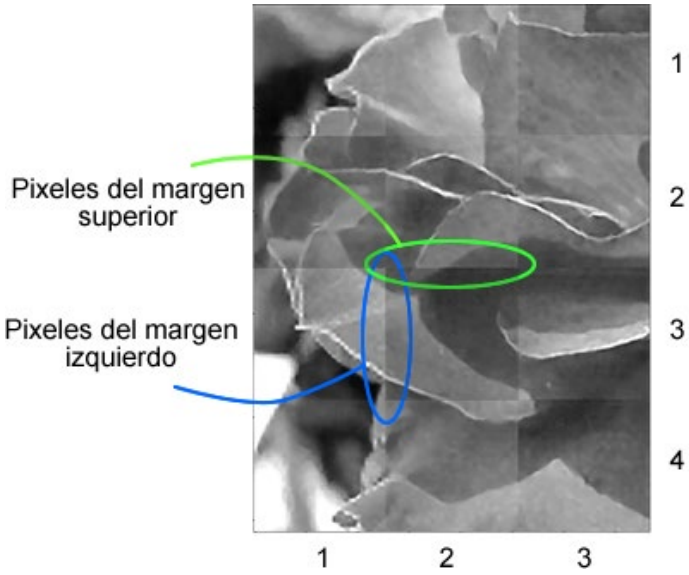


Figura 3.22: Ejemplo de los pixeles del margen izquierdo y superior de un parche.

Para los pixeles del lado izquierdo se calculan las diferencias entre los 128 pixeles que están fuera y dentro del borde del parche, estas diferencias se suman y se promedian. Estos cálculos se muestran en la ecuación 8.

$$ajusteIzquierdo = \frac{\sum_{i=0}^{127}(PMI[i]_{fuera} - PMI[i]_{dentro})}{128} \qquad (8)$$

donde:

PMI[]$_{dentro}$: Son los pixeles del margen izquierdo dentro del parche.

PMI[]$_{fuera}$: Son los pixeles del margen izquierdo fuera del parche.

Con los pixeles del lado superior se sigue un procedimiento similar, el cual se muestra en la ecuación 9.

$$ajusteSuperior = \frac{\sum_{i=0}^{127}(PMS[i]_{fuera} - PMS[i]_{dentro})}{128} \qquad (9)$$

donde:

PMS[]$_{dentro}$: Son los pixeles del margen superior dentro del parche.

PMS[]$_{fuera}$: Son los pixeles del margen superior fuera del parche.

Luego el ajuste total para el parche es el promedio de ambos ajustes, el cual se muestra en la ecuación 10.

$$ajusteTotal = \frac{ajusteIzquierdo + ajusteSuperior}{2} \qquad (10)$$

Finalmente, este ajuste total se debe sumar a todos los pixeles del parche, mediante la ecuación 11.

$$parche[x, y] = parche[x, y] + ajusteTotal \qquad (11)$$

El resultado de haber aplicado este procedimiento a cada parche es que todos adquieren un brillo similar como se muestra en la Figura 3.23.

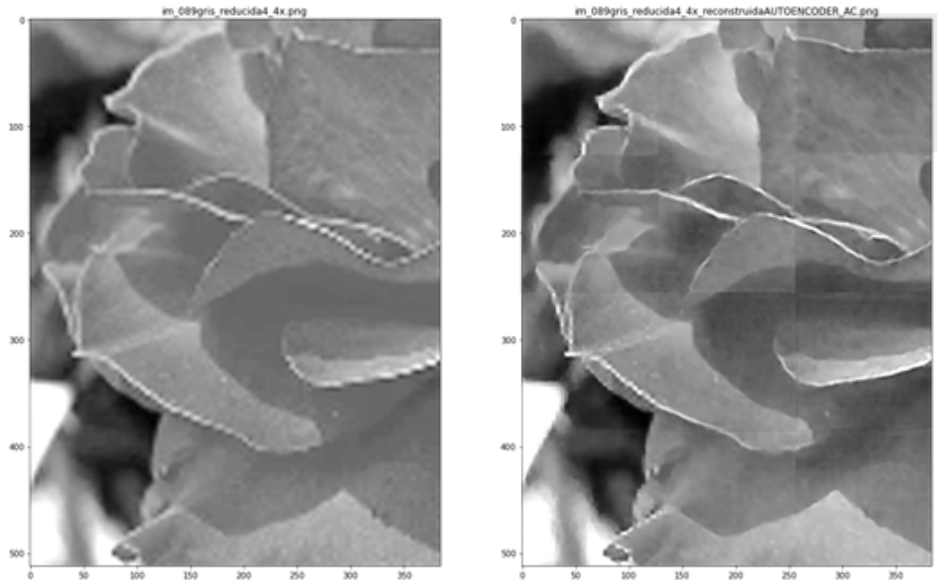


Figura 3.23: A la derecha la imagen reconstruida aplicando correcciones de brillo entre parches.

Método de difuminado de bordes de parches

El método de ajuste de brillo logro corregir la luminosidad en general entre parches, pero aún siguen visibles líneas de división entre parches. Para corregirlo el método de difuminado de bordes utiliza una máscara de difuminado de 4 pixeles mostrada en la Figura 3.24, la cual se aplica a los bordes de cada parche. Este método debe aplicarse después de haber aplicado el método de ajuste de brillo entre parches. Es posible aplicar reiterativamente el método de ajuste de brillo y difuminado un máximo de 2 o 3 veces antes de que se generen defectos visuales. En la Figura 3.25 se muestra el resultado de aplicar el método de ajuste de brillo y el difuminado de bordes aplicado dos veces.

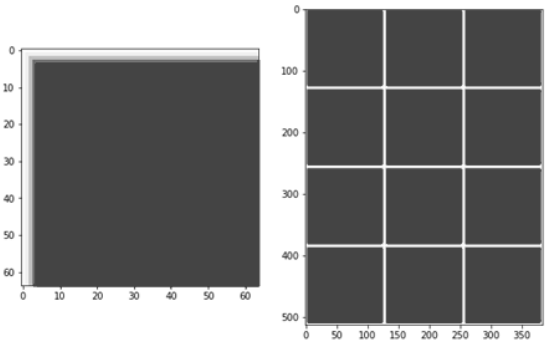


Figura 3.24: Máscara de difuminado de 4 pixeles en los bordes del parche.

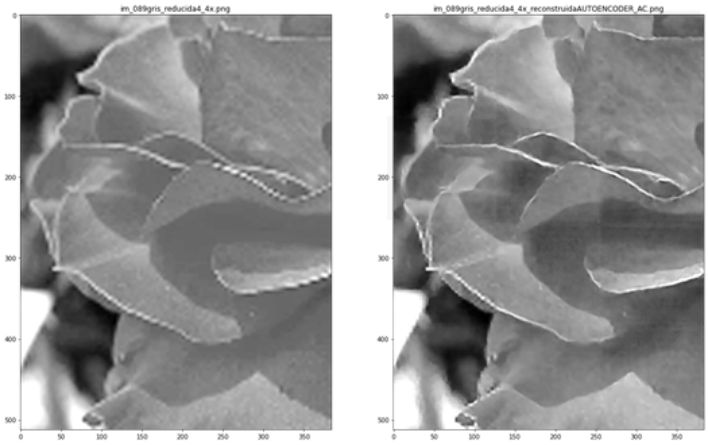


Figura 3.25: Resultado de aplicar los métodos de ajuste de brillo y difuminado de bordes aplicado dos veces.

Para aplicar la máscara de difuminado se utilizan las ecuaciones 12, 13, 14 y 15 aplicadas a cada parche y sus cuatro correspondientes lados.

$$parche[i,j] = parche[i,j] + (pixelesLadoIzquierdo[] * difuminado[i,j])) \quad (12)$$

$$parche[i,j] = parche[i,j] + (pixelesLadoSuperior[] * difuminado[i,j])) \quad (13)$$

$$parche[i,j] = parche[i,j] + (pixelesLadoDerecho[] * difuminado[i,j])) \quad (14)$$

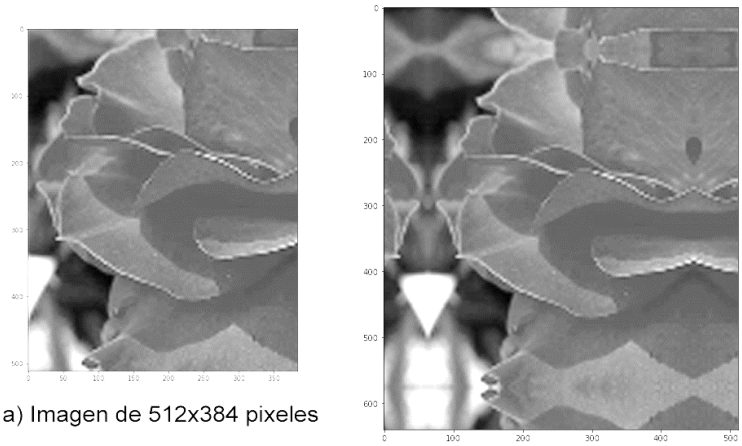
$$parche[i,j] = parche[i,j] + (pixelesLadoInferior[] * difuminado[i,j])) \quad (15)$$

Donde solo se modifican los pixeles [i,j] correspondientes a cada lado del parche correspondiente con un grosor de 4 pixeles.

Método de interpolación de parches

Los métodos de ajuste de brillo y difuminado de bordes lograron corregir el brillo y reducir las líneas de división entre parches, pero aún siguen visibles tenues líneas de división entre parches. Para solucionar esto fue necesario implementar un método de interpolación de parches, el cual consiste en aplicar la red neuronal a dos imágenes que luego serán combinadas para eliminar las líneas de división. Este método consiste en:

- Aumentar el tamaño de los bordes de la imagen de baja calidad en 64 pixeles a cada lado. Manteniendo la resolución original en el centro y reflejando el contenido de los bordes para completar el total de la imagen como se muestra en la Figura 3.26.



a) Imagen de 512x384 pixeles

b) Imagen con bordes ampliados en 64 pixeles a cada lado. Obteniendo una imagen de 640x512 pixeles

Figura 3.26. Ejemplo de aumento de bordes reflejando el contenido.

- Luego se aplicó el modelo de red neuronal a la imagen con aumento de bordes. Como la imagen es más grande en 64 pixeles en cada lado, al momento de segmentarla en parches de 128x128 obtendremos una predicción clara que ya no incluye los bordes.
- Luego se recorta la imagen resultante para que las dimensiones sean las mismas que la original.
- Finalmente se combinaron ambas imágenes usando una máscara de intensidad de bordes que se muestra en la Figura 3.27. La máscara de intensidad se calcula según las dimensiones de la imagen y permite interpolar los pixeles de los parches de las dos imágenes.

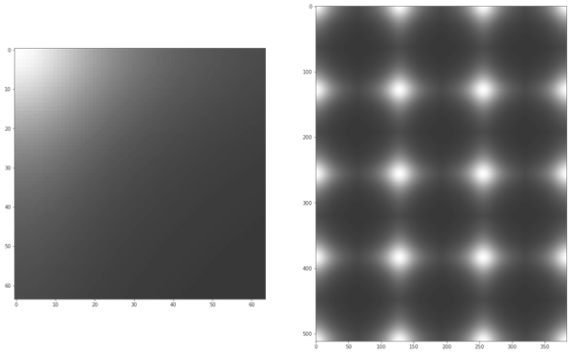


Figura 3.27: Mascara de intensidad usada para combinar las dos imágenes.

La máscara de intensidad de la Figura 3.27 fue construida por medio de una función cuadrática inversa aplicada a cada región de parche de 128x128. La función para cada eje se define en las ecuaciones 16 y 17.

$$intensidadY = \frac{1}{0.002*(y)^2+0.002*(y)+1} \quad , \text{con } y \in [0, 64) \qquad (16)$$

$$intensidadX = \frac{1}{0.002*(x)^2+0.002*(x)+1} \quad , \text{con } x \in [0, 64) \qquad (17)$$

En la Figura 3.28 se muestra la gráfica de la función de intensidad solo para el eje x.

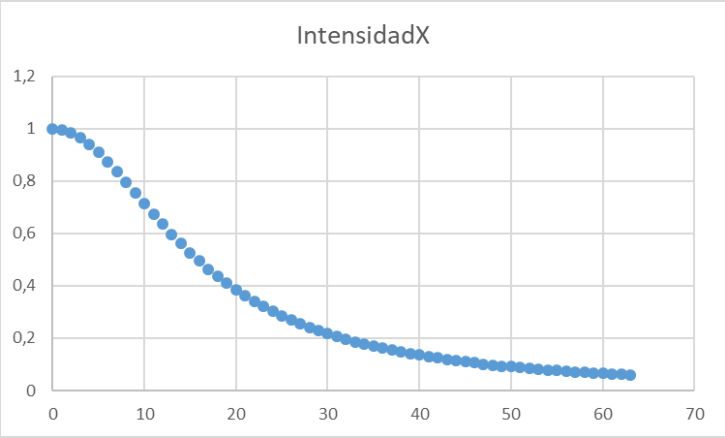


Figura 3.28: Gráfica de la función de intensidad para la construcción de la máscara.

Luego para cada pixel la intensidad es el producto de intensidades como se muestra en la ecuación 18, con lo cual se obtiene la matriz de intensidad mostrada en la Figura 3.29.

$$intensidad[x, y] = intensidadY * intensidadX \qquad (18)$$

Finalmente, para combinar ambas imágenes con la máscara se usa la ecuación 19 de interpolación para cada pixel de la imagen.

$$imagen[x, y] = \big(imagen[x, y] * (1 - intensidad[x, y])\big) + (imagenAR[x, y] * intensidad[x, y]) \qquad (19)$$

Donde "imagenAR" es la imagen con bordes ampliados y reducidos. El método de interpolación de parches puede aplicarse hasta un máximo de dos veces antes de que se

generen defectos visuales. En la Figura 3.29 se muestra el resultado de aplicar dos veces la interpolación de parches entre imágenes y los anteriores métodos.

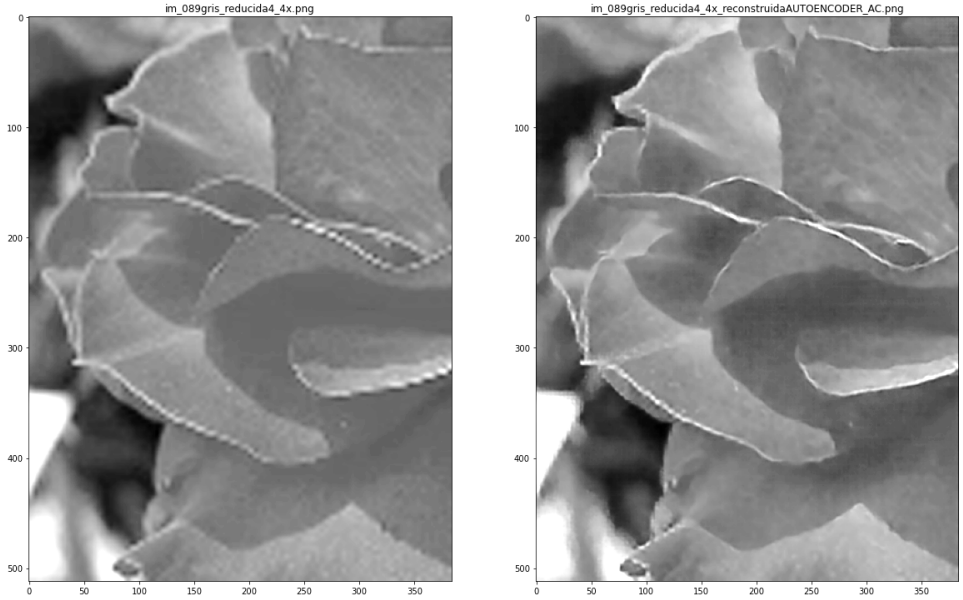


Figura 3.29: Resultado de aplicar los métodos de ajuste de brillo, difuminado de bordes e interpolación entre parches

Finalmente, el modelo completo para la optimización de imágenes en escala de grises con el uso de las dos redes neuronales, y los métodos de ajustes entre parches se muestra en la Figura 3.30.

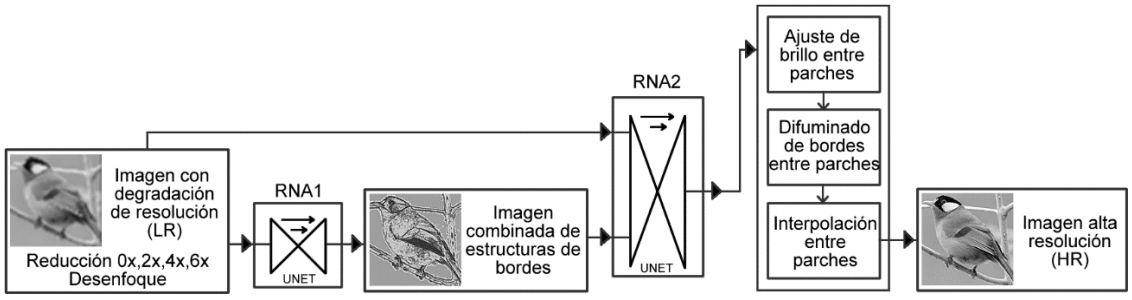


Figura 3.30: Diagrama del modelo de optimización de resolución de imágenes en escala de grises.

3.4 AJUSTES EN LA RECONSTRUCCION DE BORDES.

Debido a que las imágenes de baja resolución pasan por una transformación intermedia antes de ser optimizada por la segunda red neuronal. Entonces usando el modelo ya entrenado es posible aplicar reiteradas veces la segunda red neuronal con el objetivo de intensificar la reconstrucción de bordes. Esto se muestra en la Figura 3.31.

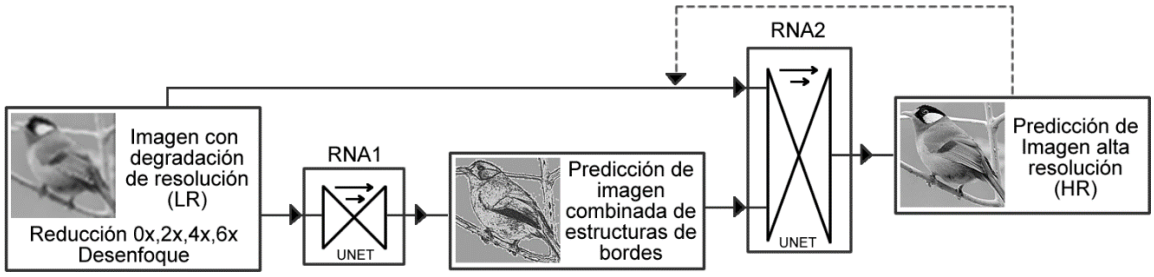


Figura 3.31: Aplicar reiteradamente la segunda red neuronal del modelo ya entrenado.

En la Figura 3.32 se muestra un ejemplo de la aplicación reiterada de la segunda red neuronal.

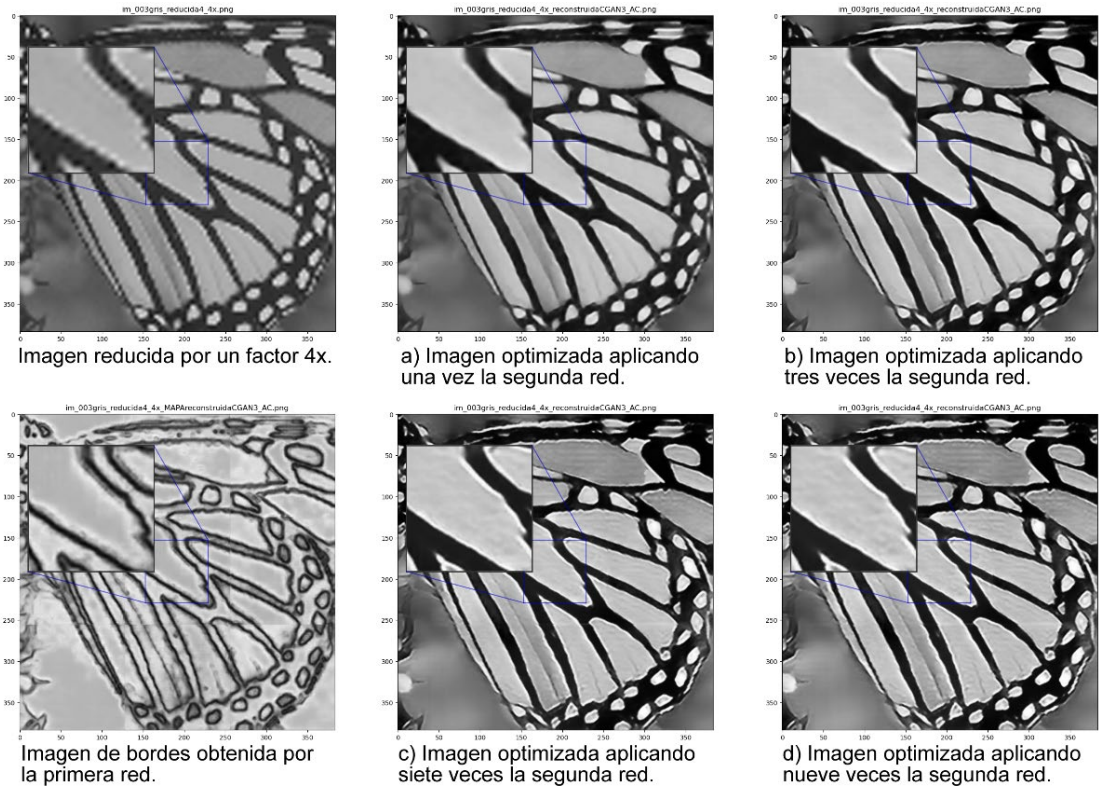


Figura 3.32: Aplicando la segunda red neuronal reiteradamente.

Por otro lado, también se comprobó el impacto que tiene la imagen intermedia de estructuras de borde en la reconstrucción de la imagen. Para ello se modificó la imagen de estructuras de borde agregando dos cuadrados, uno totalmente blanco y otro negro como se muestra en la Figura 3.33.

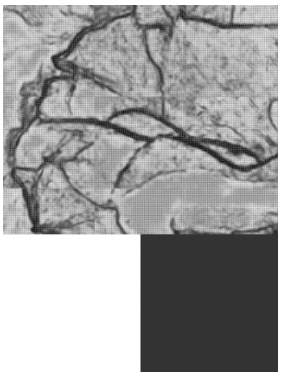


Figura 3.33: Modificando la imagen de estructuras de borde, colocando un cuadrado blanco y negro.

Luego aplicando la segunda red neuronal obtenemos la imagen de la Figura 3.34. Donde se pudo observar que la red neuronal hace una distinción evidente entre el color blanco y negro de la imagen de estructuras de borde. Lo cual tiene sentido porque el color negro está asociado con los bordes y el blanco con aquello que no es borde. Entonces la segunda red neuronal optimiza la imagen combinando la imagen de estructuras de borde para realizar la reconstrucción de bordes de la imagen.

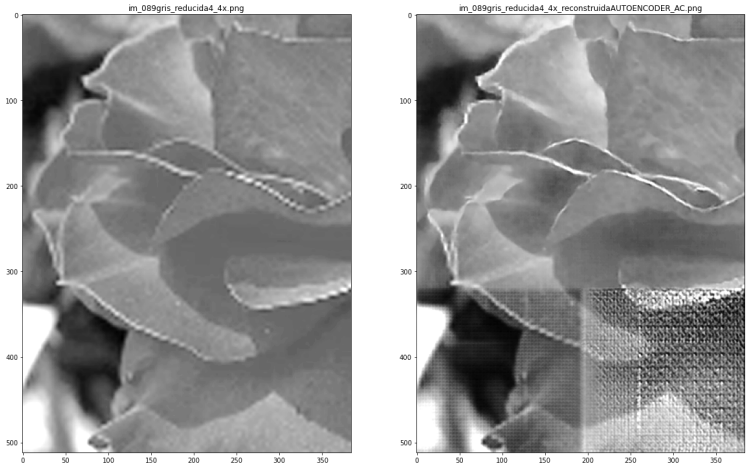


Figura 3.34: Resultado de aplicar la imagen de estructuras modificada y la segunda red neuronal.

3.5 AMPLIANDO EL MODELO PARA IMÁGENES A COLOR

El modelo fue ampliado para usarse en imágenes a color incorporando los tres canales RGB como entradas y salidas de las dos redes neuronales, y usando la imagen de estructuras de borde en RGB como se muestra en la Figura 3.35. Respecto al ajuste de brillo y unión entre parches estos se aplicaron a cada canal y posteriormente fueron unidos para obtener la imagen final de alta resolución.

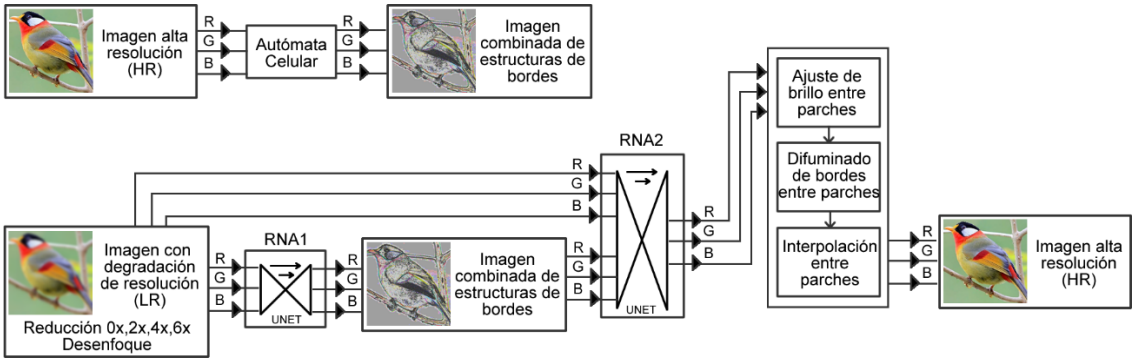


Figura 3.35: Diagrama final del modelo de optimización de resolución para imágenes a color.

En las Figuras 3.36 y 3.37 se muestra el uso del modelo para la optimización de imágenes a color.

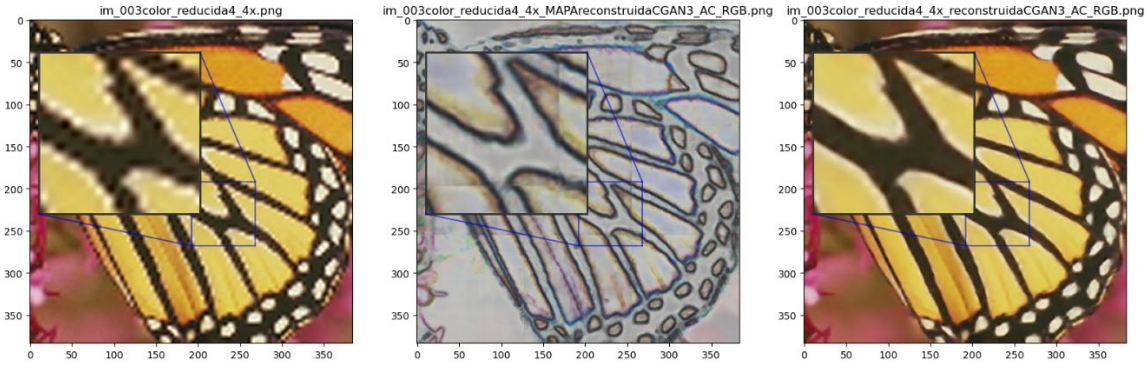


Figura 3.36: Ejemplo de imagen del conjunto de datos Set5 reducida por un factor 4x, la imagen de bordes y la imagen resultante optimizada.

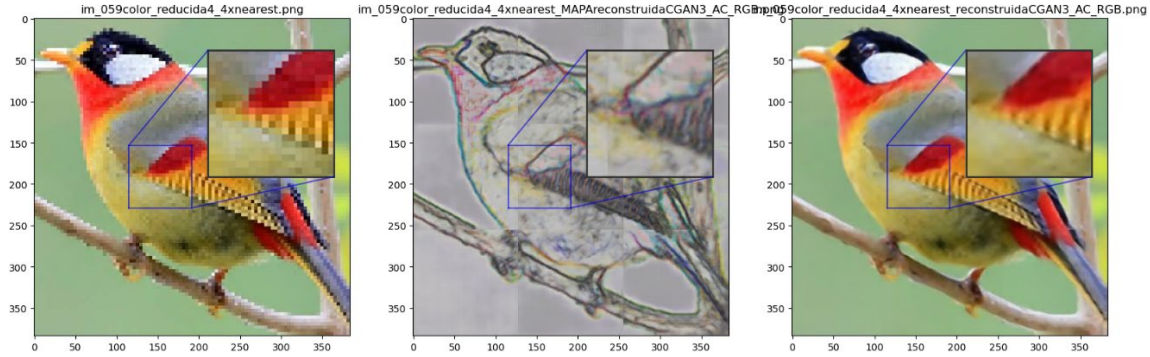


Figura 3.37: Ejemplo de imagen del conjunto de datos General100 reducida por un factor 4x y la imagen resultante optimizada.

3.6 EVALUACIÓN PRELIMINAR DE LA CAPACIDAD PREDICTIVA DEL MODELO

Para medir la capacidad predictiva del modelo se realizó una evaluación preliminar donde se compararon las imágenes originales en escala de grises con las imágenes resultantes de aplicar el modelo. Esta evaluación preliminar se usó para realizar ajustes al modelo propuesto hasta lograr resultados óptimos.

Para ello se consideraron las 50 imágenes que la red neuronal nunca observo. Se consideran las métricas MSE, PSNR, SSIM, y DISTS. En la Figura 3.38 se muestra un ejemplo de cálculo de las métricas para ejemplificar cuáles son los valores óptimos.

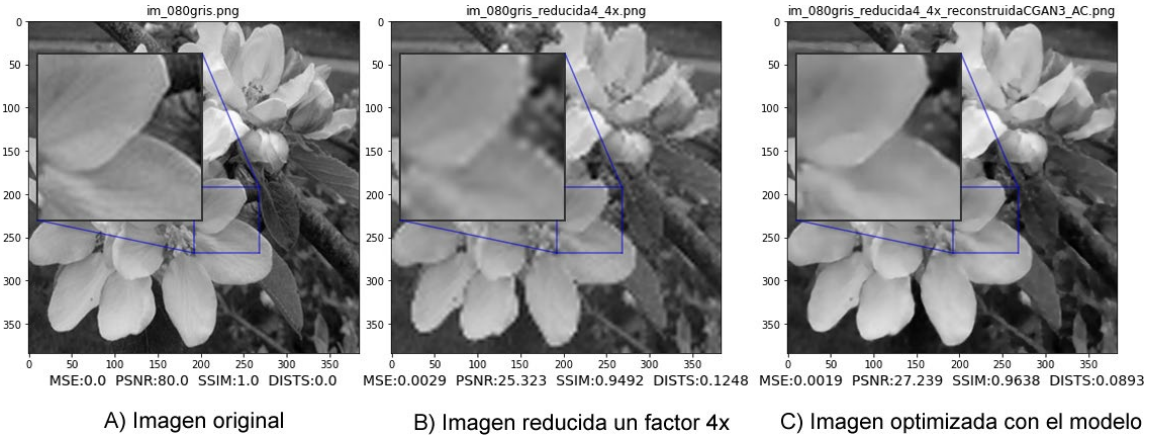


Figura 3.38: Se observan las cuatro métricas calculadas comparando: la imagen A con ella misma, la imagen A con la B, y la imagen A con la C.

Cada métrica permite comparar dos imágenes y determinar qué tan similares son entre sí, y cada una tiene un valor óptimo ideal como se observa en la Figura 3.38 en la imagen original. La descripción detallada de los valores óptimos se muestra en la Tabla 3.5.

Métrica	Valor óptimo ideal	Descripción de la comparación
MSE	0	Un valor de 0 indica que no existe error, y la dos imágenes comparadas son exactamente iguales.
PSNR	depende de la imagen y mientras más alto se considera mejor.	
SSIM	1	Un valor de 1 indica que las dos imágenes son exactamente iguales estructuralmente.
DISTS	0	Un valor de 0 indica que las dos imágenes comparadas son exactamente iguales respecto a las texturas y estructuras.

Tabla 3.5: Valores óptimos por cada métrica utilizada para comparar imágenes.

Los resultados obtenidos aplicando las métricas a las 50 imágenes se muestran en la Tabla 3.6. Donde se observa que se lograron mejoras respecto a las cuatro métricas.

Conjunto de datos	**MSE (óptimo ideal 0)**		**PSNR (óptimo ideal, más alto mejor)**		**SSIM (óptimo ideal 1)**		**DISTS (óptimo ideal 0)**	
	Reducida 4x	Optimizada RODIUCAN	Reducida 4x	Optimizada RODIUCAN	Reducida 4x	Optimizada RODIUCAN	Reducida 4x	Optimizada RODIUCAN
General100	0.0057	0.0044	23.686	24.678	0.9025	0.9189	0.1263	0.0866

Tabla 3.6: Métricas promedio obtenidas para el modelo propuesto RODIUCAN para un subconjunto de validación del conjunto de datos General100.

3.7 TIEMPO Y USO DE RECURSOS COMPUTACIONALES DEL MODELO

El tiempo y los recursos computacionales fueron analizados para la construcción del modelo propuesto. Considerando para las pruebas sin GPU una computadora Pentium I7 con 16 GB de RAM, y para las pruebas con GPU se usó GoogleColab con 12 GB de RAM. En el caso de GoogleColab las imágenes preprocesadas para el entrenamiento fueron alojadas en GoogleDrive.

Primera fase

El entrenamiento individual de ambas redes con aceleración GPU duro en total una hora. Se probó el mismo entrenamiento, pero sin el uso de GPU y tuvo un tiempo aproximado de 30 horas. La descripción detallada se muestra en las Tablas 3.7 y 3.8.

Red neuronal	**Cantidad de épocas**	**Cantidad de parámetros del modelo (en millones)**	**Tiempo de entrenamiento**
Red 1	30	4.6 M	10 minutos
Red 2	30	4.6 M	10 minutos
Total	60	9.2 M	**20 minutos**

Tabla 3.7: Tiempo de entrenamiento individual de ambas redes neuronales con GPU.

Red neuronal	**Cantidad de épocas**	**Cantidad de parámetros del modelo (en millones)**	**Tiempo de entrenamiento**
Red 1	30	4.6 M	4 horas
Red 2	30	4.6 M	4 horas
Total	60	9.2 M	**8 horas**

Tabla 3.8: Tiempo de entrenamiento individual de ambas redes neuronales sin GPU.

Segunda fase

En la segunda fase debido al aumento de datos no se pudo entrenar en GoogleColab debido a que los 12GB de RAM fueron insuficientes para cargar los datos. Entonces el entrenamiento en esta fase solo fue realizado sin GPU. El entrenamiento duro un total de 30 horas. En la Tabla 3.9 se muestra el detalle de las iteraciones de cada bloque de entrenamiento y su respectivo tiempo sin aceleración GPU.

Bloque	Tiempo de entrenamiento sin GPU
1	1 hora 18 minutos
2	1 hora 19 minutos
3	1 hora 18 minutos
4	1 hora 18 minutos
5	1 hora 18 minutos
6	1 hora 19 minutos
7	1 hora 22 minutos
8	1 hora 14 minutos

Bloque	Tiempo de entrenamiento sin GPU
9	1 hora 16 minutos
10	1 hora 17 minutos
11	1 hora 19 minutos
12	1 hora 18 minutos
13	1 hora 18 minutos
14	1 hora 21 minutos
15	1 hora 19 minutos
Total	**20 horas**

Tabla 3.9: Tiempo de entrenamiento combinado de las redes neuronales sin GPU.

Por lo tanto, en el caso de un entrenamiento sin GPU el tiempo de entrenamiento de la primera fase fue de 8 horas y de la segunda fase fue de 20 horas, haciendo un total de 28 horas.

3.8 CONSUMO ELÉCTRICO Y EQUIVALENTE EN EMISIONES DE CO2

Para las pruebas y entrenamiento sin GPU se compró una computadora de escritorio Pentium I7 con 16GB de RAM. Todas las pruebas y ejecuciones se realizaron en dicho computador durante aproximadamente cuatro meses. Lo cual se hizo evidente debido a un aumento en el consumo eléctrico regular. Por tanto, para simplificar el cálculo del consumo eléctrico requerido en el entrenamiento del modelo propuesto, se consideró el gasto de energía eléctrica promedio antes y después de la compra del computador.

El consumo eléctrico promedio por mes antes de la compra del equipo fue de 150 kilovatios y después se incrementó a 220 kilovatios por mes. Por tanto, el consumo aproximado para el funcionamiento del computador fue de unos 70 kilovatios por mes.

Luego los 70 kilovatios (kW) de consumo eléctrico promedio por mes se divide entre 31 días y entre 12 horas (el computador fue usado en promedio 12 horas por día), obteniéndose de este modo el consumo aproximado por hora mostrado en la ecuación 20.

$$p_a = \frac{70}{31 * 12} = 0.188\ kW \qquad (20)$$

Luego la ecuación del gasto de energía puede expresarse como en la ecuación 21.

$$p_t = \frac{1.58t(p_c + p_r + gp_g)}{1000} = 1.58t(p_a) \qquad (21)$$

Donde p_a es una aproximación a la expresión: $((p_c+p_r+gp_g)/1000)$, que representa el consumo eléctrico por hora en kilovatios. Esto tiene sentido porque p_a ya está expresado en kilovatios entonces ya no es necesario dividirlo entre 1000.

Por otro lado, el valor p_c aproximado para un procesador I7 es de 91W y el valor de p_r para cuatro módulos 4GB de RAM es de 40W. Por lo tanto, en el computador sin GPU el valor teórico de $p_{a_teorico}$ es 0.131 kW, como se muestra en la ecuación 22.

$$p_{a_teorico} = \frac{(p_c + p_r + gp_g)}{1000} = \frac{(91 + 40 + 0)}{1000} = 0.131\ kW \qquad (22)$$

El valor teórico $p_{a_teorico}$ es menor al valor calculado para p_a, esto tiene sentido porque existen consumos adicionales por uso de monitor, parlante, teclado y ratón. Es importante recordar que la ecuación de gasto de energía fue calculada para equipos de centros de datos que normalmente no tienen periféricos adicionales. Por tanto, es correcto usar p_a en la ecuación 21 del gasto energía para un computador personal.

Luego se reemplaza el valor de p_a y el tiempo total de entrenamiento de 28 horas en la ecuación 21. Obteniéndose un consumo eléctrico total de 8.32 kW como se muestra en la ecuación 23.

$$p_t = \frac{1.58t(p_c + p_r + gp_g)}{1000} = 1.58(t)(p_a) = 1.58(28)(0{,}188) = 8.32\ kWh \quad (23)$$

Posteriormente usando la ecuación para convertir el consumo de energía en emisiones estimadas de CO_2 en libras se tiene los resultados de la ecuación 24:

$$CO_2e = 0.954(p_t) = \ 0.954(8.32) = 7.9\ libras\ CO_2 \qquad (24)$$

Luego con la ecuación 25 se realizó la conversión al sistema MKS.

$$CO_2e = 7.9\ libras\ .\frac{0.453\ kilogramos}{1\ libra} = 3.6\ kilogramos\ CO_2 \qquad (25)$$

Por lo tanto, el entrenamiento sin GPU del modelo propuesto duro 28 horas, tuvo un consumo eléctrico de 8.32 kilo-vatios y un equivalente de 3.6 kilogramos de emisiones de dióxido de carbono.

Por otro lado, es posible estimar el consumo eléctrico teórico de entrenamiento del modelo ESRGAN, considerando 4 GPUs y una tarjeta gráfica GeForce con un consumo aproximado de 150W, y con un tiempo de entrenamiento de 4 días. El consumo eléctrico y las emisiones de CO2 se calcularon con las ecuaciones 26 y 27.

$$p_{t_ESRGAN} = \frac{1.58(t)(p_c + p_r + gp_g)}{1000} = \frac{1.58(4 * 24)(91 + 40 + 4(150))}{1000}$$
$$= 111\ kW \qquad (26)$$

$$CO_2e\ ESRGAN = 0.954(p_{t_ESRGAN}) = \ 0.954(111) = 105.8\ libras\ CO_2$$
$$= 47.9\ kilogramos\ CO_2 \qquad (27)$$

3.9 CÓDIGO FUENTE DEL MODELO Y PUBLICACIÓN

El código fuente del modelo propuesto fue estructurado etapas, en las cuales se fue estructurando un conjunto de funciones de uso general, de modo que sea posible repetir las etapas con algún otro conjunto de datos. Después de haber concluido con las etapas el resultado es una librería de funciones llamada “librería.py” y una estructura de modelo en “estructura_modelo.py”.

El modelo y las pruebas fueron almacenadas en un repositorio público en GitLab en la dirección: https://gitlab.com/rolandex25/rodiucan, donde se tiene acceso libre a los cuadernos en Python, las imágenes usadas en el entrenamiento, pruebas, y los modelos ya entrenados en formato h5. También se creó otro repositorio en GitHub en la dirección: https://github.com/rolandex25g/rodiucan, con lo fundamental para ejecutar el modelo. En la Tabla 3.10 se muestra una descripción de los archivos almacenados en el repositorio.

Carpeta	Archivos o subcarpetas	Descripción
ColabNotebooks	etapa1 imagenes y automatas.ipynb	Prueba inicial con una sola imagen.
	etapa2 aumento y reduccion de imagenes.ipynb	Pre-procesamiento de todas las imágenes.
	etapa3 prepara y entrena modelo.ipynb	Entrenar el modelo para imágenes en escala de grises.
	etapa3 prepara y entrena modeloRGB.ipynb	Entrenar el modelo para imágenes a color.
	etapa4 prueba de calidad modelo.ipynb	Comparar resultados del modelo propuesto RODIUCAN con el modelo ESRGAN
	etapa5 publicacion.ipynb	Ejemplo de uso del modelo h5 en colores.
	estructura_modelo.py	Estructura de capas de los modelos.
	libreria.py	Librería de funciones auxiliares.
datosimg	general100	Conjunto original de 100 imágenes
	general100mod128	Imágenes preprocesadas para el entrenamiento.
	general100resultado128	Resultados de la prueba preliminar
salida	redautoencoderEB128.h5	Primera RNA que convierte la imagen en escala de grises a la imagen de bordes.
	redautoencoderAC2_128.h5	Segunda RNA que convierte la imagen en escala de grises y la imagen de bordes a la imagen optimizada.
	redcombinada128_epoca1.h5	Modelo que combina la primera y segunda RNA en escala de grises.
	redautoencoderEB128RGB.h5	Primera RNA que convierte la imagen en colores a la imagen de bordes.
	redautoencoderAC2_128RGB	Segunda RNA que convierte la imagen en colores y la imagen de bordes a la imagen optimizada en colores.
	redcombinada128RGB_epoca1.h5	Modelo que combina la primera y segunda RNA en colores.

Tabla 3.10: Descripción de los archivos publicados en el repositorio GitLab.

Los cuadernos de todas las etapas fueron publicados en línea por medio de GoogleColab.

Finalmente se publicó un ejemplo demostrativo en línea mediante Hugging Face, en el cual se pudo ejecutar directamente en el navegador en la dirección: https://huggingface.co/spaces/rolandex25/RODIUCAN-Demo

En la Figura 3.39 se muestra con una imagen de ejemplo de baja resolución, la imagen de bordes generada y el resultado de la optimización. La segunda red fue repetida adicionalmente una vez más con el fin de intensificar la reconstrucción.

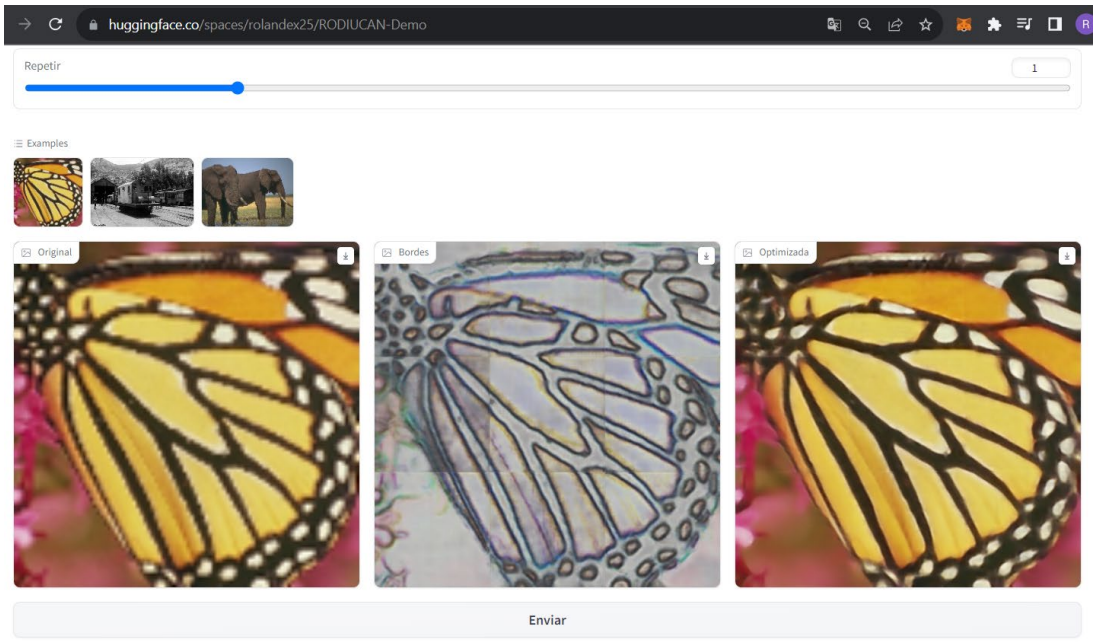


Figura 3.39: Ejemplo demostrativo del modelo RODIUCAN publicado en Hugging Face.

CAPÍTULO IV

EVALUACIÓN DEL MODELO

Para evaluar las capacidades predictivas del modelo propuesto este fue evaluado en seis diferentes conjuntos de datos. Para ello se consideraron las mismas métricas utilizadas en la evaluación preliminar: MSE, PSNR, SSIM y DISTS. Y se realizó la misma evaluación con el modelo ESRGAN para tener un punto de comparación entre ambos modelos.

Se consideraron seis diferentes conjuntos de datos típicamente usados en pruebas de modelos de aumento de resolución, como ser: Set5, Set14, historical, BSD100, Urban100, Manga109. El conjunto de datos "historical" de solo diez imágenes fue reemplazado por un conjunto de datos más grande de fotos antiguas históricas de la ciudad de La Paz que se denominó Historicas40.

En la Tabla 4.1 se muestra una descripción detallada de los conjuntos de datos usados para la evaluación.

Conjunto de datos	**Cantidad de imágenes**	**Descripción**
BSDS100	100	Fotos variadas de objetos, personas, objetos y paisajes en primer plano.
Manga109	109	Imágenes de portadas de revistas de manga.
Set5	5	Imágenes típicas para pruebas de modelos procesamiento de imágenes: butterfly.png, bird.png, etc.
Set14	14	Imágenes típicas para pruebas de modelos procesamiento de imágenes: baboon.png, lenna.png, monarch.png, etc.
Urban100	100	Fotos de sitios urbanos modernos, edificios, rascacielos, etc.
Historicas40	40	Fotos antiguas de la ciudad de La Paz

Tabla 4.1: Descripción de los conjuntos de datos usados en la evaluación de los modelos.

En la Tabla 4.2 se muestran los resultados de comparar las imágenes originales con sus versiones reducidas en un factor de 4x y optimizadas. Por ejemplo, para el conjunto de datos BSDS100 las 100 imágenes originales comparadas con sus respectivas versiones reducidas un factor de 4x tienen un MSE promedio de 0,0077. Y las imágenes originales comparadas con la versión optimizada por el modelo propuesto tienen un promedio MSE de 0.0066. Como el valor óptimo ideal de MSE es cero, entonces se evidencia que el modelo propuesto RODIUCAN tiene un nivel de optimizado. Luego, de manera similar se calcularon los valores promedio para las métricas PSNR, SSIM y DISTS.

Conjunto de datos	**MSE (óptimo ideal 0)**		**PSNR (óptimo ideal, más alto mejor)**		**SSIM (óptimo ideal 1)**		**DISTS (óptimo ideal 0)**	
	Reducida 4x	Optimizada RODIUCAN	Reducida 4x	Optimizada RODIUCAN	Reducida 4x	Optimizada RODIUCAN	Reducida 4x	Optimizada RODIUCAN
BSDS100	0,0077	0,0066	21,910	22,5220	0,8135	0,8294	0,1965	0,1673
Historicas40	0,0139	0,0094	20,472	21,6230	0,8248	0,8530	0,1769	0,1385
Manga109	0,0157	0,0087	18,769	21,3360	0,8433	0,8778	0,2093	0,1239
Set5	0,0041	0,0024	25,330	26,9300	0,9356	0,9557	0,0748	0,0388
Set14	0,0093	0,0068	21,570	22,7820	0,8482	0,8695	0,1784	0,1291
Urban100	0,0130	0,0097	19,715	21,0870	0,7881	0,8320	0,2293	0,1700

Tabla 4.2: Métricas promedio obtenidas para el modelo propuesto RODIUCAN para distintos conjuntos de imágenes.

En la tabla 4.2 se observó que las mejoras son evidentes con las cuatro métricas: MSE, PSNR, SSIM y DISTS. Las mejoras varían según el tipo de conjunto de datos y el valor óptimo ideal. También se observó que la métrica DISTS en la mayoría de los casos tiene más variaciones.

Porcentaje de mejora

Para las cuatro métricas se tienen dos valores, uno antes de la optimización y otro después, entonces el porcentaje de mejora se calcula como el **porcentaje de cambio**. Esto consiste en restar el valor anterior del nuevo y dividir el resultado por el valor anterior, y expresarlo como un porcentaje.

Para la métrica DISTS, considerando que su valor óptimo es 0, entonces el porcentaje de mejora se determina con la ecuación 28. Donde se observa que la diferencia del numerador siempre debe ser positiva. En el caso ideal si $DISTS_{optimizada}$ fuese cero, entonces el porcentaje de mejora sería del 100%.

$$mejoraDISTS = \frac{DISTS_{reducida\ 4x} - DISTS_{optimizada}}{DISTS_{reducida\ 4x}} * 100 \qquad (28)$$

Del mismo modo para la métrica MSE, considerando que su valor óptimo es 0, entonces el porcentaje de mejora se muestra en la ecuación 29.

$$mejoraMSE = \frac{MSE_{reducida\ 4x} - MSE_{optimizada}}{MSE_{reducida\ 4x}} * 100 \qquad (29)$$

Para la métrica SSIM, y considerando que su valor óptimo es 1, el porcentaje de mejora se muestra en la ecuación 30.

$$mejoraSSIM = \frac{SSIM_{reducida\ 4x} - (1 - SSIM_{optimizada})}{SSIM_{reducida\ 4x}} * 100 \qquad (30)$$

Para la métrica PSNR, y considerando que su valor óptimo debe ser mayor al anterior, el porcentaje de mejora se muestra en la ecuación 31.

$$mejoraPSNR = \frac{PSNR_{optimizada} - PSNR_{reducida\ 4x}}{PSNR_{reducida\ 4x}} * 100 \qquad (31)$$

Aplicando las ecuaciones 28, 29, 30, y 31 a los valores obtenidos en la Tabla 4.2 se calcularon los porcentajes de mejora que se muestran en la Tabla 4.3.

Conjunto de datos	**MSE** Porcentaje de mejora	**PSNR** Porcentaje de mejora	**SSIM** Porcentaje de mejora	**DISTS** Porcentaje de mejora
BSDS100	14,29	2,79	79,03	14,86
Historicas40	32,37	5,62	82,18	21,71
Manga109	44,59	13,68	85,51	40,80
Set5	41,46	6,32	95,27	48,13
Set14	26,88	5,62	84,61	27,63
Urban100	25,38	6,96	78,68	25,86

Tabla 4.3: Porcentajes de mejoras del modelo propuesto RODIUCAN.

Para el modelo ESRGAN se siguió un procedimiento igual, comparando las imágenes reducidas y optimizadas con las cuatro métricas. Para la generación de las imágenes optimizadas con ESRGAN se usó el programa CupScale. Los resultados obtenidos se muestran en la Tabla 4.4.

Conjunto de datos	**MSE (óptimo ideal 0)**		**PSNR (óptimo ideal, más alto mejor)**		**SSIM (óptimo ideal 1)**		**DISTS (óptimo ideal 0)**	
	Reducida 4x	Optimizada ESRGAN	Reducida 4x	Optimizada ESRGAN	Reducida 4x	Optimizada ESRGAN	Reducida 4x	Optimizada ESRGAN
BSDS100	0.0077	0.0072	21.910	22.174	0.8135	0.8032	0.1965	0.1059
Historicas40	0.0139	0.0099	20.472	21.180	0.8248	0.8441	0.1769	0.1142
Manga109	0.0157	0.0067	18.769	22.585	0.8433	0.8983	0.2093	0.0763
Set5	0.0041	0.0029	25.330	26.405	0.9356	0.9472	0.0748	0.0592
Set14	0.0093	0.0068	21.570	22.793	0.8482	0.8528	0.1784	0.0994
Urban100	0.013	0.0103	19.715	20.858	0.7881	0.8216	0.2293	0.1011

Tabla 4.4: Métricas promedio obtenidas para el modelo ESRGAN para distintos conjuntos de imágenes.

Luego los porcentajes de mejoras para el modelo ESRGAN se muestran en la tabla 4.5.

Conjunto de datos	**MSE** Porcentaje de mejora	**PSNR** Porcentaje de mejora	**SSIM** Porcentaje de mejora	**DISTS** Porcentaje de mejora
BSDS100	6.49	1.20	75.81	46.11
Historicas40	28.78	3.46	81.10	35.44
Manga109	57.32	20.33	87.94	63.55
Set5	29.27	4.24	94.36	20.86
Set14	26.88	5.67	82.65	44.28
Urban100	20.77	5.80	77.36	55.91

Tabla 4.5: Porcentaje de mejora del modelo ESRGAN.

CAPÍTULO V

RESULTADOS

Se logró desarrollar un prototipo de aumento resolución de imágenes digitales utilizando autómatas celulares y redes neuronales convolucionales. Para lograrlo se usaron las imágenes generadas por los autómatas celulares como una transformación intermedia entre dos redes neuronales convolucionales.

El modelo propuesto en el presente trabajo obtuvo un nivel de mejora aproximado al modelo ESRGAN y superior en cuatro de los seis conjuntos de datos evaluados, como se muestra en la Tabla 5.1 los valores resaltados indican que el modelo propuesto tuvo un mejor resultado.

Conjunto de datos	**MSE (óptimo ideal 0)**		**PSNR (óptimo ideal, más alto mejor)**		**SSIM (óptimo ideal 1)**		**DISTS (óptimo ideal 0)**	
	Optimizada ESRGAN	Optimizada RODIUCAN	Optimizada ESRGAN	Optimizada RODIUCAN	Optimizada ESRGAN	Optimizada RODIUCAN	Optimizada ESRGAN	Optimizada RODIUCAN
BSDS100	0,0072	**0,0066**	22,174	**22,522**	0,8032	**0,8294**	0,1059	0,1673
Historicas 40	0,0099	**0,0094**	21,180	**21,623**	0,8441	**0,8530**	0,1142	0,1385
Manga109	0,0067	0,0087	22,585	21,336	0,8983	0,8778	0,0763	0,1239
Set5	0,0029	**0,0024**	26,405	**26,930**	0,9472	**0,9557**	0,0592	**0,0388**
Set14	0,0068	0,0068	22,793	22,782	0,8528	**0,8695**	0,0994	0,1291
Urban100	0,0103	**0,0097**	20,858	**21,087**	0,8216	**0,8320**	0,1011	0,1700

Tabla 5.1: Métricas promedio obtenidas para el modelo propuesto RODIUCAN comparado con el modelo ESRGAN.

Para tener una medida porcentual de todas las métricas se utilizó el porcentaje de cambio. Como cada métrica tiene su propia escala no se pueden interpretar todas al 100% de mejora, excepto la métrica SSIM, la cual al tener un valor óptimo de 1 entonces si se puede asociar con un nivel porcentual máximo de 100%. Para todas las métricas estas deben compararse entendiendo que el valor porcentual más alto es el mejor.

Por ejemplo, para el conjunto de datos BSDS100 y la métrica SSIM el modelo propuesto RODIUCAN tuvo mejor desempeño porque obtuvo un 79.03% que es mayor al valor 75.81% del modelo ESRGAN. En la Tabla 5.2 se muestran los mismos resultados de la Tabla 5.1 pero expresados como una mejora porcentual. Los valores resaltados indican que el modelo propuesto tuvo mejores resultados.

	MSE Porcentaje de mejora		**PSNR** Porcentaje de mejora		**SSIM** Porcentaje de mejora		**DISTS** Porcentaje de mejora	
Conjunto de datos	ESRGAN	RODIUCAN	ESRGAN	RODIUCAN	ESRGAN	RODIUCAN	ESRGAN	RODIUCAN
BSDS100	6,49	**14,29**	1,20	**2,79**	75,81	**79,03**	46,11	14,86
Historicas40	28,78	**32,37**	3,46	**5,62**	81,10	**82,18**	35,44	21,71
Manga109	57,32	44,59	20,33	13,68	87,94	85,51	63,55	40,80
Set5	29,27	**41,46**	4,24	**6,32**	94,36	**95,27**	20,86	**48,13**
Set14	26,88	26,88	5,67	5,62	82,65	**84,61**	44,28	27,63
Urban100	20,77	**25,38**	5,80	**6,96**	77,36	**78,68**	55,91	25,86

Tabla 5.2: Comparación porcentual de mejora de los modelos RODIUCAN y ESRGAN.

En la Tabla 5.3 se muestra una comparativa del entorno de entrenamiento del modelo propuesto y ESRGAN respecto a la cantidad de datos, parámetros, y uso de GPU.

Modelo	**Nombre del conjunto de datos.**	**Cantidad de imágenes para el entrenamiento.**	**Cantidad de parámetros del modelo (en millones)**	**Datos del hardware**	**Tiempo entrenamiento**
RODIUCAN (modelo propuesto)	General100	100	9.2 M	Sin aceleración GPU, Pentium I7, 16GB RAM.	28 horas.
ESRGAN	DIV2K, Flickr2K OST	800 2650 300	16.6 M	4 GPU V100	4 a 5 días

Tabla 5.3: Comparación del entorno de entrenamiento de los modelos RODIUCAN y ESRGAN.

Respecto al consumo eléctrico y su correspondiente equivalencia en emisiones de CO_2 se obtuvieron los resultados de la Tabla 5.4.

Modelo	Tiempo de entrenamiento	Consumo eléctrico en kilovatios	Emisión de CO_2 en kilogramos
RODIUCAN (modelo propuesto)	28 horas	8.32	3.6
ESRGAN	4 días	111	47.9

Tabla 5.4: Consumo eléctrico y emisiones de CO_2 equivalentes del modelo propuesto y ESRGAN.

5.1 RESULTADOS SEGÚN LOS OBJETIVOS ESPECÍFICOS

- Se implementaron dos autómatas celulares, el primero para detectar bordes en imágenes en escala de grises con un alto nivel de detalle. Y el segundo autómata para detectar aquello que no es borde, con el objetivo de complementar la información del primer autómata.
- Se realizó el pre-procesamiento del conjunto de datos de imágenes incorporando defectos relacionados con el cambio de resolución: reducción de la resolución en un factor de 0x, 2x, 4x, 6x, y desenfoque. Las imágenes también fueron convertidas a sus versiones en escala de grises.
- Los autómatas celulares fueron aplicados a las imágenes de alta calidad en escala de grises para capturar la información, y luego ambas imágenes fueron combinadas en una sola imagen.
- Se implementaron dos redes neuronales convolucionales con arquitectura U-Net. Con un tamaño óptimo y con parches de imagen de 128x128 pixeles como entradas y salidas.
- Se entrenaron ambas redes neuronales convolucionales de forma separada y combinada y se obtuvieron parches de imágenes de alta calidad.
- Se evaluó el modelo midiendo la similitud de las imágenes estimadas con las imágenes originales. Para ello se usaron las siguientes métricas: MSE, PSNR, SSIM, DISTS. Se consideraron seis diferentes conjuntos de datos típicamente usados en problemas de aumento de resolución de imágenes.
- Se realizó la misma evaluación al modelo ESRGAN. Y se compararon los porcentajes promedio de mejora entre el modelo propuesto y ESRGAN.

5.2 RESULTADOS DE LA HIPÓTESIS

De acuerdo a las hipótesis planteadas:

H1: El uso de un modelo combinado de autómatas celulares y redes neuronales artificiales, permite optimizar la resolución de imágenes digitales.

H2: El uso de un autómata celular permite entrenar una red neuronal con un conjunto de datos reducido y optimizar el uso de recursos computacionales.

Se da respuesta con:

El uso de los autómatas celulares para crear una transformación intermedia entre dos redes neuronales artificiales convolucionales de tipo U-Net, permitió optimizar la resolución de imágenes. Logrando porcentajes de optimización aproximados a ESRGAN y superiores en 4 de los 6 conjuntos evaluados.

La incorporación del autómata permitió reducir el conjunto de datos requerido para el entrenamiento en 100 imágenes, permitiendo que el modelo pueda entrenarse sin GPU en una maquina Pentium I7 con 16GB de RAM. El entrenamiento sin GPU duro en total 28 horas, con un consumo eléctrico de 8.32 kilovatios con un equivalente de 3.6 kilogramos de emisiones de dióxido de carbono.

CAPÍTULO VI

CONCLUSIONES Y RECOMENDACIONES

6.1 CONCLUSIONES

Se logró desarrollar un modelo de aumento resolución de imágenes digitales utilizando las imágenes combinadas de dos autómatas celulares como una transformación intermedia entre dos redes neuronales convolucionales, usando una cantidad óptima de recursos computacionales.

En base a trabajos anteriores sobre autómatas celulares se encontró de forma empírica un conjunto de reglas basadas en el juego de la vida e incorporando una función de umbral para comparar pixeles similares. Se logró que los autómatas detecten con un alto nivel de detalle los bordes y no bordes de una imagen en escala de grises.

Siguiendo el estándar CRISP-DM como metodología se implementaron dos redes neuronales convolucionales con arquitectura U-Net. Luego las imágenes de los autómatas fueron usadas como una transformación intermedia entre las dos redes, para obtener un modelo de aumento de resolución.

En las evaluaciones con seis diferentes conjuntos de datos se verifico que el modelo propuesto, optimiza las imágenes de baja calidad principalmente respecto a la reconstrucción de bordes, pero que el porcentaje de mejora varía según el conjunto de datos de prueba. El modelo propuesto tiene un mayor porcentaje de mejora comparado al modelo ESRGAN en los conjuntos de datos: BSDS100, Historicas40, Set5 y Urban100. Esta mejora se hace evidente con las métricas MSE, PSNR y SSIM, pero no con la métrica DISTS. Como la métrica DISTS está más especializada en texturas siempre da un mayor porcentaje de mejora a ESRGAN.

El modelo propuesto al usar las imágenes generadas por el autómata es lo suficientemente eficiente como para poder ser entrenado con un conjunto reducido de datos de 100 imágenes y es factible su entrenamiento sin aceleración GPU.

Por otro lado, el tipo imágenes de los conjuntos de datos es muy significativo, tanto en el entrenamiento como en la validación. En particular para la optimización de imágenes con defectos ocasionados por la reducción de resolución se obtuvieron los mejores resultados usando un conjunto de datos de imágenes en primer plano donde se destaquen claramente los bordes.

Se evidencio que el tamaño del conjunto de datos es proporcional a la cantidad de recursos computacionales y uso intensivo de GPU. Con lo cual el modelo propuesto facilito el entrenamiento al usar un conjunto de datos de 100 imágenes.

También se observó que el modelo ESRGAN tiende a agregar algunos detalles ficticios que no siempre suelen coincidir con los detalles de las imágenes originales, pero que le favorecen respecto a la reconstrucción de texturas. El modelo propuesto en cambio logra reconstruir los bordes defectuosos de forma eficiente y reconstruye las texturas sin alejarse mucho de las imágenes originales esperadas.

El modelo propuesto tiene un menor rendimiento respecto a la reconstrucción de detalles de rostros humanos debido a que en el conjunto de datos de entrenamiento utilizado no cuenta con muchas imágenes de este tipo. Relacionado a este problema, en versiones más especializadas de ESRGAN se utilizaron conjuntos de datos adicionales de rostros como FFHQ para mejorar su rendimiento en ese caso particular.

Respecto a la estrategia de entrenamiento, se evidencio que separando el entrenamiento en fases se consiguen los mejores resultados, respecto al tiempo y uso de recursos.

6.2 RECOMENDACIONES

El modelo presentado podría mejorarse utilizando un conjunto de datos un poco más grande o desarrollando uno sintético basado en bordes o texturas.

Se recomienda investigar más sobre el uso de transformaciones intermedias entre las dos o más redes convolucionales, pudiendo analizar la construcción de nuevos autómatas celulares especializados en detección de texturas y patrones de iluminación. De esta forma se podría mejorar la calidad de la transformación intermedia con el objetivo de mejorar la optimización de las imágenes.

Las imágenes producidas por la primera red neuronal podrían ser usadas como parte de otro modelo para la segmentación y clasificación de objetos en imágenes.

Se puede usar el modelo propuesto con otros conjuntos de datos más específicos, como por ejemplo conjuntos de datos de solo de rostros, animales, etc. Con el fin de obtener mejores resultados respecto a la reconstrucción de texturas.

Finalmente se podría combinar el modelo presentado con otros modelos de aumento de resolución o incorporar alguna estrategia de transferencia de conocimiento. Para de esta forma aprovechar los beneficios del uso de la transformación intermedia basado en las imágenes generadas por el autómata celular.

REFERENCIAS

Adobe. (17 de 01 de 2022). *Profundidad de bits.* Obtenido de https://helpx.adobe.com/es/photoshop/using/bit-depth.html

Adobe. (29 de 04 de 2022). *Tamaño y resolución de imágenes.* Obtenido de https://helpx.adobe.com/es/photoshop/using/image-size-resolution.html

Amat Rodrigo, J. (11 de 2020). *Paralelizar bucle for con Python*. Obtenido de https://www.cienciadedatos.net/documentos/py12-paralelizar-con-python.html

Amazon AWS. (2022). *¿Qué es Python?* Obtenido de https://aws.amazon.com/es/what-is/python/

Angulo Sogamoso, K., & Gil Sierra, D. G. (2019). *DESARROLLO DE UN ALGORITMO ADAPTATIVO BASADO EN AUTÓMATAS CELULARES PARA ELIMINAR RUIDO EN IMÁGENES DIGITALES.* Obtenido de https://repository.udistrital.edu.co/bitstream/handle/11349/16324/GilSierraDaniloGustavo2019.pdf?sequence=1&isAllowed=y

Calvo, D. (8 de 12 de 2018). *Perceptrón Multicapa – Red Neuronal*. Obtenido de https://www.diegocalvo.es/perceptron-multicapa/

DeepAI. (07 de 29 de 2020). *BSD100 - 4x upscaling Dataset*. Obtenido de https://deepai.org/dataset/bsd100-4x-upscaling

Delgado, C. (17 de 7 de 2019). *Cómo calcular el índice de similitud estructural (SSIM) entre dos imágenes con Python*. Obtenido de https://ourcodeworld.co/articulos/leer/991/como-calcular-el-indice-de-similitud-estructural-ssim-entre-dos-imagenes-con-python

Diaz, B. (4 de 5 de 2022). *Los modelos de difusión claramente explicados*. Obtenido de https://impulsatek.com/los-modelos-de-difusion-claramente-explicados/

Ding, K., Ma, K., Wang, S., & Simoncelli, E. P. (22 de 5 de 2020). *Deep Image Structure and Texture Similarity (DISTS) Metric*. Obtenido de Repository of paper Image Quality Assessment: Unifying Structure and Texture Similarity: https://github.com/dingkeyan93/DISTS

Ding, K., Ma, K., Wang, S., & Simoncelli, E. P. (16 de 4 de 2020). *Image Quality Assessment: Unifying Structure and Texture Similarity*. Obtenido de https://arxiv.org/abs/2004.07728

Fiol M., E. (06 de 05 de 2021). *COLUMNA REFERENTES: AUTÓMATAS CELULARES, MÁS ALLÁ DEL JUEGO DE LA VIDA.* Obtenido de https://pmi.cl/web/2021/05/06/automatas-celulares-mas-alla-del-juego-de-la-vida/

García Mateos, G. (2017). *Procesamiento de imágenes*. Obtenido de https://dis.um.es/profesores/ginesgm/pi.html

García, M. (5 de 12 de 2021). *Clasificación automática del grado general de disfonía*. Obtenido de https://www.researchgate.net/publication/356839369_Clasificacion_automatica_del_grado_general_de_disfonia#pf26

Gomar, J. (7 de 8 de 2017). *¿Cuánto consume realmente nuestro ordenador? | Fuente de alimentación recomendada*. Obtenido de https://www.profesionalreview.com/2017/08/07/cuanto-consume-realmente-nuestro-ordenador/

González Alcón, C. (23 de 12 de 2020). *50 AÑOS DE VIDA: EL AUTÓMATA CELULAR DE CONWAY.* Obtenido de https://www.ull.es/portal/cienciaull/50-anos-de-vida-el-automata-celular-de-conway/

Google. (2022). *Colaboratory*. Obtenido de Preguntas frecuentes: https://research.google.com/colaboratory/intl/es/faq.html

Greyrat, R. (5 de 7 de 2022). *Python OpenCV: interpolación bicúbica para cambiar el tamaño de la imagen*. Obtenido de https://barcelonageeks.com/python-opencv-interpolacion-bicubica-para-cambiar-el-tamano-de-la-imagen/

Grillo, S. A. (22 de 9 de 2021). *¿Puede explicarse el universo por medio de la computación?* Obtenido de https://cienciasdelsur.com/2021/09/22/puede-explicarse-el-universo-por-medio-de-la-computacion/

Hao, K. (6 de 6 de 2019). *Training a single AI model can emit as much carbon as five cars in their lifetimes*. Obtenido de https://www.technologyreview.com/2019/06/06/239031/training-a-single-ai-model-can-emit-as-much-carbon-as-five-cars-in-their-lifetimes/

He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Obtenido de https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

Hoz Galiana, D. (2020). *Construcción de Arquitecturas Neuronales Profundas utilizando Programación Genética Guiada por Gramáticas y Algoritmos de Estimación de Distribución*. Obtenido de https://oa.upm.es/63690/

Hugging Face. (8 de 12 de 2022). *Hugging Face Hub documentation*. Obtenido de https://huggingface.co/docs/hub/main/en/index

IBM. (2022). *¿Qué es la Visión Artificial?* Obtenido de https://www.ibm.com/ar-es/topics/computer-vision

Lamba, H. (17 de 2 de 2019). *Understanding Semantic Segmentation with UNET*. Obtenido de https://towardsdatascience.com/understanding-semantic-segmentation-with-unet-6be4f42d4b47

León Batallas, A., Bermeo Paucar, J., Paredes Quevedo, J., & Torres Ordoñez, H. (2020). Una revisión de las métricas aplicadas en el procesamiento de imágenes. *RECIMUNDO - Revista Científica Mundo de la Investigación y Conocimiento*,

267-273. Obtenido de https://reciamuc.com/~recimund/index.php/es/article/download/874/1372

López Franco, C. (2022). *¿Cómo convertir una imagen RGB a escala de grises?* Obtenido de https://visionyrobotica.com/convertir-imagen-rgb-escala-de-grises/

Martinez Heras, J. (19 de 9 de 2020). *Las 7 Fases del Proceso de Machine Learning*. Obtenido de https://www.iartificial.net/fases-del-proceso-de-machine-learning/

Martinez Heras, J. (19 de 10 de 2020). *Redes Neuronales Generativas Adversarias (GANs)*. Obtenido de https://www.iartificial.net/redes-neuronales-generativas-adversarias-gans/

Matplotlib. (2 de 5 de 2022). *Matplotlib: Visualization with Python*. Obtenido de https://matplotlib.org/

Mehmood, S. (21 de 6 de 2022). *Filtros de desenfoque en OpenCV*. Obtenido de https://www.delftstack.com/es/howto/python/opencv-blur/

NumPy. (2022). *NumPy documentation*. Obtenido de https://numpy.org/doc/stable/

OpenCV. (2022). *Image Processing in OpenCV*. Obtenido de https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html

OpenCV. (2022). *Open Source Computer Vision*. Obtenido de https://docs.opencv.org/4.x/d1/dfb/intro.html

Parra, J., & Perozo, N. (1 de 1 de 2017). *Un metodo cooperativo para la segmentación de imágenes basado en autómatas celulares*. Obtenido de https://www.redalyc.org/pdf/707/70750544008.pdf

Pereira, J. (25 de 2 de 2020). *Similitud entre imágenes, análisis forense de imagen y control de calidad*. Obtenido de http://www.jpereira.net/gestion-de-color-articulos/similitud-entre-imagenes-analisis-forense-de-imagen-y-control-de-calidad

Perez Alonso, D. M. (2020). *Superresolution*. Obtenido de https://uvadoc.uva.es/bitstream/handle/10324/44117/TFG-G4667.pdf?sequence=1

Pérez, P., & Valente, M. (2018). *Fundamentos básicos del procesamiento de imágenes.* Obtenido de https://www.famaf.unc.edu.ar/~pperez1/manuales/cim/cap2.html

Phillip, I., Jun-Yan, Z., Tinghui, Z., & Alexei A, E. (26 de 11 de 2018). *Image-to-Image Translation with Conditional Adversarial Networks*. Obtenido de https://arxiv.org/abs/1611.07004

Prieto Renieblas, G., Turrero Nogués, A., Muñoz González, A., Gómez Leon, N., & Guibelalde del Castillo, E. (26 de 7 de 2017). *Structural similarity index family for image quality assessment in radiological images*. Obtenido de https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5527267/

Puga, B., Ramírez Noriega, A., & Soto, J. (7 de 2021). *Un algoritmo de detección de bordes basado en reglas deterministas*. Obtenido de https://www.researchgate.net/publication/353105130_Un_algoritmo_de_deteccion_de_bordes_basado_en_reglas_deterministas

Querejeta Simbeni, P. (7 de 2015). *Procesamiento digital de imágenes*. Obtenido de http://lcr.uns.edu.ar/fvc/NotasDeAplicacion/FVC-QuerejetaSimbeniPedro.pdf

Recuero de los Santos, P. (2 de 12 de 2021). *Tipos de aprendizaje en Machine Learning: supervisado y no supervisado*. Obtenido de https://empresas.blogthinkbig.com/que-algoritmo-elegir-en-ml-aprendizaje/

Rodríguez Abril, R. (2022). *Redes Residuales: ResNet*. Obtenido de https://lamaquinaoraculo.com/computacion/redes-residuales/

Rodríguez Abril, R. (2022). *Segmentación semántica y U-Net*. Obtenido de https://lamaquinaoraculo.com/computacion/u-net-y-segmentacion-semantica/

Rodríguez Calvo, S. (30 de 05 de 2019). *Introducción a los Autómatas Celulares.* Obtenido de https://serrodcal.medium.com/introducci%C3%B3n-a-los-aut%C3%B3matas-celulares-3c344110d9b8

Rodríguez Hernández, L. J., Ochoa Domínguez, H., Vergara Villegas, O. O., & Sossa Azuela, J. H. (2020). *Estudio comparativo de algoritmos de súper resolución.* Obtenido de https://rcs.cic.ipn.mx/2020_149_8/Estudio%20comparativo%20de%20algoritmos%20de%20super%20resolucion%20de%20una%20sola%20imagen.pdf

Rodríguez Rodríguez, R. (1 de Marzo de 2016). *Super-resolucion usando una única imagen.* Obtenido de https://cimat.repositorioinstitucional.mx/jspui/bitstream/1008/487/1/TE_1542.pdf

Rodríguez, V. (9 de 11 de 2018). *Dropout y Batch Normalization*. Obtenido de https://vincentblog.xyz/posts/dropout-y-batch-normalization

Roeder, L. (12 de 9 de 2022). *Netron*. Obtenido de https://github.com/lutzroeder/netron

Ronneberger, O., Fischer, P., & Brox, T. (18 de 5 de 2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. Obtenido de https://arxiv.org/abs/1505.04597

Rozet, F. (10 de 9 de 2022). *PyTorch Image Quality Assessment*. Obtenido de https://github.com/francois-rozet/piqa

Salgueiro, L., Marcello, J., & Vilaplana, V. (19 de 11 de 2022). *SEG-ESRGAN: A Multi-Task Network for Super-Resolution*. Obtenido de https://www.mdpi.com/2072-4292/14/22/5862

Sancho Caparrini, F. (2 de 3 de 2020). *Variational AutoEncoder*. Obtenido de http://www.cs.us.es/~fsancho/?e=232

scikit-image team. (6 de 2022). *Structural similarity index*. Obtenido de https://scikit-image.org/docs/dev/auto_examples/transform/plot_ssim.html

Sharma, A. (26 de 4 de 2021). *Variational Autoencoder in TensorFlow*. Obtenido de https://learnopencv.com/variational-autoencoder-in-tensorflow/

Strubell, E., Ganesh, A., & McCallum, A. (5 de 6 de 2019). *Energy and Policy Considerations for Deep Learning in NLP*. Obtenido de https://arxiv.org/abs/1906.02243

Suárez, P., & Villavicencio, M. (24 de 02 de 2017). *Detección de Contornos utilizando el Algoritmo Canny en Imágenes Cross-Espectrales Fusionadas*. Obtenido de Canny Edge Detection in Cross-Spectral Fused Images: https://www.redalyc.org/journal/5722/572262176002/html/

Super-Resolution, C. D. (22 de 9 de 2019). *Common Datasets for Image Super-Resolution*. Obtenido de https://cvnote.ddlee.cc/2019/09/22/image-super-resolution-datasets

TensorFlow. (26 de 1 de 2022). *pix2pix: traducción de imagen a imagen con una GAN condicional*. Obtenido de https://www.tensorflow.org/tutorials/generative/pix2pix

TensorFlow. (2022). *Plataforma de extremo a extremo de código abierto para el aprendizaje automático*. Obtenido de https://www.tensorflow.org/?hl=es-419

Unión Internacional de Telecomunicaciones. (3 de 2011). *Recomendación UIT-R BT.601-7*. Obtenido de https://www.itu.int/dms_pubrec/itu-r/rec/bt/R-REC-BT.601-7-201103-I!!PDF-S.pdf

Unipython. (5 de 4 de 2018). *Algoritmo de Canny*. Obtenido de https://unipython.com/algoritmo-de-canny/

Vallalta Rueda, J. F. (2022). *Aprendizaje supervisado y no supervisado*. Obtenido de https://healthdataminer.com/data-mining/aprendizaje-supervisado-y-no-supervisado/

Wang, C. (12 de 02 de 2022). *Super Resolution Dataset*. Obtenido de https://github.com/ChaofWang/Awesome-Super-Resolution/blob/master/dataset.md

WANG, Z., CHEN, J., & C. H. HOI, S. (10 de 2021). *Deep learning for image super-resolution: A survey*. Obtenido de https://ink.library.smu.edu.sg/cgi/viewcontent.cgi?article=7961&context=sis_research

Wikipedia. (06 de 06 de 2022). *Autómata celular.* Obtenido de https://es.wikipedia.org/wiki/Aut%C3%B3mata_celular

Wikpedia. (2022). *Red neuronal artificial.* Obtenido de https://es.wikipedia.org/wiki/Red_neuronal_artificial

Xintao, W. (27 de 7 de 2021). *ESRGAN (Enhanced SRGAN)*. Obtenido de https://github.com/xinntao/ESRGAN

Xintao, W. (17 de 8 de 2021). *Real-ESRGAN*. Obtenido de Training time: https://github.com/xinntao/Real-ESRGAN/issues/38

Xintao, W., Ke, Y., Shixiang, W., Jinjin, G., Yihao, L., Chao, D., . . . Xiaoou, T. (17 de 9 de 2018). *ESRGAN: Enhanced Super-Resolution*. Obtenido de https://arxiv.org/pdf/1809.00219.pdf

Xintao, W., Liangbin, X., Chao, D., & Ying, S. (17 de 8 de 2021). *Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data*. Obtenido de https://arxiv.org/pdf/2107.10833.pdf

ANEXOS

ANEXO A. FRAGMENTO DE CÓDIGO DEL AUTÓMATA CELULAR PARA DETECTAR BORDES.

```
#Aplica el autómata para detectar bordes
#con un umbral de distancia de intensidad de 3
def
aplicar_automata6var(xposini,xposfin,xcarpeta_img,xnom_img,xcarpeta_map
img,xnom_mapimg,xcarpeta_secuenciaimg,xnom_secimg,maxsec):
    for i in range(xposini,xposfin):
        egrises =
cv2.imread(xcarpeta_img+'/'+getNombreArchivo(i)+xnom_img+'.png')
        egrises = cv2.cvtColor(egrises, cv2.COLOR_BGR2GRAY)

        original_nuevoy=int(egrises.shape[0])
        original_nuevox=int(egrises.shape[1])
        egrises = cv2.resize(egrises,
dsize=(original_nuevox*2,original_nuevoy*2),
interpolation=cv2.INTER_CUBIC)

        nuevoy=int(egrises.shape[0])
        nuevox=int(egrises.shape[1])
        #automata crear estructuras
        matrizI=np.copy(egrises)
        matrizA=np.copy(egrises)
        for y in range(0,nuevoy):
            for x in range(0,nuevox):
                if y % 2 == 0 or x % 2 == 0:
                    matrizA[y,x]=1*255
                else:
                    matrizA[y,x]=0*255
        matrizI_AB=ampliarBordesImagen(matrizI, 1)
        #automata iterar maxsec veces
        for ra in range(0,maxsec):

plt.imsave(xcarpeta_secuenciaimg+'/'+getNombreArchivo(i)+xnom_secimg+'_
'+str(ra)+'.png',matrizA,cmap='gray',vmin=0, vmax=255)
            cop_matrizA=np.copy(matrizA)
            matrizA_AB=ampliarBordesImagen(matrizA, 1)
            for y in range(0,nuevoy):
                for x in range(0,nuevox):

numsimilares=contarVecinasSimilares(matrizI_AB,nuevoy,nuevox,y,x,3)

numvivas=contarVecinasVivas(matrizA_AB,nuevoy,nuevox,y,x)+numsimilares
                    if(matrizA[y,x]==VG_VIVO):
                        if numvivas<2:
                            if numsimilares>3:
                                cop_matrizA[y,x]=VG_MUERTO
                        else:
                            if(numvivas==2 or numvivas==3):
                                pass
                            else:
                                if numsimilares>3:
                                    cop_matrizA[y,x]=VG_MUERTO
```

```
                    else:
                        if numvivas==3:
                            if numsimilares<=3:
                                cop_matrizA[y,x]=VG_VIVO
            matrizA=np.copy(cop_matrizA)
        #guardar imagen

plt.imsave(xcarpeta_mapimg+'/'+getNombreArchivo(i)+xnom_mapimg+'.png',m
atrizA,cmap='gray',vmin=0, vmax=255)
```

ANEXO B. FRAGMENTO DE CÓDIGO DE LA ESTRUCTURA DE LAS REDES NEURONALES.

```
import tensorflow as tf
from tensorflow.keras import layers, losses
from tensorflow.keras.models import Model

def
capa_codificador(capa_entrada,num_filtros,aplicar_batchnormalization,re
ducir):
  x=tf.keras.layers.Conv2D(num_filtros, kernel_size=(4,4),
padding='same', strides=reducir, use_bias=False)(capa_entrada)
  if(aplicar_batchnormalization==True):
      x=tf.keras.layers.BatchNormalization()(x)
  x=tf.keras.layers.LeakyReLU()(x)
  return x

def capa_decodificador(capa_entrada,num_filtros,skip_conection,
aplicar_drop_out,aplicar_batchnormalization,ampliar):
  x=tf.keras.layers.Conv2DTranspose(num_filtros, kernel_size=4,
strides=ampliar, padding='same', use_bias=False)(capa_entrada)
  if(aplicar_batchnormalization==True):
      x=tf.keras.layers.BatchNormalization()(x)
  if(aplicar_drop_out==True):
      x=tf.keras.layers.Dropout(0.3)(x)
  x=tf.keras.layers.ReLU()(x)
  x=tf.keras.layers.Concatenate()([x,skip_conection])
  return x

def funcion_de_perdida_red1(val_verdadero, val_predicho):
    #Error cuadrático medio
    return
tf.reduce_mean(tf.square(tf.subtract(val_verdadero,val_predicho)))

def funcion_de_perdida_red2(val_verdadero, val_predicho):
    return
tf.reduce_mean(tf.square(tf.subtract(val_verdadero,val_predicho)))

def funcion_de_perdida_red_combinada(val_verdadero, val_predicho):
    return
tf.reduce_mean(tf.square(tf.subtract(val_verdadero,val_predicho)))

def RedAutoencoderEB():
    e0=layers.Input(shape=(128, 128,1))

    e1 =
capa_codificador(e0,72,aplicar_batchnormalization=False,reducir=2)#Ent
128x128, Sal 64x64
    e2 =
capa_codificador(e1,80,aplicar_batchnormalization=True,reducir=2)#Ent
64x64, Sal 32x32
    e3 =
capa_codificador(e2,96,aplicar_batchnormalization=True,reducir=2) #Ent
32x32, Sal 16x16
```

```
    e4 =
capa_codificador(e3,112,aplicar_batchnormalization=True,reducir=2)#Ent
16x16, Sal 8x8
    e5 =
capa_codificador(e4,128,aplicar_batchnormalization=True,reducir=2)#Ent
8x8, Sal 4x4
    e6 =
capa_codificador(e5,160,aplicar_batchnormalization=True,reducir=2)#Ent
4x4, Sal 2x2
    e7 =
capa_codificador(e6,192,aplicar_batchnormalization=True,reducir=2)#Ent
2x2, Sal 1x1

    e8 =
capa_decodificador(e7,192,e6,aplicar_drop_out=True,aplicar_batchnormali
zation=True,ampliar=2)#Ent 1x1, Sal 2x2
    e9 =
capa_decodificador(e8,160,e5,aplicar_drop_out=True,aplicar_batchnormali
zation=True,ampliar=2)#Ent 2x2, Sal 4x4
    e10 =
capa_decodificador(e9,128,e4,aplicar_drop_out=True,aplicar_batchnormali
zation=True,ampliar=2)#Ent 4x4, Sal 8x8
    e11 =
capa_decodificador(e10,112,e3,aplicar_drop_out=False,aplicar_batchnorma
lization=True,ampliar=2)#Ent 8x8, Sal 16x16
    e12 =
capa_decodificador(e11,96,e2,aplicar_drop_out=False,aplicar_batchnormal
ization=True,ampliar=2)#Ent 16x16, Sal 32x32
    e13 =
capa_decodificador(e12,80,e1,aplicar_drop_out=False,aplicar_batchnormal
ization=True,ampliar=2)#Ent 32x32, Sal 64x64
    e14=layers.Conv2DTranspose(72, kernel_size=4, strides=2,
activation='relu', padding='same')(e13)#Ent 64x64, Sal 128x128
    ef=layers.Conv2D(1, kernel_size=(4, 4), activation='tanh',
padding='same')(e14)

    return Model(e0,ef)

def RedAutoencoder():
    e0=layers.Input(shape=(128, 128,2))

    e1 =
capa_codificador(e0,72,aplicar_batchnormalization=False,reducir=2)
    e2 =
capa_codificador(e1,80,aplicar_batchnormalization=True,reducir=2)
    e3 =
capa_codificador(e2,96,aplicar_batchnormalization=True,reducir=2)
    e4 =
capa_codificador(e3,112,aplicar_batchnormalization=True,reducir=2)
    e5 =
capa_codificador(e4,128,aplicar_batchnormalization=True,reducir=2)
    e6 =
capa_codificador(e5,160,aplicar_batchnormalization=True,reducir=2)
    e7 =
capa_codificador(e6,192,aplicar_batchnormalization=True,reducir=2)
```

```
    e8 =
capa_decodificador(e7,192,e6,aplicar_drop_out=True,aplicar_batchnormali
zation=True,ampliar=2)
    e9 =
capa_decodificador(e8,160,e5,aplicar_drop_out=True,aplicar_batchnormali
zation=True,ampliar=2)
    e10 =
capa_decodificador(e9,128,e4,aplicar_drop_out=True,aplicar_batchnormali
zation=True,ampliar=2)
    e11 =
capa_decodificador(e10,112,e3,aplicar_drop_out=False,aplicar_batchnorma
lization=True,ampliar=2)
    e12 =
capa_decodificador(e11,96,e2,aplicar_drop_out=False,aplicar_batchnormal
ization=True,ampliar=2)
    e13 =
capa_decodificador(e12,80,e1,aplicar_drop_out=False,aplicar_batchnormal
ization=True,ampliar=2)
    e14=layers.Conv2DTranspose(72, kernel_size=4, strides=2,
activation='relu', padding='same')(e13)
    ef=layers.Conv2D(1, kernel_size=(4, 4), activation='tanh',
padding='same')(e14)

    return Model(e0,ef)

redautoencoder_eb=RedAutoencoderEB()
redautoencoder=RedAutoencoder()
redautoencoder_eb.compile(optimizer=tf.keras.optimizers.legacy.Adam(lea
rning_rate=0.001), loss=funcion_de_perdida_red1)
redautoencoder.compile(optimizer=tf.keras.optimizers.legacy.Adam(learni
ng_rate=0.001), loss=funcion_de_perdida_red2)


def RedCombinada(g_model,d_model):
    for layer in d_model.layers:
        if not isinstance(layer, tf.keras.layers.BatchNormalization):
            layer.trainable = False
    e0 = layers.Input(shape=(128, 128,1))
    salida_red1 = g_model(e0)
    union=tf.concat([e0,salida_red1],-1)
    salida_d = d_model(union)
    return Model(e0, [salida_red1,salida_d])

redcombinada = RedCombinada(redautoencoder_eb, redautoencoder)
redcombinada.compile(optimizer=tf.keras.optimizers.legacy.Adam(learning
_rate=0.001), loss=funcion_de_perdida_red_combinada)
```

DOCUMENTACIÓN

La Paz 14 de Marzo, 2023

Señor.
Ph D. José Maria Tapia Baltazar
DIRECTOR CARRERA DE INFORMÁTICA
FAC. CIENCIAS PURAS Y NATURALES
UNIVERSIDAD MAYOR DE SAN ANDRÉS
Presente

Ref. AVAL DE CONFORMIDAD DEL PERFIL DE TESIS

Señor Director:

Tengo a bien hacerle conocer que el Univ. **Rolando Quispe Mamani**, con C.I. **4886089 LP** ha desarrollado la propuesta de investigación en la modalidad de tesis con el título "**OPTIMIZACIÓN DE RESOLUCIÓN DE IMÁGENES DIGITALES USANDO AUTÓMATAS CELULARES Y REDES NEURONALES ARTIFICIALES**", llegando a la versión final del trabajo.

En mi calidad de tutor y, habiendo verificado el cumplimiento de los objetivos trazados, me corresponde **dar mi conformidad y/o Aval** para su respectiva aprobación y posterior emisión de la resolución respectiva. En este sentido, el estudiante debe proceder con los trámites requeridos para dar paso a la lectura y defensa pública de la tesis mencionada.

Sin otro particular me despido con las consideraciones más distinguidas.

Atentamente,

MSc Rosa Flores M.
Tutor de Tesis